

جامعة طرابلس

كلية العلوم – قسم علوم الحاسب الآلي
المرحلة التمهيدية للعلوم التطبيقية

أساسيات الحاسوب والبرمجة
CS011

Introduction to Computer & programming
Using Python

الفصل الدراسي خريف 2022/2023

الحاسب الآلي Computer

تعريف الحاسب الآلي

هو مجموعة من الوحدات المتصلة مع بعضها البعض والتي تقوم على استقبال البيانات كمدخلات من خلال وحدات الإدخال و تخزينها ومن ثم تقوم بمعالجتها بتنفيذ العمليات الحسابية و المنطقية عليها وإخراجها على شكل معلومات عن طريق وحدات الإخراج.

أجيال الحاسب الآلي Computer Generation

مرت صناعة الحاسب الآلي بمراحل تطور متعددة وقد أطلق على كل هذه المراحل (جيل) بناء التطور التكنولوجي المتبع في صناعات الحاسب الآلي حتى وصلت الى الجيل الخامس.

الجيل الأول

أستخدم في تصميم هذا الجيل من الحاسبات ما يعرف بالصمامات المفرغة (Vacuum Tubes) حيث صنع جهاز الحاسب الآلي كبير الحجم والذي امتاز بالتكلفة العالية وبطء السرعة مع استهلاك عالي للطاقة الكهربائية مما أدى الى ارتفاع درجة الحرارة حيث أستخدمت وحدات التكييف لتبريدها

الجيل الثاني

أستخدم في تصميم هذا الجيل الترانزستور بدلاً من الصمامات المفرغة مما أدى الى تقليل حجم و وزن الجهاز عما كان عليه ولكن زادت سرعة تنفيذ العمليات مع تقليل استهلاك الطاقة الكهربائية وبالتالي أنخفضت درجة الحرارة. استخدم في هذا الجيل لغات برمجة ذات المستوى العالي بدلاً من اللغات الرمزية أو لغة الآلة في برمجة الحواسيب، حيث أن المختصون تمكنوا من برمجته بلغتين Cobol ، Fortran

الجيل الثالث

تطورت الأجهزة في هذا الجيل وتميز هذا الجيل باستخدام الدوائر الكهربائية المتكاملة (IC) المصنوعة من رقائق السيلكون وهي عبارة عن مواد شبه موصلة أقل حجم من الترانزستور ، وبذلك أصبح الحاسوب أصغر حجمًا مما سبق وأقل تكلفة.

الجيل الرابع

هي بداية ظهور أجهزة الحاسوب الدقيقة Micro Computer ، استخدم هذا الجيل الدوائر المتكاملة واستخدم الشرائح والمعالجات الدقيقة ، حيث أدخلت تعديلات من حيث نظم التشغيل ونقل البيانات ووحدات الإدخال والإخراج والقدرة على التخزين وسرعة استرجاع المعلومات وظهر في هذا الجيل الحاسب الآلي الشخصي PC

الجيل الخامس

بدأت في هذه المرحلة ظهور أجهزة الحاسوب المحمولة والحاسوب بحجم الكف ، كما انتشرت الهواتف الذكية والجهاز الإلكتروني الأخرى ، كما تميز هذا الجيل ببداية عصر الذكاء الاصطناعي (Artificial Inteligent) لإنتاج حاسبات ذكية تحاكي قدرات الإنسان العقلية مع التطور في شبكات الحاسوب والزيادة الهائلة في السرعة مع زيادة القدرات التخزينية الضخمة.

أهم مميزات الحاسب الآلي

- السرعة العالية : القدرة على إجراء حسابات معقدة على كميات كبيرة من البيانات خلال أجزاء قليلة من الثانية.
- الدقة الفائقة : يعطينا إجابات دقيقة وصحيحة من خلال قيامه بحسابات معقدة بدقة فائقة بنسبة 100.1%
- التخزين : له القدرة على تخزين كميات هائلة من البيانات والرجوع لها متى يحتاج لها
- الاتصال : توصيل جهاز الحاسوب بالشبكات الحاسوبية يعطيه مزايا إضافية يمكن استغلاله بشكل أكبر وأوسع.

مكونات جهاز الحاسب الآلي Computer Components

يتكون جهاز الحاسب الآلي من المكونات المادية (Hardware) و المكونات البرمجية (Software)

المكونات المادية (Hardware)

ان المكونات المادية لجهاز الحاسب الآلي تعني كافة الاجزاء و الأجهزة المادية التي يتطلبها أي جهاز إلكتروني، ليشمل كافة العناصر المتواجدة بلوحة الدائرة الكهربائية، بما في ذلك بطاقات الرسومات ووحدة المعالجة المركزية، اللوحة الأم ومراوح التهوية ومصدر الطاقة وكاميرا الويب الى اخره..

المكونات البرمجية (Software)

وتشمل المكونات البرمجية كل البرامج المستخدمة بما في ذلك أنظمة تشغيل أجهزة الحاسب الآلي والتي تعمل بمكونات تشغيل مختلفة ، لكن رغم من ذل كافة أجهزة الحاسب الآلي تحمل بداخلها مكونات رئيسية والتي لا تختلف مهما تنوعت برامجها الافتراضية او بما يسمى “السوفت وير” الذي يتم تشغيلها على جهاز الحاسوب

دورة معالجة البيانات Data Processing Cycle

تلعب البيانات دورًا في غاية الأهمية في كافة أنواع الأنظمة، حيث تشكل المادة الخام المستخدمة في تصنيع المعلومات بعد الخضوع للمعالجة، وحتى يتم معالجة البيانات لتخرج إلينا على هيئة معلومات ذات فائدة؛ لا بد من تتوفر نظام معالجة خاص يؤدي هذا الغرض فظهر نظام

معالجة البيانات في الحاسوب.



مكونات نظام معالجة البيانات في الحاسوب

يتألف نظام معالجة البيانات في الحاسوب من مجموعة من المكونات الثابتة، وهي:

- وحدة المعالجة المركزية (Central Processing Unit)، ويرمز لها اختصارًا بـ CPU ، تؤدي دور المفسر الأول والدقيق للبيانات والمعلومات والمعالج لها.
- الذاكرة (main memory) وتستخدم عادةً لتخزين البيانات المعالجة والأولية فيها، إلا أنها تفقد كل المعلومات فور انقطاع التيار الكهربائي أو إغلاق جهاز الحاسوب، إذ تعرف بأنها عشوائية.
- وحدات الإدخال والإخراج: يمكن القول بأنها الوسيلة التي يقوم الإنسان بواسطتها إدخال البيانات المراد معالجتها إلى جهاز الحاسوب؛ ثم يصار إلى معالجتها واستعراضها على وحدات الإخراج كالشاشة مثلًا في صورة المعلومات المفيدة.
- وحدة الحساب والمنطق: وهي تلك الوحدة المستخدمة في تخزين البيانات فيها على هيئة أعداد بعد خضوعها للعمليات المنطقية.
- العنصر البشري: يكمن دور العنصر البشري سواء كان مُصنّع أو مبرمج بأنه معطي الأوامر ومدخل البيانات ليتم معالجتها.

لغات البرمجة – Programming Languages

لغات برمجة الحاسوب هو مفهوم واسع كبير يحتوي على أسماء عدة شتى قد يختار فيها الداخل الجديد إلى مجال علوم الحاسوب لتعلم البرمجة، فقد يتساءل أي لغة برمجة يجب أن أتعلّم أولاً، وما هي أشهر لغة برمجة تُستعمل على نطاق واسع استفيد منها، وما هي أسهل لغة برمجة ابدأ بها وغيرها من الأسئلة المحيرة، فإن فتحت مثلاً قائمة لغات البرمجة على الانترنت فستجد عشرات لغات البرمجة المذكورة وهو ما يزيد المشكلة

فهناك العديد من لغات البرمجة وأقسامها واستخداماتها سواء لبرمجة الحواسيب أو الأجهزة الذكية أو الأجهزة الإلكترونية وحتى الروبوتات لذلك لغات البرمجة عموماً تكون ركيزة صلبة تعتمد عليها في اختيار لغة البرمجة التي تناسب المجال الذي تخطط الدخول له إن أردت تطوير مسارك المهني ودخول عالم البرمجة.

تعريف البرمجة

هي العملية التي تستطيع بواسطتها إنجاز فكرة معينة أو حل مشكلة ما عن طريق تقسيم الفكرة أو المشكلة إلى خطوات متتالية قابلة لإعادة التكرار وصولاً إلى النتيجة المطلوبة. فلو سألت أحدهم ما هو ناتج العملية الرياضية التالية (3-2)*5 فقد يعطيك مباشرة الجواب 5 لكن ما فعله الدماغ هو عملية تحليل للمسألة ووضع طريقة لحلها وهذا ما يُدعى بوضع خوارزمية. Algorithm.

وخلاصة القول أن البرمجة هي طريقة لترتيب وتنظيم مسألة ما للحصول على النتيجة بالمعنى العام، أما في عالم الحواسيب فهي وسيلة للتخاطب مع هذه الأجهزة. أما الأسلوب المقترح لتنفيذ هذه الخطوات فهي الخوارزمية، وستكون عندها لغة البرمجة هي الوسيلة التي نخبر فيها الحاسوب أو التجهيز كيفية تنفيذ الخوارزمية لحل المشكلة المعروضة.

ما هي لغات البرمجة؟

لغات البرمجة تشير ببساطة على أنها وسيلة للتواصل بين البشر وجهاز الحاسب الآلي أو بعض الآلات المهيأة لتنفيذ برامج متغيرة والتي تُدعى ، ونظراً للتقدم التكنولوجي الهائل ودخول تقنيات المعلومات إلى مختلف نواحي الحياة ، ازداد توجه الصانعين إلى إنتاج الآلات و تجهيزات قادرة على التخاطب والتفاعل مع المستخدم لتنفيذ وظائف متعددة كأجهزة الصّراف الآلي ونقاط الخدمة الذاتية والهواتف الذكية وحتى التجهيزات المنزلية والسيارات، وكلما زاد تعقيد هذه التجهيزات وتعددت مهامها احتاجت إلى طريقة فعّالة لتخبرها بما هو مطلوب منها. وهنا تأتي أهمية لغات البرمجة وضرورة وجود أنواع مختلفة من لغات البرمجة وفق مستويات مختلفة لتأمين إدارة تلك التجهيزات والتواصل الأمثل معها.

انواع لغات البرمجة

هنالك المئات من لغات البرمجة وجدت لتحقيق أهداف مختلفة وحتى هذه اللحظة تظهر لغات برمجة جديدة باستمرار، لذا تصنف هذه اللغات إلى أنواع عدة بالاعتماد على وظائف كل لغة وتطبيقاتها وطريقة معالجتها وغيرها من المقاييس ونوضح في الصورة التالية بعض الأنواع أو الفئات التي تنضوي تحتها لغات البرمجة.

تصنف لغات البرمجة إلى الأنواع التالية:

أنواع لغات البرمجة وفق مستوى الترميز

- لغة الآلة Machine language
- لغات التجميع Assembly languages
- لغات البرمجة عالية المستوى High level languages

أنواع لغات البرمجة وفق طريقة معالجة التعليمات

- اللغات المصرفة (الترجمة) Compiled Languages
- اللغات المفسرة Interpreted languages
- اللغات الهجينة المصرفة المفسرة Hybrid Languages

أنواع لغات البرمجة وفق أسلوب تنظيم الشيفرة

- لغات البرمجة الوظيفية Functional Programming
- لغات البرمجة الإجرائية Procedural Programming
- لغات البرمجة الكائنية Object-oriented Programming

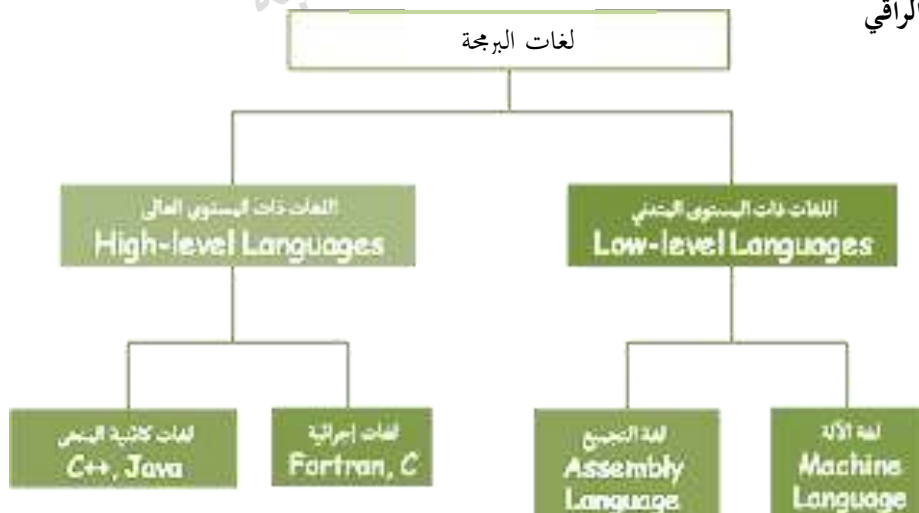
أنواع لغات البرمجة وفق مجالات الاستخدام

- لغات البرمجة عامة الغرض General Purpose programming Languages
- لغات البرمجة خاصة الغرض Special Programming Languages

ما هي لغات البرمجة السائدة (أشهر لغات البرمجة)؟

تعتمد شهرة لغات البرمجة على التطبيقات المستخدمة لاجلها فمن بين اللغات المستخدمة وأشهرها والتي تُستعمل في بناء تطبيقات سطح مكتب والتي تعمل على مختلف أنظمة التشغيل الحاسوبية هي لغة جافا Java ولغة بايثون Python حيث تستخدم لغة البايثون في مجال الذكاء الاصطناعي وتعلم الآلة والتعامل مع البيانات وغيرها، أما في مجال الويب، فأشهر لغة فيه حاليًا هي لغة جافا سكربت JavaScript فيمكنك باستعمالها تطوير الواجهات الأمامية Frontend وأصبح بإمكانك أيضًا تطوير الواجهات الخلفية Backend عبر بيئة Node.js وكما يمكنك باستعمال تقنيات الويب تطوير تطبيقات لسطح المكتب تعمل على كل أنظمة التشغيل باستعمال إطار العمل Electron.js .

لغات المستوى المتدني ولغات المستوى الراقى



لغة الآلة Machine Languages

تسمى اللغة الثنائية حيث أنها تتكون من سلسلة من 0,1 هي اللغة الوحيدة التي يفهمها جهاز الحاسب الآلي وكل حاسب له لغته الخاصة . و التعليمات في لغة الآلة لابد أن تكون سلسلة من الموز الثنائية (0, 1) حيث ان الدوائر الإلكترونية الداخلية للحاسب مصنعة من عناصر إلكترونية عادة ما تكون في حالة من اثنين (Off 0) أو (On 1).

يجب تحويل كل البرامج المكتوبة باللغات الراقية والتي يفهمها الانسان الى لغة الآلة بواسطة وسيط ، والذي يسمى المترجم Compiler حتى تتمكن معدات الحاسب من التفاهم معها ، و تتميز لغة الآلة بسرعة التنفيذ لأنها تخاطب CPU مباشرة دون وسيط.

عيوب لغة الآلة

- ✓ لغة غير مرنة
- ✓ صعوبة كتابتها وتعديلها وتصحيحها
- ✓ يجب على المبرمج معرفة تركيب الحاسب وعناوينه الداخلية
- ✓ لغة غير عمومية
- ✓ خاصة بكل جهاز على حدة لأنها مرتبطة بالمعالج نفسه
- ✓ لا تصلح لجهاز آخر

لغة التجميع Assembly Languages

ظهرت في أوائل الخمسينات حيث قام عالم الرياضيات Grace Hopper مع فريق من البحرية الأمريكية بتطوير لغة تمثل التعليمات المختلفة في لغة الآلة باستخدام رموز symbolic وسميت اللغة الرمزية Symbolic Language حيث تستخدم مختصرات ورموز يسهل حفظها وكتابتها لكل تعليمة من لغة الآلة.

يجب تحويل البرنامج المكتوب بلغة الأسمبلي الى لغة الآلة عن طريق المجمع Assembler وهو برنامج خارجي وهو بمثابة المترجم ولكن يعاب على لغة التجميع Assembly بأنها مازالت لغة غير عمومية بحيث يقتصر تصميم البرنامج للعمل على جهاز معين ، كما انها لغة صعبة ومملة لكتابة التطبيقات ولكنها لازالت تستخدم في تصميم المترجمات.

اللغات ذات المستوى العالي

سميت بهذا الاسم لبعدها عن لغة الآلة وقرىها من اللغة الطبيعية للإنسان ، و لا ترتبط بجهاز معين بل تنفذ على أكثر من جهاز ، وتحتاج لبرنامج وسيط (Compiling Program) " المترجم " ليقوم بتحويل البرنامج المصدر Source Code المكتوب بأحد اللغات الراقية او العالية المستوى إلى البرنامج الهدف Object Code المكتوب بلغة الآلة ومن ثم يقوم جهاز الحاسوب بتنفيذ التعليمات الواردة في البرنامج.



البرنامج المصدري (Source Code) : عبارة عن مجموعة من الأوامر و التعليمات المكتوبة بشكل منطقي و تسلسلي بإحدى لغات البرمجة

برنامج الهدف (Object Code) : هو البرنامج الخالي من الأخطاء مكتوب بلغة الآلة قابل للتنفيذ على جهاز الحاسوب

مميزات اللغات ذات المستوى العالي

(1) المرونة

- سهولة كتابة وتعديل البرامج المكتوبة بهذه اللغات
- سهولة اكتشاف الأخطاء وتعديلها وتصحيحها

2. العمومية

- نظراً لاستقلالها عن نوع وتفاصيل الجهاز الذي تعمل عليه.

عيوب اللغات ذات المستوى العالي

- البطء مقارنة بلغة الآلة

كيفية كتابة البرامج؟

عند الانتهاء والتحليل من تصميم الحل لمسألة ما نقوم ببناء البرنامج وتهيئته للتنفيذ على جهاز الحاسب الآلي

بناء البرنامج Building a program

عندما نقوم بكتابة البرنامج وحتى يفهم جهاز الحاسب الآلي الاوامر وتنفيذها يجب إجراء عملية الترجمة بتحويل البرنامج الى ملف تنفيذي يسمى Executable file بلغة الآلة وتتم هذه العملية في الخطوات الثلاثة الآتية :

(1) كتابة البرنامج Writing and editing the program

(2) ترجمة البرنامج Compiling the program

(3) ربط البرنامج مع مكتبة البرامج الفرعية المطلوبة Linking the program with the required library modules

تنفيذ البرنامج Program Execution

بمجرد ربط البرنامج فإنه يكون قابلاً للتشغيل وجاهزاً للتنفيذ

لتنفيذ البرنامج فإنه ينبغي استخدام أحد أوامر برنامج التشغيل مثل Run لتحميل البرنامج إلى الذاكرة الرئيسية لتنفيذه ، والبرنامج المسؤول على عملية التحميل إلى الذاكرة الرئيسية هو وظيفة أحد برامج نظام التشغيل ويدعى المحمل Loader ، بحيث يقوم يقوم بتحديد مكان البرنامج التنفيذي و ومن ثم تحميله في الذاكرة وبعد ان يصبح كل شيء جاهزاً تسلم عملية التحكم في الحاسب إلى البرنامج ويبدأ التنفيذ الفعلي

التنفيذ الفعلي للبرنامج

الخطوات المستخدمة للتنفيذ الفعلي بعد اتمام عملية التحميل حيث يقرأ البرنامج المنفذ البيانات لمعالجتها إما من خلال إدخال المستخدم لها أو يقرأها من ملف ، ثم بعد معالجة البيانات يقوم البرنامج بإخراج النتائج إما على شاشة المستخدم أو طباعتها أو كتابتها في ملف وبعد انتهاء البرنامج يقوم بإخبار نظام التشغيل ليقوم بإزالة البرنامج من الذاكرة.

مفهوم قواعد البيانات – Database Concept

تعريف قاعدة البيانات Database

مفهوم – قاعدة البيانات Database هي مجموعة من عناصر البيانات المنطقية المرتبطة مع بعضها البعض بعلاقة رياضية، وتكون قاعدة البيانات من جدول واحد أو أكثر. ويمكن أن نعرف قاعدة البيانات Database ببساطة بأنها عبارة عن ملف ضخم يمكن فيه ترتيب المعلومات التي نريد تخزينها بشكل مرتب ومنظم، ويمكن استرجاع هذه المعلومات والتعديل عليها في أي وقت. وبالتالي فإن أهميتها تكمن بأنك تستطيع معالجة وتخزين بيانات المستخدمين في مكان واحد و بكل سهولة.

مميزات قواعد البيانات

- ترتيب بيانات المستخدمين بشكل يسهل الوصول إليها و التعامل معها.
- تحديد أنواع البيانات التي يتم تخزينها بدقة مثل نصوص، أرقام، تواريخ، عملات إلخ..
- وضع شروط على البيانات التي سيتم تخزينها بالإضافة إلى إمكانية وضع قيم افتراضية.
- الوصول إلى المعلومات بشكل سريع جداً في حال تم استخدام الفهارس (Indexes).
- عدم الحاجة إلى تكرار المعلومات و هذه إحدى أهم الميزات.
- إمكانية إنشاء نسخ احتياطية (Backups) من قاعدة البيانات بكل سهولة.

نظام إدارة قواعد البيانات وتصنيفاتها Database Management System -DBMS

يُعدّ نظام إدارة قواعد البيانات تجميعاً من البرامج التي تُمكن المستخدمين من إنشاء قواعد البيانات (Database) ، والحفاظ عليها، والتحكم في جميع عمليات الوصول إليها، كما يُعدّ الهدف الأساسي لنظام إدارة قواعد البيانات هو توفير بيئة ملائمة وفعالة للمستخدمين لاسترجاع المعلومات وتخزينها.

عناصر أو مكونات قاعدة البيانات

تتكون قاعدة البيانات من العناصر الآتية

- **المخطط:** تُنظّم البيانات في المخطط في جدول واحد أو عدّة جداول، كما قد تحتوي قاعدة البيانات على عدّة مخططات
- **الجدول:** يتم تخزين البيانات في الجداول المكوّنة من أعمدة تسمى الحقول ، حيث يتراوح عددها بين عمودين إلى مئة عمود أو أكثر حسب نوع البيانات المخزنة
- **السجل:** يتم تسجيل البيانات داخل الصفوف في الجدول، لذا قد تحتوي قاعدة البيانات على مئات أو آلاف الصفوف في الجدول
- **الحقل:** يختلف نوع البيانات في كل حقل، فقد تكون على شكل تواريخ، أو قيم رقمية، أو أعداد صحيحة، أو قيم أبجدية رقمية ومجموعة الحقول تكون بما يعرف بالسجل

مثال: سجل موظف في قاعدة البيانات

يمكن ان يكون لنا مئات من الموظفين في شركة ما حيث كل موظف له سجله الخاص في قاعدة البيانات بحيث يتكون السجل الواحد من عدة حقول كالآتي:

رقم الموظف – اسم الموظف – درجة الموظف – تاريخ التعيين – الراتب – والقسم التابع له (سجل يتكون من 6 حقول)

أنواع قواعد البيانات

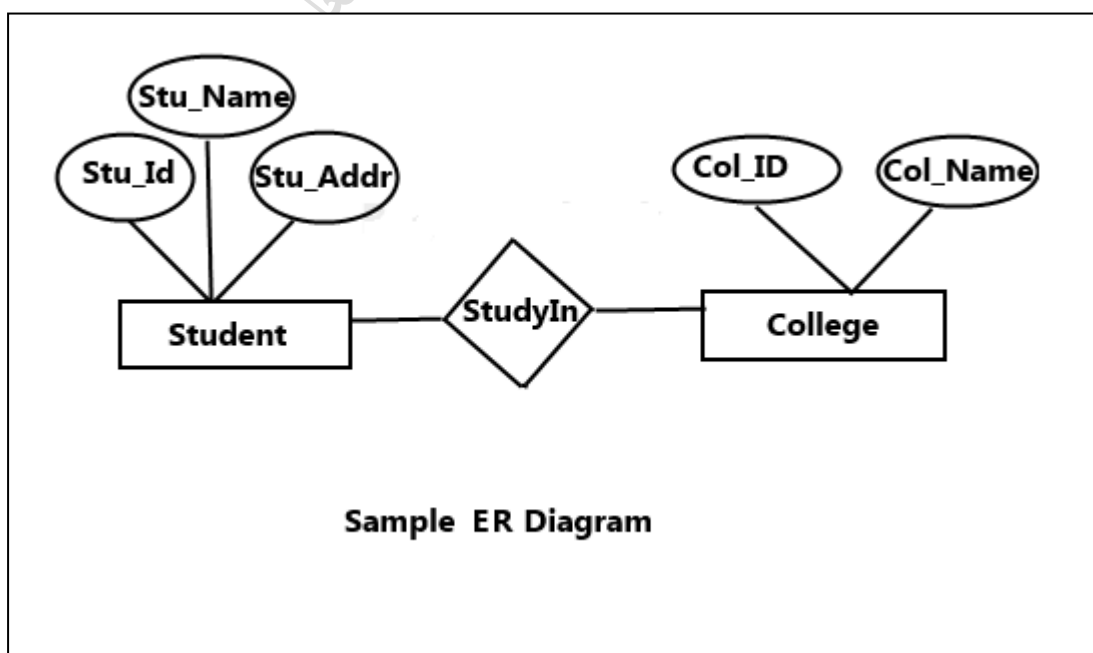
تم تطوير عدّة أنواع من قواعد البيانات

- **قواعد البيانات المُسطحة:** تتكوّن من سجلات تحتوي كيانات ذات قوائم بسيطة؛ كقواعد بيانات أجهزة الحاسوب الشخصية.
- **قواعد البيانات الهرمية:** تنظم البيانات على شكل هرمي بحيث تتفرّع السجلات فيها إلى عدّة فئات أصغر، كما ترتبط السجلات ببعضها بروابط فردية على مستويات مختلفة.
- **قواعد البيانات الشبكية:** ترتبط مجموعة من السجلات بمجموعة أخرى في قاعدة البيانات الشبكية بروابط متعددة، أو مؤشرات؛ وتُستخدم في قواعد البيانات الخاصة بالشركات والتجارة الإلكترونية.
- **قواعد البيانات العلائقية:** تنظم البيانات في صف واحد وتُسمى العلاقة، والتي قد ترتبط ببعضها البعض حسابياً بإعطاء المعلومات المطلوبة، كما تُستخدم قاعدة البيانات العلائقية لغة الاستعلام البنائية بالإنجليزية (Structured Query Language - SQL)
- **قواعد البيانات كائنية التوجه:** تُعتبر من قواعد البيانات الأكثر مرونة؛ فهي تُخزّن، وتُعالج البيانات المعقّدة، وتُنظّمها في فئات هرمية، مثل: قواعد البيانات الموجهة نحو الأرقام حيث تحتوي على الإحصائيات، والجداول، والبيانات المالية، والعلمية، والتقنية الأولية.

نموذج علاقة الكيانات Entity Relationship Model - ER Diagram

هو أحد الأساليب الشائعة لوضع تصور لقاعدة البيانات العلائقية وهو يعتمد على:

- تقسيم النظام إلى ما يسمى كيانات (مثل كيان موظف وكيان قسم في نظام شركة).
- كل كيان يحتوي على خصائص معينة تصفه وتحدده (مثل الاسم والعنوان... لكيان موظف).
- تحديد العلاقات بين هذه الكيانات وتوصيف خصائص هذه العلاقات
- تمثيل ذلك من خلال رسم يعبر عنه يسمى مخطط علاقة الكيانات



شبكات الحاسوب Computer Networks

ظهرت شبكات الحاسوب في نهاية الستينيات وأوائل السبعينيات من القرن الـ 20؛ وذلك عندما تم إنشاء شبكة (ARPANET) التابعة لوزارة الدفاع الأمريكية، حيث كانت فكرة الشبكات قبل ذلك قائمة على ربط أجهزة طرفية بأجهزة كمبيوتر رئيسية، ولم تكن فكرة ربط جهازين حاسوب أو أكثر بحيث تكون جميع الأجهزة المربوطة على الشبكة قادرة على تقاسم الموارد الموجودة عبر الشبكة بشكل متساوٍ، ومن الجدير بالذكر أنّ مُصطلح الشبكات نفسه لم يُستخدم بشكل صريح مع شبكة (ARPANET)، إلا أنّ البروتوكولات وطريقة الاتصال التي قامت عليها هذه الشبكة جعلتها من الناحية الفنية أول شبكات الحاسوب ظهوراً عبر التاريخ

تعريف شبكة الحاسوب

وهي عبارة عن مجموعة من أجهزة الحاسوب والأجهزة الأخرى التي تتصل ببعضها البعض عبر وسائط اتصال، مما يُتيح مشاركة عدد من الموارد بين المستخدمين، كأجهزة الطابعات وأجهزة المسح الضوئي، كما أنها تسمح بمشاركة الملفات والبرامج المختلفة، ومن الميزات الأخرى لشبكات الحاسوب سهولة الوصول إلى المعلومات الموجودة على الشبكة من قبل المستخدمين الآخرين، وتستخدم هذه الأجهزة المتصلة بالشبكة نظاماً من القواعد يُسمى بروتوكولات الاتصال من أجل نقل المعلومات عبر التكنولوجيات السلكية أو اللاسلكية

كيف تعمل شبكة الحاسوب

تُعدّ العقد والروابط كتلتي البناء الأساسيتين في شبكة الحاسوب. قد تكون عقدة الشبكة جهاز اتصال بيانات (DCE)، مثل المودم أو الموزّع أو المبدّل، أو جهاز بيانات طرفياً (DTE)، مثل اثنين أو أكثر من الحواسيب والطابعات. ويشير الرابط إلى وسيط نقل يربط بين عقدتين. وقد تكون الروابط مادية، مثل أسلاك الكابلات أو الألياف البصرية، أو مساحة فارغة مستخدمة بواسطة الشبكات اللاسلكية. في شبكة الحاسوب النشطة، تتبع العقد مجموعة من القواعد أو البروتوكولات التي تحدد كيفية إرسال البيانات الإلكترونية واستلامها عبر الروابط. وتحدد بنية شبكة الحاسوب تصميم هذه المكونات المادية والمنطقية. وتوفر المواصفات للشبكة من حيث المكونات المادية والتنظيم الوظيفي والبروتوكولات والإجراءات.

مكونات شبكة الحاسوب Computer Network composition

تتكون شبكة الحاسوب من مكونات مادية (Hardware) وبرمجيات (Software)

تنقسم المكونات المادية إلى:

1. الحاسبات computers بشتى أنواعها
2. الكروت Interfaces والوسائط Media
3. الأجهزة الملحقة (Peripheral devices) مثل Bridges , Switches , Amplifiers , Repeaters , Hubs , Gateways , Routers وغيرها

تنقسم البرمجيات Software إلى:

1. برامج نظم تشغيل الشبكة Operating system
2. بروتوكولات الاتصال Protocols
3. نظم إدارة الشبكة Network management systems

أنواع بنية شبكة الحاسوب

ندرج تصميم شبكة الحاسوب تحت فئتين واسعتين:

بنية خادم العميل Client - Server

في هذا النوع من شبكة الحاسوب، قد تكون العُقد في شكل خادم أو عميل. وتوفر عُقد الخادم موارد، مثل الذاكرة أو قوة المعالجة أو البيانات، إلى عُقد العميل. ويمكن أيضاً أن تدير عُقد الخادم سلوك عقدة العميل. ويمكن أن تتصل عدة أجهزة قائمة على العميل ببعضها البعض بدون مشاركة الموارد. على سبيل المثال، تخزن بعض أجهزة الحاسوب في الشبكات المؤسسية البيانات وإعدادات التكوين. وتكون هذه الأجهزة عبارة عن خوادم في الشبكة. ويمكن للأجهزة القائمة على العميل الوصول إلى هذه البيانات عن طريق تقديم طلب إلى جهاز الخادم.

بنية نظير إلى نظير Peer to Peer

في بنية نظير إلى نظير (P2P)، تتساوى الحواسيب المتصلة في القوى والامتيازات. ولا يوجد خادم مركزي للتنسيق بينها. ويمكن أن يمثل كل جهاز في شبكة الحاسوب إما عميلاً أو خادماً. ويمكن لكل نظير مشاركة بعض من موارده، مثل الذاكرة وقوة المعالجة، مع شبكة الحاسوب بالكامل. مثلاً، تستخدم بعض الشركات بنية P2P لاستضافة التطبيقات التي تستهلك الذاكرة بكثافة، مثل عرض الجرافيك ثلاثي الأبعاد، عبر الأجهزة الرقمية المتعددة.

أنواع شبكات الحاسوب

صنّف شبكات الحاسوب إلى أنواع مختلفة تبعاً للمنطقة الجغرافية التي تغطيها الشبكة، أو حسب التصميم الهندسي

أولاً (الشبكات حسب المنطقة الجغرافية

الشبكة المحلية Local Area networks - LAN

عبارة عن نظام مترابط محدود في الحجم والحيشة الجغرافية. وهي عادةً تصل أجهزة الحاسوب والأجهزة الأخرى في نطاق مكتب أو مبنى واحد. وتُستخدم من قبل الشركات الصغيرة و اعتبارها شبكة تجريبية لعمليات بناء النماذج صغيرة النطاق.

الشبكات العريضة Wide Area Network - WAN

الشبكة المؤسسية التي تمتد عبر مبانٍ ومدن ودول، يُطلق عليها اسم الشبكة العريضة، بينما تُستخدم الشبكات المحلية لنقل البيانات بسرعات عالية ضمن نطاق قريب، تكون الشبكات العريضة مجهزة للاتصالات بعيدة المسافة التي تتسم بالأمان والموثوقية.

شبكات مزود الخدمة Service Network Provider

تسمح شبكات مزود الخدمة للعملاء بتأجير سعة ووظائف شبكية مقدمة من خلال هذا المزود. ومن بين الأمثلة لمزودي خدمة الشبكة شركات الاتصالات وشركات نقل البيانات ومزودو الاتصالات اللاسلكية ومزودو خدمة الإنترنت ومشغلو القنوات التلفزيونية الذين يوفرّون وصولاً إنترنت عالي السرعة.

الشبكات السحابية Cloud Network

من الناحية المفاهيمية، الشبكة السحابية يمكن رؤيتها على أنها WAN مزودة ببنية أساسية مقدمة من خدمة قائمة على السحابة. وتكون بعض قدرات الشبكة وموارد المنظمة أو كلها مستضافة في منصة عامة أو سحابية خاصة وتتوفر عند الطلب. وتستخدم الأعمال اليوم الشبكات السحابية لتسريع وقت الوصول إلى السوق وزيادة النطاق وإدارة التكاليف بشكل فعال. أصبح نموذج الشبكة السحابية المنهج القياسي لإنشاء التطبيقات وتقديمها للمؤسسات الحديثة.

ثانياً (الشبكات حسب التصميم الهندسي (طبولوجيا الشبكات)

- **الشبكة الخطية (Bus Topolog)** تتميز أنه لا يوجد فيها وحدة تحكم مركزية، وتتكون من كيبل واحد ترتبط به أجهزة الحواسيب، وتنقل البيانات فيما بينها من خلاله بمساعدة أداة للنقل يطلق عليها اسم الموصل، أو الناقل.
- **الشبكة الحلقية (Ring Topology)** ويستخدم فيها كيبل على شكل دائري لربط عدد من الأجهزة على هيئة حلقة، ويكون الجهاز المركزي جزءاً من هذه الحلقة، وتتميز المعلومات فيها بأنها تتحرك باتجاه واحد فقط عبر الكيبل، ولها عيوب كثيرة؛ ومنها أن تعطل جهاز واحد يتسبب في تعطل كامل الشبكة، بالإضافة إلى تعقيد إدارتها.
- **التجمية (Star)** : يستمد هذا الشكل تصميمه من اسمه فعلياً، فتكون الشبكة مُصمَّمة على شكل نجمة؛ بحيث يكون كل جهاز في الشبكة متصلاً بشكل مباشر مع الجهاز الرئيسي للشبكة، ويكون هذا الجهاز الرئيسي مسؤولاً عن الشبكة، وعن طريقة ومسار نقل البيانات ما بين الأجهزة، لكن عيب هذا الشكل أنه إذا ما توقف الجهاز الرئيسي أو حدث له عطلٌ فستتوقف الشبكة كلها.
- **الشبكة الشبكية (Mesh Topology)** يعتمد هذا النوع من الشبكات على أن يكون كل جهاز متصل بطريقة مباشرة مع بقية أجهزة الشبكة، وذلك باستخدام أنواع خاصة من الكوابل، وتعتبر هذه الشبكة من الشبكات المعقدة، إذ تكون الكوابل كثيرة التشعب، مما يزيد صعوبة إدارتها وصيانتها.
- **الشبكة اللاسلكية (Wireless Topology)** وهي أحدث أنواع الشبكات، وتعتمد على التقنيات اللاسلكية نحو تقنية إرسال الراديو، ومن الممكن أن تكون شبكة مستقلة، أو جزء من شبكة سلكية كبيرة.

أجهزة الربط في شبكات الحاسوب

يتم تأمين الاتصال والتفاعل بين الأجهزة الموجودة ضمن شبكة الحاسوب من خلال ما يعرف بأجهزة الشبكات (Networking Devices)؛ فشبكات الحاسوب ليست مجرد أجهزة حاسوب وكيبلات تربط فيما بينها فقط، ويوضح ما يأتي بعض أشهر الأجهزة التي يتم استخدامها لتوصيل أجهزة الشبكة ببعضها البعض

- **الجسر (Bridge)** : وهو جهاز يقوم بإعادة توجيه المعلومات الواردة إليه نحو جهاز آخر تبعاً لعنوان مادي (Physical Address) مُحدّد.
- **المحور (Hub)** : وهو جهاز يقوم بتوجيه المعلومات الواردة إليه إلى جميع الأجهزة المرتبطة به عبر الشبكة.
- **الموزّع (Switch)** : وهو جهاز يقوم بإعادة توجيه البيانات الخالية من الأخطاء، بشكل صحيح إلى جهاز الكمبيوتر المعني، تبعاً لرقم المنفذ (Port) الذي يرتبط به الكمبيوتر مع الموزّع.
- **الموجه (Router)** : وهو جهاز يقوم بإعادة توجيه البيانات والمعلومات الواردة إليه إلى جهاز كمبيوتر مُعيّن، تبعاً للعنوان المنطقي لذلك الجهاز، ويُعرف العنوان المنطقي بعنوان IP الخاص بالجهاز.
- **المكرّر (Repeater)** : وهو جهاز يقوم باستقبال الإشارات ليُضخمها، ثم يُعيد توجيهها من جديد لتصل لمسافات أطول.

فوائد شبكات الحاسوب

تتميّز شبكات الحاسوب بالعديد من الفوائد بالنسبة لمستخدميها، ومنها ما يأتي:

- **تخزين المعلومات على كمبيوتر مركزي**: يُمكن من خلال شبكات الحاسوب تخزين الملفات التي يعمل عليها المستخدمون على كمبيوترات مركزية أو ما يُعرف بالخوادم (Servers) ؛ والتي تكون بدورها مُتاحة للاستخدام من قبل مُستخدمي جميع الأجهزة المرتبطة بالشبكة.
- **مشاركة البيانات والمعلومات**: تساهم شبكات الحاسوب في مُشاركة البيانات عبر جميع الأجهزة المرتبطة بالشبكة.

- **زيادة موثوقية البيانات:** وذلك من خلال وجود نسخ احتياطية للبيانات الموجودة على أكثر من جهاز كمبيوتر عبر الشبكة، بحيث إذا تعطلت إحدى النسخ الموجودة على جهاز معين وتعذر الوصول إليها، فإنه يمكن الوصول إليها من أي جهاز آخر عبر الشبكة.
- **تعزيز أمن المعلومات:** حيث يتم منح كل مستخدم عبر الشبكة صلاحيات محددة، تُتيح له استخدام بيانات أو تطبيقات محددة دون غيرها، مما يمنع أي مستخدم لا يمتلك صلاحية من الاطلاع على المعلومات أو البيانات التي لا يحق له الاطلاع عليها.
- **مشاركة الموارد:** تساهم شبكات الحاسوب في إمكانية مشاركة الأجهزة والتطبيقات المختلفة كأجهزة الطابعات ومحركات الأقراص الثابتة بين مستخدمي الشبكة.
- **الاتصال والتواصل الفعلي:** توفر شبكات الحاسوب للمستخدمين إمكانية إرسال واستقبال الرسائل بالوقت الفعلي الحقيقي، ومن خلال أجهزة مختلفة عبر الشبكة.

علم الحاسوب - المرحلة التمهيدية للعلوم التطبيقية

البوابات المنطقية الأساسية Logical Gates

تُعدّ البوابات المنطقية Logical Gates عنصراً أساسياً في أي نظام رقمي، حيث تكون على شكل دائرة إلكترونية بسيطة تتواجد في الحواسيب ممثلةً بالنظام الثنائي (Binary Number System) و المبني على (0 ، 1) وتُقسم البوابات المنطقية إلى نوعين رئيسيين؛ البوابات المنطقية الأساسية والبوابات المنطقية المشتقة وفيما يأتي أنواع البوابات المنطقية الأساسية:

بوابة (AND) : تُسمّى (و)، وتضمّ مدخلين ومخرجاً واحداً، وتُعامل معاملة عملية الضرب في الرياضيات، كما هو موضح أدناه



- 0 (AND) 0 تُعطي 0
- 0 (AND) 1 تُعطي 0
- 1 (AND) 0 تُعطي 0
- 1 (AND) 1 تُعطي 1

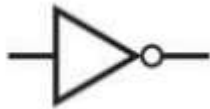
بوابة (OR) : تُسمّى (أو)، وتضمّ مدخلين ومخرجاً واحداً، وتُعامل معاملة عملية الجمع في الرياضيات، كما هو موضح أدناه



- 0 (OR) 0 تُعطي 0
- 1 (OR) 0 تُعطي 1
- 0 (OR) 1 تُعطي 1
- 1 (OR) 1 تُعطي 1

بوابة (NOT) : تُسمّى ليس أو العاكس أو بوابة النفي المنطقية ، وتضمّ مدخلاً واحداً ومخرجاً واحداً، وتُمثّل العاكس لأنها تُعطي

نتيجة مُعكّسة كما هو موضح أدناه



- 0 (NOT) تُعطي 1
- 1 (NOT) تُعطي 0

و - AND

X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

$$z = x \cdot y = xy$$



أو - OR

X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

$$z = x + y$$



ليس - NOT

X	Z
0	1
1	0

$$z = \bar{x} = x'$$



استخدامات البوابات المنطقية

فوائد البوابات المنطقية لا تعدّ ولا تُحصى، فهي تدخل في العديد من الصناعات التكنولوجية، ومن أبرزها ما يأتي

- بناء معالجات الأجهزة الإلكترونية.
- برمجة الحواسيب.
- صناعة مضخات خزانات المياه.
- صناعة الأجهزة والمعدات الطبية.
- صناعة أشباه الموصلات
- صناعة الساعات الرقمية والمؤقتات الزمنية

العبارات المنطقية

العبرة المنطقية هي العبرة القابلة للصواب True أو الخطأ False فمثلا:

العبرة : (True and False) هي عبارة منطقية وقيمتها خطأ False

العبرة : (9 < 6) هي عبارة منطقية وقيمتها خطأ False

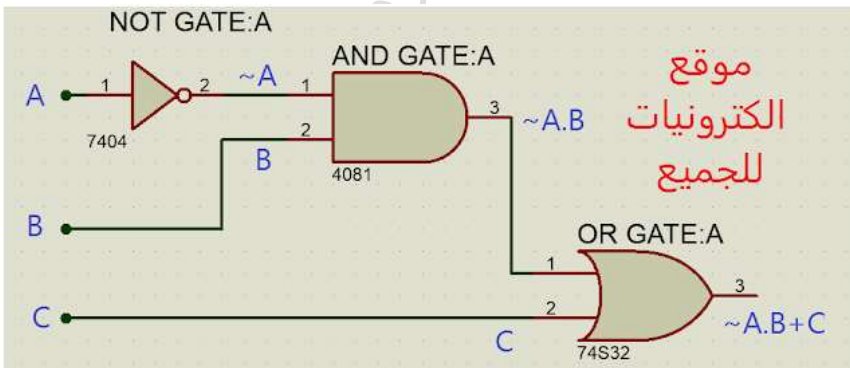
العبرة: (8 > 9 or 4 تساوي 3) ي أيضا هنا منطقية وقيمتها True ، ويجب أن نلاحظ أن معاملا المؤثر المنطقي قد تكون تعابير منطقية أو قد يكونا تعابير علائقية.

كيفية قراءة مخططات الكترونية تشمل على بوابات منطقية

يتم قراءة وتحليل المخططات الالكترونية التي تشمل على بوابات منطقية عن طريق فرض المتغيرات على مداخلها ثم تتبع المنطق الحاصل على تلك المتغيرات المنطقية فمثلا المعادلة المنطقية الآتية

$$\bar{A}.B + C$$

الناجمة من الدارة المنطقية التالية ، تم استخراجها عن طريق التتبع في المخطط الخاص بتلك الدارة على النحو الآتي:



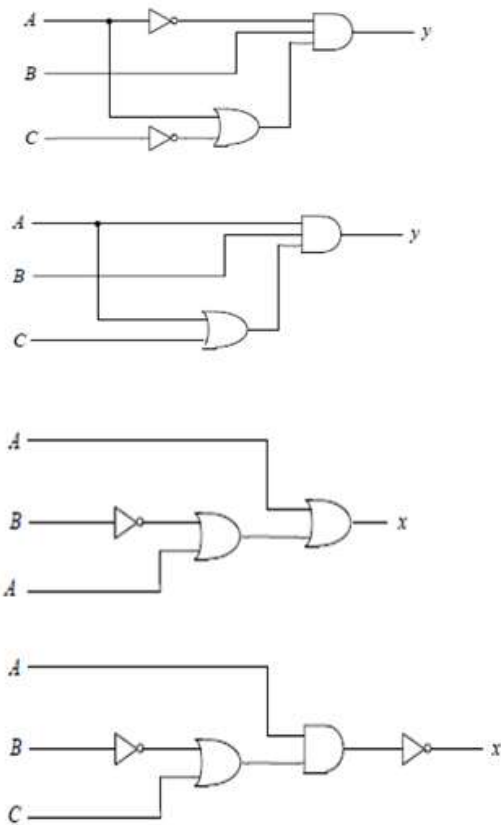
نلاحظ نفي المتغير A ليصبح $\bar{A}$ ومن ثم يدخل الى البوابة المنطقية AND مع المتغير B ليصبح المخرج عن تلك البوابة $\bar{A}.B$ ، واخيرا تدخل القيمة المنطقية $\bar{A}.B$ مع المتغير المنطقي C في البوابة المنطقية OR لتكون النتيجة النهائية للعملية المنطقية $\bar{A}.B + C$ وهو ما يمثل الاخراج النهائي

بعد معرفتنا لتلك المعادلة ، أصبح بالإمكان معرفة متى يكون الناتج المنطقي للمعادلة $\tilde{A}.B + C$ مساوياً للمنطق 1 أو 0 ويتم ذلك حسب الجدول الصواب (المنطقي) الآتي :

A	B	C	$\tilde{A}$	$\tilde{A}.B$	$\tilde{A}.B + C$
0	0	0	1	0	0
1	0	0	0	0	0
0	1	0	1	1	1
1	1	0	0	0	0
0	0	1	1	0	1
1	0	1	0	0	1
0	1	1	1	1	1
1	1	1	0	0	1

تمرين:

باستخدام النظام الثنائي و جداول الصواب (المنطقي) أوجد قيمة كل من (X) و (Y) في الدوائر المنطقية الآتية :



الأنظمة العددية – Number Systems

الحاسوب في الحقيقة لا يفهم إلا لغة واحدة وتسمى لغة الآلة (Machine Language) ولا يوجد بهذه اللغة إلا رمزان هما الصفر والواحد أو النظام العددي الثنائي (Binary digits) ومن الصعب التخاطب مع الحاسوب بهذه اللغة. ويجري تحويل كل الحروف والأرقام والرموز والأوامر إلى سلسلة من الأصفار والواحدات باستخدام برامج خاصة (compilers أو interpreters) حتى يتمكن الحاسوب من فهمها وتنفيذ المطلوب. وتستخدم عدة أنظمة ترميز أو تشفير "code systems" لتحويل الحروف والأرقام والرموز والأوامر إلى لغة الآلة من أشهرها نظام الترميز (ASCII).

ونظام الترميز (ASCII) هو عبارة عن مجموعة من الرموز الرقمية التي تمثل الحروف والأرقام والرموز الخاصة ، وتستخدم على نطاق واسع في نقل البيانات بين أجهزة الكمبيوتر. (فمثلا سلسلة الأرقام 01000001 بالنظام الثنائي والذي يكافئ 65 بالنظام العشري يمثل حرف A في نظام الترميز القياسي الأمريكي لتبادل المعلومات ASCII). ونظرا لأن الحاسوب يعتمد على الأنظمة الرقمية فسنقوم بدراسة بعض الأنظمة الرقمية وكيفية التحويل من نظام رقمي إلى نظام رقمي آخر.

يوجد العديد من الأنظمة العددية علي سبيل المثال:

1. النظام العشري (Decimal system) – نظام عددي أساسه 10 ويتكون من الرموز (0, 1, 2, 3, 4, 5, 6,

7, 8, 9). يعتبر النظام العشري أكثر أنظمة العد شيوعا واستعمالا من قبل الإنسان. وتمثل الأعداد في النظام العشري بواسطة الأساس 10.

$$\text{على سبيل المثال العدد العشري } 125 \text{ يمكن كتابته } 1 \times 10^2 + 2 \times 10^1 + 5 \times 10^0 \\ \text{والعدد } 7129.45 = 7 \times 10^3 + 1 \times 10^2 + 2 \times 10^1 + 9 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$

2. النظام الثنائي (Binary system) – نظام عددي أساسه 2 ويتكون من رمزين هما (0, 1). وهذا النظام الذي يتعامل به الحاسوب.

3. النظام الثماني (Octal system) – نظام عددي أساسه 8 ويتكون من الرموز (0, 1, 2, 3, 4, 5, 6, 7).

4. نظام السادس عشر (Hexadecimal) – نظام عددي أساسه 16 ويتكون من الرموز (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F). ويستعمل في علم الحاسوب لتسهيل كتابة الشفرات الثنائية.

الجدول التالي على سبيل المثال يبين الاعداد العشرية من 0 إلى 16 وما يقابلها في النظم الثلاث الأخرى:

العشري	الثنائي	الثماني	السادس عشر
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

التحويل من النظام الثنائي إلى النظام العشري:

الرقم الثنائي وعادة يكتب على النحو التالي (الرقم)₂ علي سبيل المثال:

• $10_{10} = 8 + 0 + 2 + 0 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 2(1010)$ بالنظام العشري.

• أما الرقم $2(1101.01)$ = $2(1101) + 2(.01)$

$13 = 8 + 4 + 0 + 1 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 2(1101)$

$0.25 = 0 + \frac{1}{4} = 0 \times 2^{-1} + 1 \times 2^{-2} = 2(.01)$

• إذا $2(1101.01)$ = $2(13 + .25)_{10}$ أو 13.25 بالنظام العشري.

القاعدة العامة للتحويل من النظام الثنائي إلى النظام العشري:

• الجزء الصحيح $d = a_n \times 2^n + a_{n-1} \times 2^{n-1} + \dots + a_1 \times 2^1 + a_0 \times 2^0$

• الجزء الكسري $f = b_{-1} \times 2^{-1} + b_{-2} \times 2^{-2} + \dots + b_{-(m-1)} \times 2^{-(m-1)} + b_{-m} \times 2^{-m}$

• المكافئ بالنظام العشري $d + f$

التحويل من النظام العشري إلى النظام الثنائي:

أولاً: تحويل الجزء الصحيح

للتحويل من النظام العشري ونشير له بـ d إلى النظام الثنائي ونشير له بـ b نحتاج لإيجاد المعاملات a_i حيث $d = a_n \times 2^n + a_{n-1} \times 2^{n-1} + \dots + a_1 \times 2^1 + a_0 \times 2^0$

ويمكن أداء ذلك بتكرار تنفيذ الجملتين التاليتين طالما أن قيمة d لا تساوي صفر :

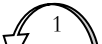
- حساب ناتج قسمة العدد علي 2 أي $d = d // 2$. حيث المؤثر $//$ يعطي العدد الصحيح فقط من ناتج القسمة.
- حساب باقي قسمة العدد بالنظام العشري علي 2 ونشير له بـ m . أي أن $m = d \% 2$

مثال: أوجد المقابل الثنائي للعدد 10_{10}

1	2	5	
1//2=0	2//2=1	5//2=2	10//2=5
1	0	1	0

إذاً المقابل بالثنائي = $2(1010)$

مثال: أوجد المقابل الثنائي للعدد 615_{10}

										
1//2	2//2	4//2	9//2	19//2	38//2	76//2	153//2	307//2	615//2	d//2
=0	=1	=2	=4	=9	=19	=38	=76	=153	=307	
1	0	0	1	1	0	0	1	1	1	 1%2

إذاً المقابل بالثنائي = $2(1001100111)$

ثانياً: تحويل الجزء الكسري

ويمكن أداء ذلك بتكرار تنفيذ الجملتين التاليتين طالما أن قيمة d لا تساوي صفر :

- حساب ناتج حاصل ضرب العدد d في 2 أي $d = d * 2$.
- تجزئة ناتج حاصل الضرب إلى جزء صحيح $\text{int}(d)$ وجزء كسري $d = d - \text{int}(d)$

مثال: أوجد المقابل الثنائي للعدد $_{10}(0.125)$

$d = d * 2$	$0.125 * 2 = 0.25$	$0.25 * 2 = 0.5$	$0.5 * 2 = 1.0$
$\text{int}(d)$	0	0	1
$d = d - \text{int}(d)$	0.25	0.5	0

إذاً المقابل بالثنائي للعدد العشري $_{102}(0.125) = _2(0.001)$

مثال: أوجد المقابل الثنائي للعدد $_{10}(0.6875)$

	$0.6875 * 2 = 1.375$	$.375 * 2 = .75$	$.75 * 2 = 1.5$	$.5 * 2 = 1.0$
$\text{int}(d)$	1	0	1	1
$d = d - \text{int}(d)$	0.375	0.75	0.5	0

إذاً المقابل بالثنائي $_{2}(0.1011) =$

$$\begin{aligned} \text{مثال: أوجد المقابل الثنائي للعدد } _{10}(10.6875) &= _{10}(10) + _{10}(0.6875) \\ _2(0.1011) + _2(1010) &= \\ _2(1010.1011) &= \end{aligned}$$

ملاحظة:

ليس كل الكسور العشرية قابلة للتحويل إلى كسر ثنائي علي سبيل المثال الكسر $_{10}(0.1)$. كما الحال بالنظام العشري ليس كل الكسور الإعتيادية قابلة للتحويل إلى كسور عشرية على سبيل المثال $_{10}(\frac{1}{3})$

التحويل من النظام الثماني إلى الثنائي:

لتحويل أي عدد ثماني إلى مكافئه الثنائي نستبدل كل رقم من أرقام العدد الثماني بمكافئه الثنائي المكون من ثلاث خانوات و بذلك ينتج لدينا العدد الثنائي المكافئ للعدد الثماني المطلوب تحويله.

مثال: حول العدد الثماني $(772.5)_8$ إلى مكافئه الثنائي:

$$\begin{array}{cccc} 7 & 7 & 2 & 5 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 111 & 111 & 010 & 101 \end{array}$$

$$(772.5)_8 = (111111010.101)_2$$

التحويل من النظام الثنائي إلى الثماني:

لتحويل الأعداد الثنائية الصحيحة إلى ثمانية تتبع الخطوات التالية:

- 1- نقسم العدد الثنائي إلى مجموعات كل منها مكون من ثلاث خانات، و يجب أن نبدأ التقسيم من الرقم الأقل أهمية (LSD) .
- 2- إذا كانت المجموعة الأخيرة غير مكتملة فإننا نضيف في نهايتها الرقم صفر حتى تصبح مكونة من ثلاث خانات ثنائية.
- 3 - نضم الأرقام الثمانية معاً للحصول على العدد المطلوب.
- 4 - في حالة الكسور الثنائية نبدأ بالتقسيم إلى مجموعات من الخانة القريبة على الفاصلة.

التحويل من النظام الست عشري إلى الثنائي:

لتحويل أي عدد من النظام السداسي عشر إلى مكافئه الثنائي تتبع الآتي:

مثال حول العدد السداسي عشر (D39A)₁₆ إلى مكافئه الثنائي.

- نستبدل الخانات المكتوبة بدلالة الحروف إن وجدت في العدد بالأعداد العشرية المكافئة لها.
- نقوم بتحويل الاعداد العشرية لما يكافئها من الاعداد الثنائية بحيث يكون كل عدد ثنائي به أربع خانات.
- ثم نضم الأرقام الثنائية مع بعضها لنحصل على العدد المطلوب: $(D39A)_{16} = (1101001110011010)_2$

D	3	9	A
↓	↓	↓	↓
13	3	9	10

التحويل من النظام الثنائي إلى النظام الست عشري:

لتحويل أي عدد صحيح من النظام الثنائي إلى الست عشري تتبع الآتي:

1. نقسم العدد الثنائي إلى مجموعات كل منها يتكون من 4 خانات مع مراعاة أن يبدأ التقسيم من الرقم الأقل أهمية (LSD) .
مثال: العدد الثنائي التالي 101001101101111001101 يصبح تقسيمه إلى مجموعات كالتالي: 1 0100 1101 1011 1100 1101
2. إذا كانت المجموعة الأخيرة غير مكتملة فإننا نضيف في نهايتها الصفر حتى تصبح مكونة من أربعة خانات :

0001	0100	1101	1011	1100	1101
------	------	------	------	------	------

3. نحول كل مجموعة ثنائية إلى مكافئها في النظام العشري:

0001	0100	1101	1011	1100	1101
1	4	13	11	12	13

4. نستبدل كل رقم عشري (من الخطوة السابقة) أكبر من 9 بدلالة حروف النظام السداسي عشر:

1	4	13	11	12	13
1	4	D	B	C	D

5. نظم الأرقام الناتجة مع بعضها لنحصل على المطلوب بالنظام السادس عشر 14DBCD

6. إذا كان العدد الثنائي كسراً نبدأ بالتقسيم إلى مجموعات من الخانة القريبة على الفاصلة ثم نتبع باقي الخطوات المشروحة سابقاً.

1.1 تمارين

1. قم بالتحويلات الآتية:

- العدد الثنائي (10101.11) يساوي () في النظام العشري
- العدد العشري (15) يساوي () في النظام السادس عشر
- العدد السادس عشر (AF.D) يساوي () في النظام الثماني
- العدد الثماني (37.21) يساوي () في النظام الثنائي

2. إحسب المكافئ بالنظام العشري للقيم بالنظام الثنائي.

المكافئ بالنظام العشري	الرقم بالنظام الثنائي
	$2(100101)$
	$2(111111)$
	$2(1100.101)$
	$2(0.1100011)$

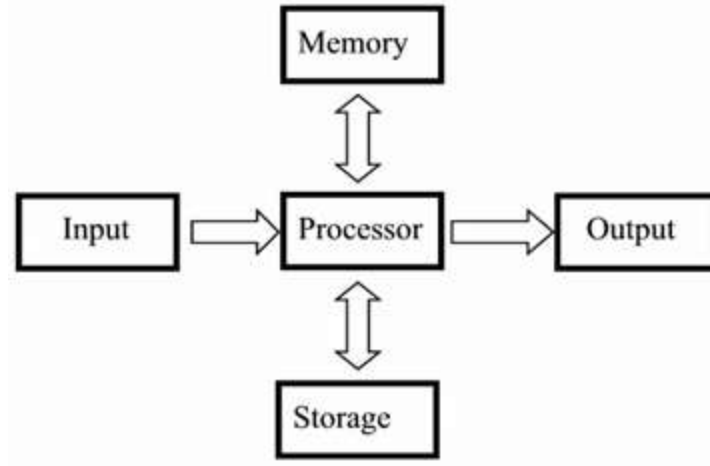
3. إحسب المكافئ بالنظام الثنائي للقيم بالنظام العشري.

الرقم بالنظام العشري	المكافئ بالنظام الثنائي
25.75	
36.8125	
99.275	
64.111	
128.0625	

أساسيات الحاسوب والبرمجة

1.1. مفاهيم أساسية:

من المفاهيم الأساسية للحاسب الآلي اعتباره آلة تستقبل مدخلات معينة (inputs) ثم تقوم بمعالجة (Process) وإجراء بعض الحسابات (calculate) على هذه المدخلات بناءً على تعليمات محددة (instructions) وإرجاع نتائج تلك المعالجات والحسابات (outputs).



لغات البرمجة: ببساطة هي وسيلة للتخاطب مع الحاسوب. في الحقيقة لا يفهم الحاسوب إلا لغة واحدة وتسمى لغة الآلة (Machine Language) ولا يوجد بهذه اللغة إلا رمزان هما الصفر والواحد (Binary digits) ومن الصعب التخاطب مع الحاسوب بهذه اللغة.

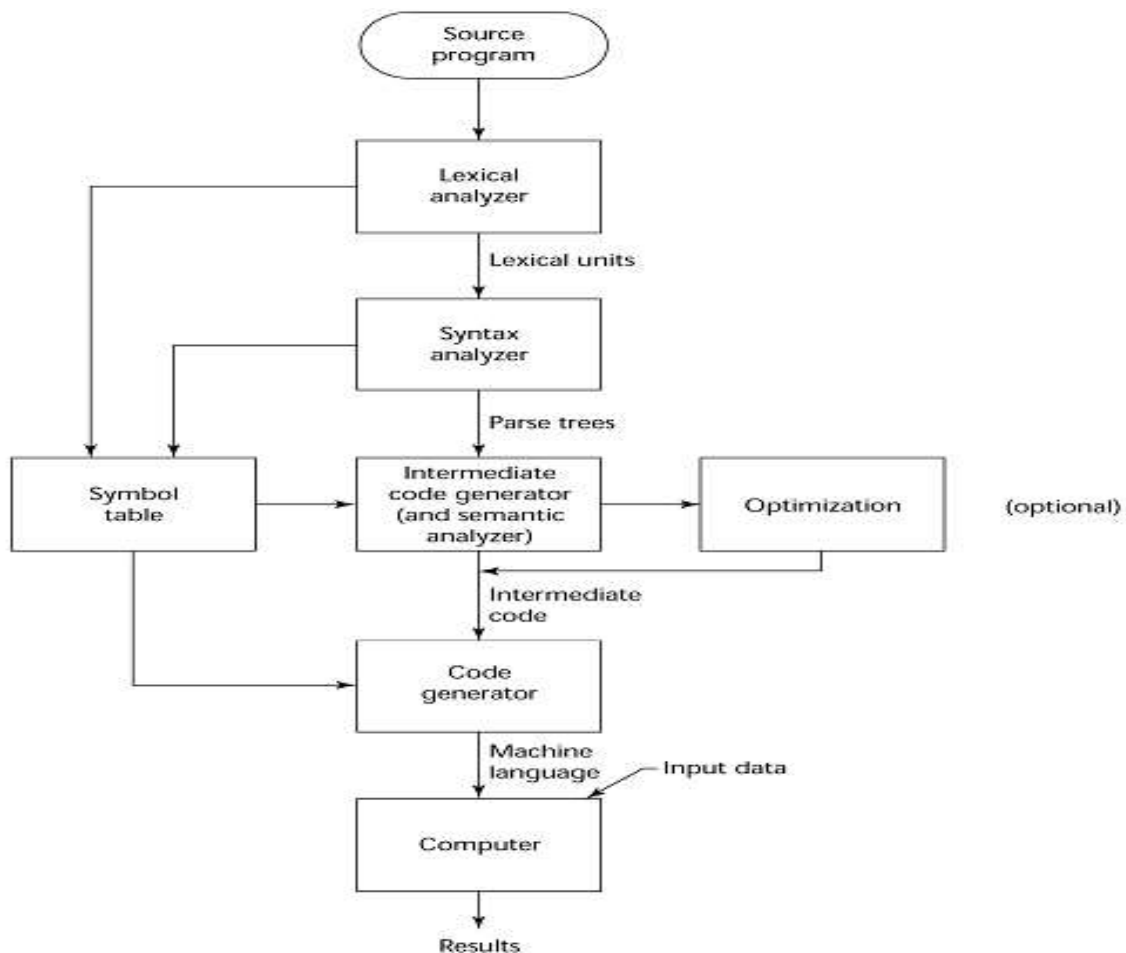
تصنيف لغات البرمجة:

- التطبيقات العلمية – (MATLAB, FORTRAN)
- التطبيقات التجارية – (COBOL)
- الذكاء الاصطناعي – (ML, PROLOG, LISP)
- برمجة النظم – (C/C++, Assembly Language)
- برمجة الأنترنت – (Java script, HTML, Python)
- التطبيقات ذات الأغراض الخاصة – special purpose languages : (GPS, GIS, Crystal Report, CAD)

أنواع لغات البرمجة:

1. المترجمة (Compiled) : الترجمة الكلية ويتم فيها ترجمة البرنامج المكتوب بأحد لغات البرمجة على سبيل المثال (Fortran, Pascal, C/++, ADA) إلى لغة الآلة دفعة واحدة قبل الشروع في التنفيذ، ومن خصائصها (بطء الترجمة، سرعة أثناء التنفيذ). وتتم عملية الترجمة بعدة مراحل كما هو موضح في (الشكل 1.1)

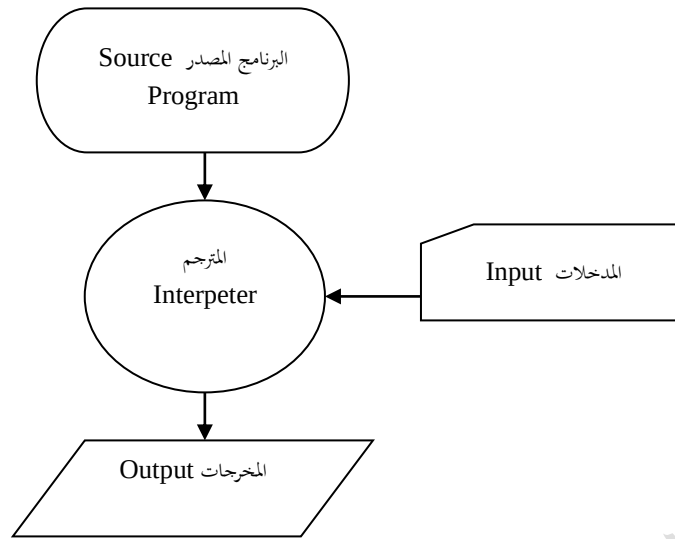
2. **المفسرة (Interpreted)** : الترجمة الفورية ويتم فيها عملية الترجمة والتنفيذ جملة جملة ومن اللغات الشائعة التي تستخدم هذا الأسلوب (Lisp, Basic, Matlab, Perl, Python). ومن خصائصها (سهولة الاستخدام ، بطء في التنفيذ نسبياً). انظر شكل (1.2)
3. **الهجينة (Hybrid implementation systems)** : وهو خليط من النوعين السابقين. انظر شكل (1.3)



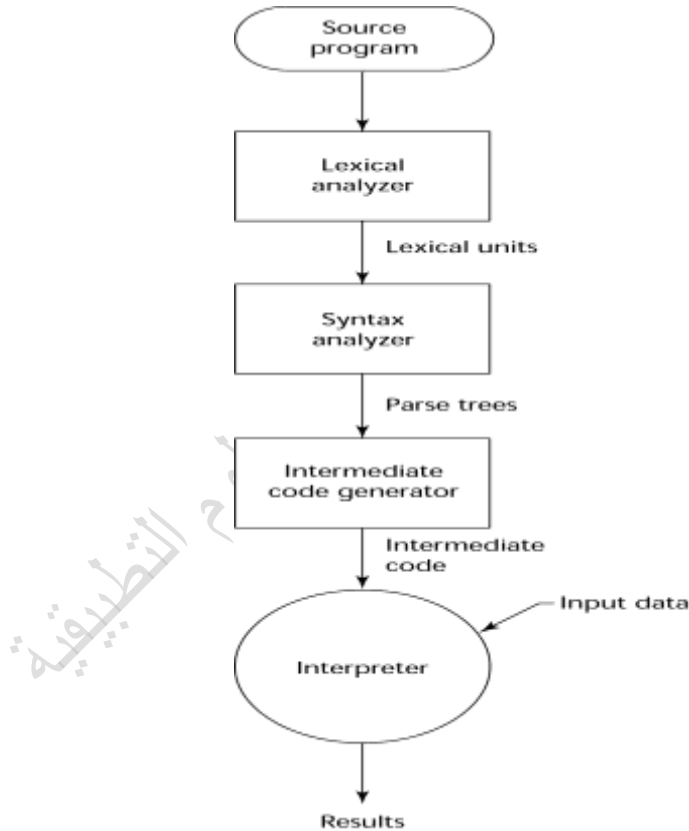
الشكل 1.1 يبين مراحل تفسير البرنامج المصدر

يمكن تلخيص الشكل 1.1 بالخطوات التالية : (كيف يتم تفسير البرنامج) ؟

- تفحص البرنامج المصدر لغوياً وتكوين مفردات البرنامج (lexical units).
- تفحص البرنامج تركيبياً وبنوياً. وينتج (parse tree).
- تحويل البرنامج المصدر إلى مكافئ بلغة أخرى ويسمى بـ intermediate code.
- تحويل الـ intermediate code إلى لغة الآلة Machine Language.



شكل (1.2) مراحل ترجمة البرنامج المصدر



الشكل 1.3 يبين النوع المجين

الغرض من هذا الأسلوب هو المحاولة لجعل البرنامج قابل للتنفيذ علي أكثر من منصة تشغيل Unix,linux,Window,..., (etc.) وأكثر من معمارية للمعالج (Intel, Motorola, ..., etc) وبدون الحاجة إلى إعادة عملية تحويل البرنامج المصدر إلى برنامج بلغة الآلة (Object code).

كيف يتم تنفيذ البرامج بالحاسوب ؟

- مرحلة Linking Process : بعد انتهاء مرحلة الترجمة إلى لغة الآلة وتكوين البرنامج بلغة الآلة (Object code) وهذا البرنامج يعتمد علي معمارية المعالج ويختلف من معالج إلى آخر تتم عملية ربط هذا البرنامج مع عدد من البرامج الأخرى سواء كانت موجودة ضمناً مع اللغة أو يجب إدراجها حتي يتم تكوين ملف قابل للتنفيذ ويسمى بـ (Executable code) من قبل الحاسوب.
- مرحلة Loading : يتم في هذه المرحلة تحميل البرنامج التنفيذي إلى ذاكرة الحاسوب من قبل نظام التشغيل (Operating System) لبدأ عملية التنفيذ.

نظام العمل بوحدة المعالجة المركزية CPU operation:

يتم العمل داخل وحدة المعالجة بدورتين (دورة التعليمة instruction cycle ، دورة التنفيذ execution cycle):

- دورة التعليمة Instruction cycle – يتم خلالها جلب التعليمة من ذاكرة الحاسوب وتفسيرها عن طريق وحدة التحكم Control unit وتستغرق زمناً يسمى بـ زمن التعليمة.
- دورة التنفيذ Execution cycle – يتم خلالها تنفيذ التعليمة عن طريق وحدة الحساب والمنطق Arithmetic and Logical unit وتستغرق زمن يسمى بـ زمن التنفيذ.

ما هو نظام التشغيل Operating System ؟

- نظام التشغيل عبارة علي برامج معقد ويحتوي علي الملايين من الجمل التنفيذية، وهو واجهة التعامل بين البشر وجهاز الحاسوب. هذا البرنامج المسئول عن إدارة وتنسيق الأنشطة المختلفة داخل الحاسوب ويقدم الخدمات التالية:
- تنفيذ البرامج.
 - إدارة وتنسيق التعامل مع المكونات المادية لجهاز الحاسوب مثل (لوحة المفاتيح، الطابعات، وحدات التخزين الإضافية مثل القرص الصلب، الذاكرة الرئيسية، ... الخ).
 - حماية وتنظيم عملية الولوج في الملفات
 - استعادة الوضع السوي بعد حدوث الخطأ Error Recovery
 - اكتشاف الأخطاء والاستجابة لها Error detection and response

هل يوجد نظام تشغيل صالح لكل الأجهزة ؟

لا – للأسباب التالية:

- نظام التشغيل هو البرنامج الأول والرئيسي للتعامل بين المستخدمين وعتاد الحاسوب. نظام التشغيل يتم إعدادة إعداداً خاصاً للعمل مع المعالج.
- معمارية المعالج Processor Architecture.
- نظم التشغيل تجدد وتطور باستمرار.

أمثلة لنظم التشغيل :

- DOS – Disk Operating System
- OS/2
- عائلة Windows وتشمل الإصدارات من القديم إلى الحديث (Windows 3.x ، 95 ، NT ، 98 ، 2000 ، ME ، XP ، 2003 server ، VISTA ، 7) .
- Macintosh operating system
- UNIX
- LINUX

هل تختلف لغات البرمجة عن لغات البشر ؟ نعم

لأن المستهدف من هذه اللغات هو التخاطب مع جهاز الحاسوب.

خصائص لغات البرمجة:

- محدودية المفردات والقواعد اللغوية.
- الجمل لا تقبل التفسير بأكثر من طريقة (الوضوح).

مراحل تطور لغات البرمجة تاريخيا:

بما ان لغات البرمجة صُنفت تاريخيا على اساس اجيال الحاسب الآلي فما كان من البديهي ان تمر بمراحل التطوير بما يتواءم والاجيال مع ظهور العديد من اللغات المستخدمة حسب الحاجة فهناك ، نجد الكثير من لغات البرمجة الحديثة التي نضجت وبدأت تأخذ مكانها في عالم البرمجة وخاصة برمجة الانترنت علي سبيل المثال (python, C#, PHP, html, xhtml).

الجدول التالي يبين برنامج بلغات مختلفة لعرض الجملة المشهورة بأول برنامج لتعليم لغة برمجة : والجملة هي : Hello World.

Fortran	Program Hello write(*,*) " Hello World" End Program
C	#include <stdio.h> main() { printf("Hello World\n"); }
C++	#include <iostream.h> main() { cout<<"Hello World\n"; }

COBOL	IDENTIFICATION DIVISION. PROGRAM-ID. HELLOWORLD . ENVIRONMENT DIVISION. DATA DIVISION . PROCEDURE DIVISION. BEGIN. DISPLAY " " LINE 1 POSITION 1 ERASE EOS. DISPLAY "HELLO, WORLD." LINE 15 POSITION 10. STOP RUN. END.
Java	class HelloWorld { public static void main (String args[]) { System.out.print ("Hello World \n"); } }
Pascal	Program Hello (input, output); Begin Writeln ('Hello World'); End.
Perl	Print "Hello World\n";
Python	Print ('Hello World')
Smalltalk	Transcript show:'Hello World';cr
Visual Basic	Private Sub Form_Load() Msgbox "Hello World" End sub
VRML	#VRML V1.0 ascii AsciiText { String "Hello World" justification LEFT }
HTML	<html> <title>Hello World </title> <head/> <body> Hello World <body/> <html/>
C#	static void Main(string[] args) { console.WriteLine("Hello World"); }
Assembly Language	TITLE EXASM1 (COM) Display message on screen ;----- CODESG SEGMENT PARA 'CODE' ASSUME CS:CODESG, DS:CODESG, SS:CODESG, ES:CODESG ORG 100H BEGIN: JMP MAIN

	<pre> ;----- MESSAGE DB 'Hello World', 13, 10, '\$' ;----- MAIN PROC NEAR LEA DX, MESSAGE MOV AH,9 INT 21H RET MAIN ENDP CODESG ENDS END BEGIN </pre>
--	---

ما معنى برنامج ؟

مجموعة من الجمل مكتوبة بإحدى لغات البرمجة يتم تنفيذها حرفياً من قبل الحاسوب توضح كيفية تحقيق غرض معين، مثل حل معادلات رياضية، معالجة بيانات الطلاب الدراسية أو معالجة نصوص، . . . الخ.

أساسيات البرمجة واحدة بجميع اللغات ولكن الاختلاف في التفاصيل، وتتمثل الأساسيات في :

- المدخلات (Input) : إدخال البيانات من (لوحة المفاتيح، ملفات، ماسحة ضوئية، Bar code reader، . . . الخ).
- المخرجات (Output) : عرض النتائج والمعلومات (على الشاشة، إرسالها لملفات، طباعة على الورق، رسوم بيانية، صوت، صورة، حركية مثل فيديو، . . . الخ).
- جمل معالجة : رياضية، علائقية، منطقية، نصية.
- جمل شرطية : وهي جمل للتحكم في مسار تنفيذ الجمل بناء على شروط معينة.
- جمل التكرار : وتختص بتنفيذ الجمل أكثر من مرة إلى غاية أن يتحقق شرط معين أو التنفيذ أكثر من مرة طالما تحقق شرط معين.

متى نحتاج لكتابة برنامج ؟

من بعض الحالات التي نحتاج فيها للبرامج التالي:

- تحليل المعادلات الرياضية المعقدة والمتكررة والتي لا تحتمل الخطأ.
- تخزين ومعالجة البيانات الضخمة مثل البيانات الشخصية والبحث عن معلومة معينة.
- معالجة البيانات المالية كالمصارف.
- الاتصالات ونقل البيانات.

1.2 حل المسائل (Problem solving)

إن أي تطبيق لحل المسائل (لأداء أحد الأعمال السابقة أو غيرها) باستخدام الحاسوب هو عمل يتطلب الكثير من التخطيط والعمل الذهني لأن جهاز الحاسوب لا يتمتع بالذكاء وإنما جهاز قادر على تنفيذ المهام ويتميز بالسرعة على التنفيذ. إن تصميم خطوات حل المسائل يستلزم اتباع الخطوات التالية:

1. **تحديد المسألة** (تحليل المسألة): وهو أمر ضروري حتى نتمكن من التصميم الجيد للبرنامج و يعنى تحديد "بوضوح" المعطيات (المدخلات) للبرنامج والنتائج المستهدف (المخرجات) للمسألة المراد حلها، ويشمل نوع البيانات ووسط التعامل (واجهات) لإدخال البيانات ووسط عرض النتائج (بيانات رقمية ورمزية، رسومات، صورة، صوت... الخ) وكذلك هل المطلوب حفظ البيانات المدخلة والنتائج بوسائل حفظ البيانات المختلفة مثل القرص الصلب، ... الخ. أي فهم المسألة جيداً.
2. **تصميم الحل**: الخوارزمية (Algorithm) - أي تحديد الخطوات والعمليات اللازمة لحل المسألة " بوضوح" بحيث تكون غير قابلة للتفسير بأكثر من طريقة ، إما بكتابة الخطوات باستخدام أحد اللغات الحية أو باستخدام المخططات الإنسيابية (Flow Chart) وهي غير مناسبة للحاسوب، تم تحويل الخوارزمية أو المخطط الإنسيابي إلى برنامج بأحد لغات البرمجة، وفي حالة المسائل المعقدة يتم تجزئة المسألة إلى عدد من المسائل الجزئية الصغيرة.
3. **تجربة البرنامج** وتنفيذه على جهاز الحاسوب ببيانات عشوائية وبيانات معروفة النتائج مسبقاً.
4. **التصحيح** (debugging): مراجعة الخطوات في حالة وجود أخطاء وتصحيحها ويمكن تصنيف الأخطاء إلى 3 أنواع:
 - **خطأ لغوي (Syntax Error)**: خطأ يحدث نتيجة لمخالفة قواعد اللغة وعادة ما يتم اكتشافه من قبل البرنامج المترجم. مثلاً: (2+1) جملة صحيحة، لكن (2+1+2) جملة غير صحيحة.
 - **خطأ منطقي (Semantic Error)**: وهذا النوع من الأخطاء لا تكتشفه لغة البرمجة، وعادة ما يتم اكتشافه من قبل المبرمج بتجربة البرنامج ببعض البيانات التي يتوقع نتائج معالجتها. في هذا الخطأ البرنامج ينفذ وينتج مخرجات ولكن المهمة التي أنجزها ليست المهمة المطلوبة، أي بعبارة أخرى البرنامج الذي تم كتابته ليس البرنامج المطلوب.
 - **خطأ تنفيذي (Runtime Error)**: ويسمى أحياناً بـ Exceptions. وهذا النوع من الأخطاء يحدث نتيجة حدوث خطأ أثناء تنفيذ جمل البرنامج مثلاً: الفيض (Over Flow, Under Flow)، القسمة على صفر.
5. **تحسين البرنامج** ويشمل:
 - المدخلات.
 - المخرجات.
 - العمليات: على سبيل المثال حساب مجموع الأعداد من 1..n. هناك طريقتين للحل: الأول والبديهي ولكن ليس الأفضل هو جمع الأعداد بداية من 1 إلى غاية n وتتطلب هذه المهمة n عملية جمع. أي أن عمليات الجمع تتأثر بقيمة n.
 - والحل الأمثل هو حساب مجموع هذه الأعداد باستخدام العلاقة التالية:

$$\text{مجموع الأعداد} = \frac{(1+n)n}{2}$$
 وتحتاج هذه المهمة 3 عمليات فقط مهما كانت قيمة n (عملية جمع، وعملية ضرب، وعملية قسمة).
 - تسهيل استخدام البرنامج من قبل الغير (مثلاً بإضافة تعليقات توضح مهام البرنامج comments).

1.3 الخوارزميات Algorithms

الخوارزمية: عبارة عن مجموعة من الخطوات التي عند تنفيذها تصل إلى الحل المطلوب. وسميت بهذا الاسم نسبة للعالم المسلم محمد بن موسى الخوارزمي الذي وضع أسلوباً محدد لحل المسائل الجبرية علي صورة مجموعة من الخطوات التي باتباعها نصل إلى الحل المطلوب. بعبارة أخرى هي خطوات تبين طريقة حل المشكلة.

بعد تحديد المسألة وفهمها بصورة صحيحة، نأتي إلى خطوة تصميم الحل لهذه المسألة. فبعد أن تترسخ خطوات حل المسألة في دماغ المصمم نحتاج إلى إبراز هذه الخطوات (التي تمثل الخوارزمية) إلى الوجود ويتم ذلك بعدة طرق منها الآتي :

- **الطريقة النصية :**

في هذه الطريقة يتم التعبير عن الخوارزمية باستخدام خطوات مكتوبة بلغة معينة، قد تكون من اللغات الطبيعية مثل : الإنجليزية أو العربية أو ... أو شفرة شبيهة البرنامج (pseudo-code) ويمكن استخدام أحد لغات البرمجة وقد تم استخدام لغة Algol في بداية النهضة في علم الحاسوب لغرض عرض الخوارزميات في البحوث والنشرات والدوريات العلمية .

- **الطريقة الرسومية (Flow Chart) :**

وهي طريقة تستخدم الأشكال الرسومية للتعبير عن الخوارزمية. حيث أن كل شكل له مدلول خاص.

ولكن في هذه الدروس سنتبع منهجا جديدا وهو التعبير عن الخوارزمية باستخدام لغة برمجة (Python) وهي لغة بسيطة وسهلة وقابلة للتنفيذ على جهاز الحاسوب علي عكس الطريقتين السابقتين (الطريقة النصية، الطريقة الرسومية) واللتان لهما العديد من السلبيات وهذا ما نلاحظه من بعض الأمثلة.

في الأمثلة التالية سنستعرض بعض المسائل وطريقة الحل باستخدام الطريقة النصية والطريقة الرسومية.

1.4. الخوارزمية النصية :

مثال 1 : اكتب خوارزمية لقراءة نصف قطر دائرة وحساب المساحة والمحيط..

تصميم الحل :

- **المعطيات :** نصف قطر الدائرة (نق) وقيمة ط تقريبا 3.14159
- **المعالجة :** تبين كيفية (خطوات إيجاد المطلوب) للوصول إلى المطلوب. وفي هذا المثال نحتاج إلى معادلة المساحة ومعادلة المحيط.

المساحة (م = ط × نق²) و المحيط (ح = 2 × ط × نق)

- **المخرجات:** عرض قيمة كل من المساحة والمحيط.

مبدئيا لتصميم حل متكامل لهذا المثال نقوم بتفكيك المسألة إلى ثلاث أجزاء كالاتي:

1.	البداية
2.	إدخال المعطيات
3.	المعالجة
4.	عرض المطلوب
5.	النهاية

المرحلة الثانية من التصميم هي تحسين التصميم أي كتابة الخطوات بتفاصيل أكثر. وبذلك تكون الخوارزمية قد اكتملت. وهي كالتالي:

الخوارزمية :
1. البداية
2. اقرأ قيمة نق
3. $ط = 3.14159$
4. احسب قيمة المساحة: $م = ط \times نق^2$ احسب قيمة المحيط: $ح = 2 \times ط \times نق$
5. طباعة قيمة م و قيمة ح
6. النهاية

خطوة البداية تنبه المتتبع للخوارزمية أن المتتبع يبدأ من هنا، وخطوة النهاية تدل على نهاية الخوارزمية.

كيفية التأكد من صحة الخوارزمية ؟

تتبع الخوارزمية يدويا بقيم مختلفة لنصف القطر وهذا غير مقبول في الخوارزميات المعقدة والتي تشمل العديد من العمليات الحسابية والعلاقية المعقدة.

مثال 2 : اكتب خوارزمية لحساب متوسط درجات الطلبة الدارسين بمقرر CS100.

تصميم الحل :

- **المعطيات:** عدد الطلبة ويشار له بـ ن والدرجة بـ د.
- **المعالجة :** تبين كيفية (خطوات إيجاد المطلوب) الوصول إلى المطلوب. الخطوات اللازمة لتحقيق المطلوب هي :
- جمع جميع الدرجات ويشار له بـ ج.
- **حساب المتوسط** $م = \frac{ج}{ن}$
- 1. **المخرجات:** عرض قيمة المتوسط م.

ومبدئيا لتصميم حل متكامل لهذا المثال نقوم بتفكيك المسألة إلى ثلاث أجزاء كالتالي :

1. البداية
2. إدخال المعطيات
3. المعالجة
4. عرض المطلوب
5. النهاية

المرحلة الثانية من التصميم هي تحسين التصميم أي كتابة الخطوات بتفاصيل أكثر. خطوة المعالجة في هذا المثال تحتاج إلى تفصيل أكثر، لإيجاد المتوسط نحتاج إلى قراءة الدرجة وإضافتها إلى الدرجات السابقة لها في الإدخال، هذا الفعل يتكرر حتى قراءة آخر درجة،

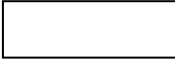
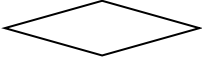
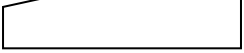
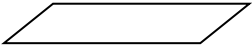
ثم قسمة هذا المجموع على عدد الطلبة للحصول على المتوسط. وبذلك تكون الخوارزمية قد اكتملت. وهي كالتالي:



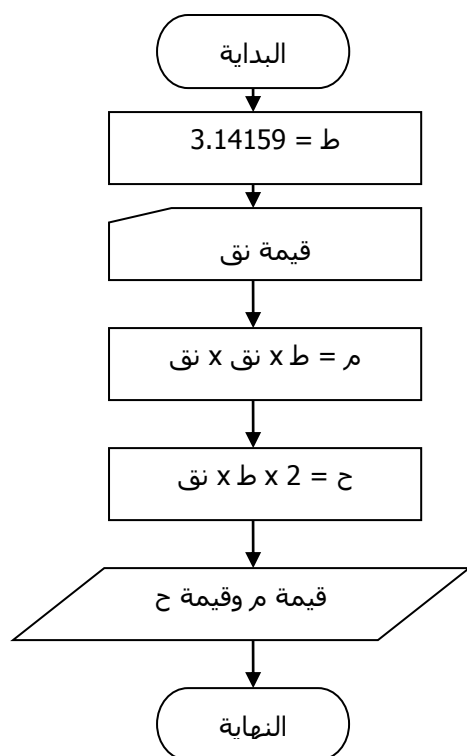
1.5. الخوارزمية الرسومية :

المخطط الانسيابي (Flow Chart): ويسمى بمخطط سير العمليات - وهو طريقة تخطيطية باستخدام أشكال هندسية وأسهم توضح مسار العمليات، وكل شكل هندسي يعبر عن نوع من الأوامر أو التعليمات المستخدمة في حل المسألة والأشكال الهندسية هي علي سبيل المثال:

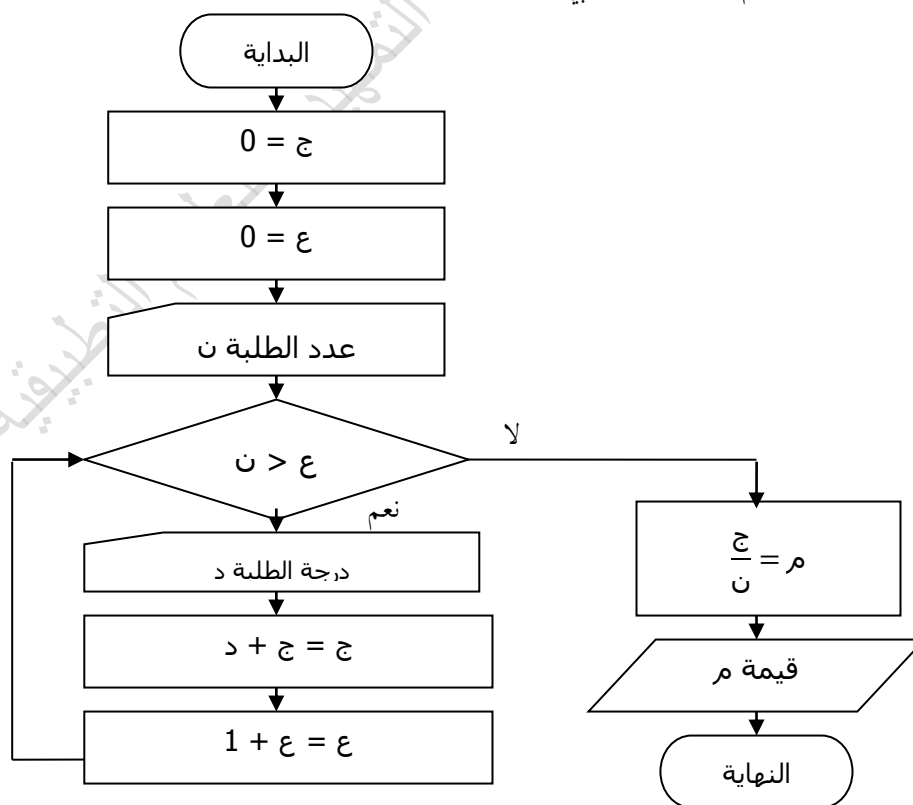
جدول يبين الاشكال المستخدمة في المخطط الانسيابي

	البداية أو النهاية
	جمل التعيين والمعالجة
	جملة شرطية
	الإدخال
	الإخراج
	الربط
	اتجاه سير العمليات

مثال 3: أعد كتابة المثال 1 باستخدام المخطط الانسيابي.



مثال 4: أعد كتابة المثال 2 باستخدام المخطط الانسيابي.



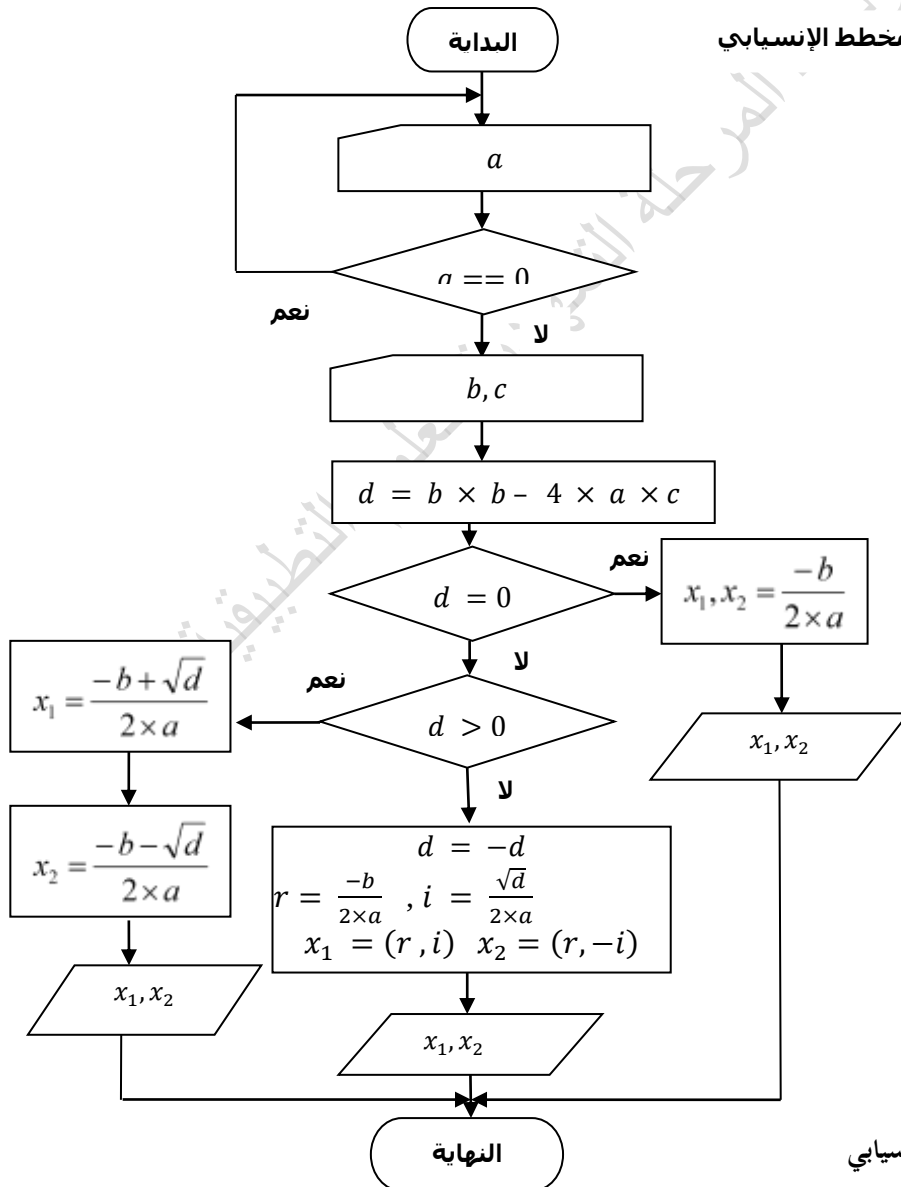
مثال 5 ارسم المخطط الانسيابي لحل معادلة من الدرجة الثانية إي إيجاد قيم لـ x التي تحقق المعادلة: $ax^2 + bx + c = 0$ تصميم الحل :

- المعطيات : قيم المتغيرات الآتية : c, b, a . وأن $a \neq 0$.
- المعالجة : تبين كيفية الوصول إلى المطلوب (خطوات إيجاد المطلوب). في هذا المثال نحتاج إلى القانون التالي، مع الأخذ في الاعتبار في حالة قيمة ما تحت الجذر سالبة.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

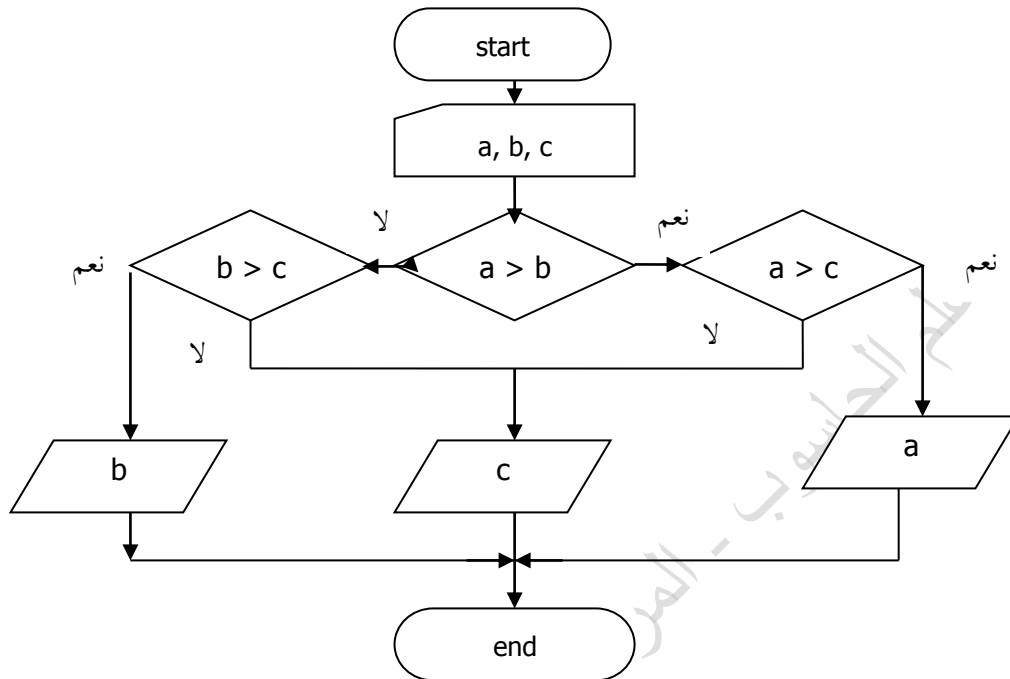
- المخرجات: عرض قيمتي ويشار إليهما بـ x_1, x_2
- التحليل: نلاحظ :

- قيمة a يجب أن لا تساوي صفر
- إذا كان تحت الجذر يساوي صفر وفي هذه الحالة للمعادلة قيمة واحدة لـ x . إي أن $x_1 = x_2$
- إذا كان تحت الجذر قيمة موجبة وفي هذه الحالة للمعادلة قيمتين حقيقيتين ويشار لهما بـ x_1, x_2 .
- إذا كان تحت الجذر قيمة سالبة وفي هذه الحالة للمعادلة قيمتين مركبتين من جزء حقيقي وتخيلي ويشار لهما بـ x_1, x_2 .



قصور الحل الخوارزمي والمخطط الإنسيابي

من الواضح والملاحظ أن استخدام المخطط الإنسيابي لكتابة خطوات حل المسألة يعاني من ضياع الوقت برسومات قد تعقد الحل في المسائل الكبيرة المعقدة، وتعتمد المخططات الإنسيابية على الأسهم لتوجيه سير العمليات (adhoc method) مما يجعل البرنامج صعب التتبع والتصحيح نتيجة لتشابك الأسهم وعلى سبيل المثال المخطط التالي: وهو مخطط إنسيابي لإيجاد أكبر قيمة من بين 3 قيم.



"وبالرغم من بساطتها، فإن المخطط الإنسيابي يوفر إمكانيات قوية وفعالة بسبب حرية الحركة والانتقال من مكان لآخر داخل المخطط حسب ما تملية منطقيات الحل دون التقيد بتركيبات أو سياق منطقي متقن. وينتج عن ذلك مخططا معقد من كثرة الخطوط المتداخلة والنقلات المفاجئة. وفي مرحلة البرمجة إما ان يستجيب المبرمج لمثل هذا المخطط باستخدام التعليمة السحرية المشهورة GOTO أو أن يهمل المخطط الإنسيابي ويتقيد بقواعد البرمجة الحديثة وأهمها البرمجة الهيكلية Structured Programming والبرمجة المرئية Visual Programming والبرمجة الشيئية Object-Oriented Programming وينتج عنه ذلك تضارب في وثائق المسألة يعوق حسن التنفيذ والصيانة والتطوير فيما بعد.¹

وفي النهاية نحتاج إلى لغة برمجة لتحويل المخططات الإنسيابية أو الخوارزميات النصية إلى برنامج لتنفيذها على الحاسوب ، وهذا جعل كثير من المبرمجين يتجاوزون هذه المرحلة وخاصة في الوقت الحاضر لتوفر ورخص أجهزة الحاسوب وكذلك البرامج واللغات المفتوحة المصدر. بالإمكان الإستعانة بلغة برمجة بسيطة، سهلة الاستعمال مرنة، تسمح بالبناء التدريجي للحل، متوافقة مع الأساليب الحديثة في البرمجة مثل ما يعرف بـ Object Oriented Programming (OOP)، مما يوفر للمصمم فرصة للتحقق من طريقة حله للمسألة مباشرة بدون الحاجة إلى خطوة إعادة كتابة الخوارزمية إلى شفرة معينة. وهذا يوفر على المصمم كثيرا من الوقت والجهد.

¹ مصطفى عبدالعال، إسماعيل بن زاهية. تحسين المخططات الإنسيابية لأغراض البرمجة الهيكلية. المجلة الليبية للعلوم، منشورات جامعة الفاتح / كلية العلوم (العدد 16 - أ 2008).

تمارين

1. اكتب خوارزميات نصية للتمارين 2 و 3 و 4

2. ارسم المخطط الانسيابي لقراءة قيمة صحيحة لـ n من لوحة المفاتيح وحساب وعرض مضروب العدد ولنشر له بـ $nfact$
 $nfact = 1 \times 2 \times 3 \times \dots \times (n - 1) \times n$

3. ارسم المخطط الانسيابي لقراءة قيمة صحيحة لـ $n < 1$ وحساب وعرض ناتج المعادلة :

$$Sum = (1 - 1/4) \times (1 + 1/9) \times \dots \times (1 \pm 1/n^2)$$

4. ارسم المخطط الانسيابي لقراءة قيمة صحيحة لـ $n < 1$ وحساب وعرض ناتج المعادلة :

$$prod = \frac{1}{1^2} \times \frac{3}{2^2} \times \frac{5}{3^2} \times \dots \times \frac{2n-1}{n^2}$$

5. تم تكليفك من قبل شركة المدار للإتصالات الهاتفية لرسم المخطط الانسيابي لحساب وعرض تكلفة المكالمات الصادرة من المشتركين ويشار له بـ $cost$ بمعرفة الزمن المستغرق للمكالمة بالدقائق ويشار له بـ $time$ ويستخدم الجدول التالي لحساب التكلفة:

زمن المكالمة بالدقائق (time)	تكلفة الدقيقة الواحدة بالدرهم (price)
من 1 إلى 5 دقائق	100
من الدقيقة السادسة وحتى العاشرة	80
من الدقيقة الحادية عشر إلى نهاية المكالمة	50

6. ارسم المخطط الانسيابي لقراءة الـ 3 درجات ويشار لهم بـ $g1$, $g2$, $g3$ علما بأن الدرجة أكبر أو تساوي صفر وأصغر أو تساوي 100. تم حساب وعرض متوسط مجموع أعلي درجتين من الثلاث درجات.

يمكنك التحقق بالاستعانة بالجدول التالي سبيل المثال:

شكل الناتج المطلوب عرضه	fg	g3	g2	g1
Final grade = 65	$(70+60)/2=65$	70	60	40
Final grade = 70	$(80+60)/2=70$	80	50	60
Final grade = 55	$(55+65)/2=60$	50	65	55

2. خوارزميات بسيطة التركيب

في هذا الفصل سنعرض مجموعة من المسائل بسيطة التركيب ولا تحتاج إلى هياكل خوارزمية معقدة لحلها. حل هذه المسائل لا يتكون إلا من خطوات ادخال واخراج وبعض العمليات الرياضية.

قبل كل ذلك لنستطيع تنفيذ الخوارزميات النصية على جهاز الحاسوب رأينا أن نستخدم لغة Python كأداة تعبير عن الخوارزمية وبذلك يمكننا أن نعتبر برنامج مكتوب بهذه كخوارزمية نصية يمكن تنفيذها مباشرة على الحاسوب. وقبل البدء في حل هذا النوع من المسائل سنقدم بعض مفردات لغة Python التي ستساعدنا على كتابة حلول المسائل.

2.1. لماذا لغة Python ؟

هذه اللغة تعتبر لغة حديثة ومن خصائصها :

- سهولة التعلم والإستعمال.
- الجمل والتراكيب بسيطة.
- تعتمد على التنسيق والمحادثة في الجمل الشرطية، الحلقات، والوظائف.
- لها جمل لمعالجة الأخطاء الإستثنائية (Exception Handling).
- تدعم البرمجة (Object Oriented Programming).
- البناء التدريجي والديناميكي للبرنامج.
- تدعم الأسلوب البنائي (Structured Approach) (لا يوجد أمر **GOTO** الذي يجعل البرنامج صعب المتبع).
- لغة مفسرة (Interpreted)، ويعني القدرة على تجربة جمل لوحدها بدون الحاجة لعملية كتابة وتخزين وترجمة برنامج مما يقلل من الجهد والوقت لتطوير البرنامج.
- مفتوحة المصدر (open source) مما جعل العديد من المهتمين إبدأ الملاحظات والإسهامات لتطوير وتحسين اللغة.
- متوافقة مع العديد من أنظمة التشغيل المختلفة مما جعل عملية نقل وتبادل البرامج بين أنظمة التشغيل المختلفة سهلة ولا تحتاج إلى تغيير يذكر.
- وتصنف اللغة من قبل البعض من ضمن اللغات النصية "scripted languages".

للحصول علي معلومات عامة والإصدارات وما هو الجديد ؟ عن اللغة بإمكان تصفح موقع اللغة : www.python.org

تنبيه :

- جميع الكلمات المكتوبة بالخط العريض المائل هي كلمات خاصة بلغة Python أو وظائف يمكن إستدعائها وعليه سيتم تبني هذا العرف خلال هذه الدروس.
- معظم لغات البرمجة ومن ضمنهم لغة Python تحتفظ بعدد من الكلمات المحجوزة ولا يسمح بإستخدامها كأسماء لمتغيرات أو دوال، وهذه الكلمات يجب كتابتها بالحروف الصغيرة. وينتج خطأ لغوي في حالة إستعمالها كمتغيرات أو كتابتها بأحرف كبيرة. والكلمات هي:

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

- تم إعتقاد واستخدام إصدار اللغة.

Python 3.2.2 (default, Sep 4 2011, 09:51:08) [MSC v.1500 32 bit (Intel)]
on win32

- كتابة البرامج ولو كانت بسيطة بإعتماد أسلوب الوظائف أو ما يعرف بـ (Functional Programming) وهو ما تميزت به لغة Lisp وأتبع نفس الأسلوب في لغة C/C++ وكذلك فلسفة (OOP) بالدروس المتقدمة للبرمجة، وكذلك التركيز على تعليم الأسلوب التفكيكي وإستخدام أسلوب "فرق تسد" (divide and conquer) ويعتمد هذا الأسلوب على تفكيك المسألة المعقدة إلى مسائل جزئية لتبسط الحل وإجراء الاختبارات والتصحيح وبعد ذلك تجميع وبناء الحل النهائي للبرنامج.

- الأمثلة التي ستعرض في هذا الكتاب ستستخدم العرف التالي:

○ العلامات >>> إستخدامها مترجم لغة Python لينبه المستخدم على أنه جاهز لإستقبال أوامره.

○ تنفيذ البرنامج يتم من خلال استدعاء الوظيفة مثلا : main() على النحو التالي

>>> main()

- في لغة Python الجمل التي تم إزاحتها عدد من الفراغات (وهي على هامش واحد) إلى جهة اليمين هذه الجمل تقع تحت سيطرت جملة التحكم التي تسبقها. وينتهي مجال تأثير جملة التحكم عند أول جملة تخرج ناحية اليسار عن هامش الجمل السابقة وتكون في نفس مستوى جملة التحكم.

2.2. القيم والمتغيرات Values and variables

1. القيم Values :

تنقسم القيم بلغة Python إلى الأنواع التالية :

- قيم رقمية صحيحة (Integer) مثل 502، -140، 12345، ... أي اعداد بدون الفاصلة العشرية.
- قيمة رقمية كسرية (Float) مثل 0.1234، -45.، 3.1415، ... أي اعداد بالفاصلة العشرية.
- قيم رقمية تخيلية مثل 5j، -45j أي بإضافة الحرف اللاتيني **j** يمين العدد.

- قيم نصية مثل "برمجة"، "Programming"، "CS100"، "لغات برمجة"، "1234"، ... أي مجموعة من الرموز محاطة بعلمتي التنصيص المفردة (') أو المزدوجة (").
- قيم منطقية وتشمل القيمتين True, False فقط. لاحظ شكل الحرف الأول في الكلمتين.

تنبيه :

ما الاختلاف بين القيمتين (1234، "1234") ؟
هل يمكن التحويل من 1234 إلى "1234" والعكس وكيف ؟

2. المتغيرات variables

- المتغير هو إسم يشير إلى عنوان لتخزين قيمة من القيم المذكورة أعلاه.
 - يتكون إسم المتغير من الحروف اللاتينية والأرقام 0,1,2,3,4,5,6,7,8,9 والرمز (_) على أن يبدأ بحرف.
 - عادة تستعمل الرمز (_) لربط كلمتين أو أكثر.
- يمكننا تخيل المتغير على أنه مكان (صندوق) توضع فيه قيمة معينة وقابلة للتغير.

ملاحظة :

- ضرورة التمييز بين الشرطة السفلية (_) وإشارة الناقص (-) فهذه الأخيرة غير مسموح بها في اسم المتغير.
- عند اختيار اسم المتغير من الأفضل أن يكون الاسم يدل على المسمى، مما يجعل البرنامج سهل التتبع والتوثيق.

مثلا :

- المتغير name نستخدمه لتخزين اسم شخص أو شيء.
- المتغير birth_date نستخدمه لتخزين تاريخ ميلاد شخص.

أمثلة لأسماء جائزة كأسماء متغيرات:

name, phone_no, cs100, student_id, birthdate

تنبيه :

لاحظ استخدام الأحرف الصغيرة في أسماء المتغيرات بدلا من الأحرف الكبيرة، وذلك تمييزا لأسماء المتغيرات من أسماء الثوابت. هذه القاعدة ليست ملزمة ولكنها عادة تم الاتفاق عليها في كتابة البرامج، وسوف نقوم باتباعها. لاحظ أيضا أن لغة Python تميز بين الحرف الصغير والكبير، فمثلا المتغير sum لا يكافئ المتغير Sum في لغة Python.

المخرجات والمدخلات :Inputs and Outputs

المخرجات :

● لعرض نص :

القاعدة العامة لعرض نص على الشاشة: استخدام علامة التنصيص المزدوجة. او المفردة. مثلا:

<code>print("هذا مثال")</code>	استخدام علامة التنصيص المزدوجة.
<code>print('هذا مثال')</code>	استخدام علامة التنصيص المفردة.
عند تنفيذ هذه الجملة يظهر على الشاشة النص التالي: هذا مثال	

مثال 1: اكتب وظيفة تقوم بعرض الجملة Hello world!

ولتنفيذ البرنامج يتم كتابة اسم `main()` كما هو مبين. في هذا المثال أستخدم الأمر `print` لعرض النص الموجود بين علامات التنصيص " " على شاشة الحاسوب.

<pre>def main(): print("Hello, World!")</pre>	} الوظيفة
<pre>>>>main()</pre>	

Hello, World!	← الاستدعاء

	← ناتج البرنامج

مثال 2 : برنامج لعرض مجموعة من النصوص.

<pre>def main(): print("Jack and Jill went up a hill") print("to fetch a pail of water;") print("Jack fell down, and broke his crown,") print("and Jill came tumbling after. ")</pre>
<pre>>>>main()</pre>

Jack and Jill went up a hill to fetch a pail of water; Jack fell down, and broke his crown, and Jill came tumbling after.

● لعرض قيمة عددية:

تستخدم الجملة `print` لعرض قيمة (أو قيم) عددية كآلاتي: مثلا

لو عندنا المتغيران الأتيان $x = 100$ و $y = 50.45$ وأردنا أن نعرض قيمتيهما على الشاشة نستخدم الجملة التالية:
<code>print(x,y)</code>
فعند تنفيذ هذه الجملة يظهر على الشاشة 100 50.45

المدخلات:

كيف يتم إسناد قيمة لمتغير ؟

أولاً : عن طريق إدخال القيمة من لوحة المفاتيح، باستخدام الوظيفة `input("prompt")`

الوظيفة `input` أثناء التنفيذ تقوم بعرض النص الموجود بين علامات التنصيص " " على الشاشة لمساعدة المدخل للبيانات علي معرفة المطلوب إدخاله من لوحة المفاتيح علي سبيل المثال `student_name = input("Enter your name : ")`, هذه رسالة للمستخدم لإدخال اسمه. حيث أن في هذه الحالة الكلمة `prompt` أخذت القيمة النصية: `Enter your name`. الجملة `input('prompt')` تقرأ القيمة المدخلة من لوحة المفاتيح كقيمة نصية وإسنادها إلى المتغير المكتوب معها في نفس السطر كالآتي: `student_number = input('enter_number:')` مثلاً لو أن المستخدم ادخل القيمة 12345 من لوحة المفاتيح فإن الجملة `input` ستسند هذه القيمة إلى المتغير `student_number` كقيمة نصية أي أن هذا المتغير سيحتوي على النص التالي '12345' وليس الرقم 12345 مثال3: برنامج يقرأ رقم القيد و اسم الطالب و المتوسط المتحصل عليه في الثانوية و عرضها على شاشة العرض.

```
def main():
    student_id = input("Enter student id : ")
    student_name = input("Enter student name : ")
    student_avg = input("Enter high school average : ")
    print("\nstudent id : ", student_id)
    print("\nstudent name : ", student_name)
    print("\nhigh school average : ", student_avg)

>>>main()
Enter student id : 123456789
Enter student name : Mohammed Ali
Enter high school average : 88.5
-----
student id : 123456789
student name : Mohammed Ali
high school average : 88.5
```

في هذا المثال تم تعيين 3 قيم تم إدخالها من لوحة المفاتيح لكل من :

1. قيمة رقمية صحيحة للمتغير `student_id` والذي يمثل رقم القيد.
2. قيمة نصية للمتغير `student_name` والتي تمثل اسم الطالب.
3. قيمة رقمية كسرية للمتغير `student_avg` والذي يمثل النسبة المئوية المتحصل عليها الطالب بالثانوية.

ملاحظات:

- من ضمن جملة النص بجملة `print` يوجد الرمز `\n` ويعني الانتقال إلى سطر جديد.

- رقم القيد والمتوسط تم تخزينها نصيا وليس رقميا بالإصدار Python 3.1.1 وسيتم توضيح السبب وكيفية تحويلها رقمية لاحقا.
 - تنسيق الجمل بداية من اليسار من عمود واحد وفي حالة الإخلال بالقاعدة يحدث خطأ لغوي (Syntax Error).
 - الخط العريض الداكن هو النص الموجود في جملة **input**
 - والبيانات ذات الخط غير الداكن تم إدخالها من قبل المستخدم
 - مابين علامات التنصيص المزدوجة الثلاثية ("" "") يعتبر جمل توضيحية لقارئ البرنامج وليس جمل يقوم الحاسوب بتنفيذها وبالإمكان استخدام اللغة العربية كما هو موضح بالمثل.
 - كما يمكن استخدام الرمز # في حالة أن الجملة التوضيحية لا تتجاوز سطر
- # This comment is written in a single line

ثانيا : إسناد قيم للمتغيرات بإستخدام جملة التعيين (Assignment statement)

- مثل جملة لإسناد القيمة النصية "programming language" للمتغير course_name نكتب الجملة التالية
course_name = 'programming language'
- مثلا لإسناد القيمة الرقمية 2013 إلى المتغير year نكتب الجملة التالية
year = 2013

تنبيه :

ما نوع القيمة التي تم تخزينها بالجملة التالية ؟
student_id = "123456789"
تم تخزينها كقيمة نصية ولا يمكن استخدامها في جمل حسابية إلا بعد تحويلها إلى قيمة رقمية من النوع الصحيح ويتم ذلك باستخدام الوظيفة (int student_id) وإعادة إسنادها لنفس المتغير أو متغير آخر.
Student_id = int(student_id)

2.4. العمليات التي تجرى على البيانات :

تنقسم العمليات التي تجرى على البيانات بحسب نوع المؤثر والتي تشمل الآتي:

• المؤثرات الحسابية Arithmetic Operators

رموز خاصة تمثل العمليات الحسابية (الجمع، الضرب ،). حيث أن طرفا المؤثر يطلق على كل واحد منهما معامل (operand). الجدول التالي يعرض المؤثرات الحسابية في لغة Python:

+	مؤثر الجمع addition
-	مؤثر الطرح subtraction
*	مؤثر الضرب multiplication
/	مؤثر القسمة division
//	مؤثر آخر للقسمة ويحذف الكسر من الناتج في حالة وجوده
%	مؤثر باقي القسمة remainder
**	مؤثر الأس exponentiation

المؤثر // تم استحدثه بالإصدار 3.0 فما فوق، ومن خصائصه أنه يتماشى مع المنطق الرياضي حيث أن ناتج عملية القسمة طبيعياً رقم كسري وليس صحيح وبالإمكان الحصول على الجزء الصحيح من الرقم حسب الطلب وليس ضمناً كما هو بالعديد من لغات البرمجة الأخرى.

جدول إسبقية تنفيذ المؤثرات الحسابية في تعابير حسابية

المؤثر	التنفيذ في حالة تساوي الأولوية
()	من اليسار إلى اليمين
**	من اليمين إلى اليسار
- مثلاً 3-	من اليسار إلى اليمين
%, //, /, *	من اليسار إلى اليمين
+, -	من اليسار إلى اليمين

• المؤثرات العلائقية Relational Operators

رموز خاصة تمثل العمليات العلائقية (المقارنة : أكبر من ، أصغر من ،). الجدول التالي يعرض المؤثرات العلائقية في لغة Python:

الرمز	المعنى	الرمز	المعنى
==	يساوي	>=	أكبر من أو يساوي
!=	لا يساوي	<=	أصغر من أو يساوي
>	أكبر من		
<	أصغر من		

• المؤثرات المنطقية Logical Operators

رموز خاصة تمثل العمليات المنطقية (الربط المنطقي بين التعابير الرياضية). الجدول التالي يعرض المؤثرات المنطقية في لغة Python:

and	الناتج يساوي True في حالة طرفي المؤثر قيمتهما True, عدا ذلك الناتج يساوي False
or	الناتج يساوي False في حالة كلا طرفي المؤثر قيمتهما False, عدا ذلك الناتج يساوي True
not	الناتج عكس المدخل : إذا المدخل True الناتج يكون False وإذا كان المدخل False الناتج يكون True

• التعابير الرياضية Mathematical Expressions

جمل تتكون من خليط من القيم والمتغيرات والمؤثرات (حسابية و علائقية و منطقية) بصيغة معينة للحصول على قيمة معينة. مثلاً من التعابير الحسابية جمع مقدارين رقميين أو مجموعة مقادير تحتوي على العديد من العمليات الحسابية.

الجدول التالي يبين كيفية تحويل تعبير رياضي إلى تعبير بلغة Python.

التعبير الرياضي	التعبير بلغة Python
$Q + P$	$Q + P$
$Q - P$	$Q - P$
$Q \times P$	$Q * P$
$\frac{Q}{P}$	Q / P
Q^P	$Q ** P$
$\sqrt{Q}$	$Q ** 0.5$

- استخدم المؤثر / لعملية القسمة حتي تتمكن من كتابة التعابير التي تحتوي علي عملية قسمة بسطر واحد فمثلا : $\frac{-b}{2 \times a}$ تكتب كالاتي $-b / (2 * a)$
- وكذلك استخدام الرمز ** لعملية الأس يمكننا من كتابة التعبير التالي Q^P الى الشكل التالي $Q ** P$ وبذلك نتمكن من كتابة التعابير التي تحتوي علي عمليات أسية بسطر واحد.

• تحويل تعابير جبرية إلى تعابير مكافئة بلغة PYTHON.

Algebra: $m = \frac{a + b + c + d + e}{5}$

Python: $m = (a + b + c + d + e) / 5$

Algebra: $y = mx + b$

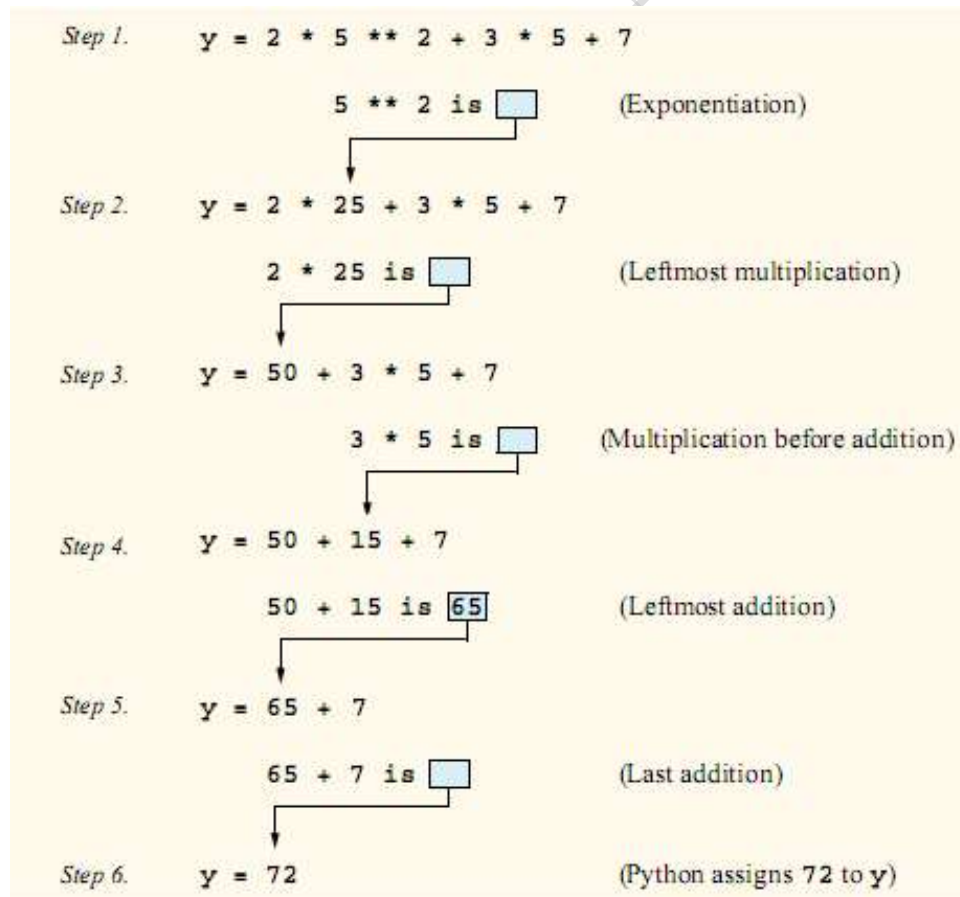
Python: $y = m * x + b$

2.5. تقييم التعابير الرياضية :

1. التعابير الحسابية :

يتم حساب الناتج النهائي لأي تعبير رياضي وفق أولويات محددةضمنية بلغة البرمجة علي سبيل المثال :

- التعبير الرياضي التالي $y = 2 * 5 ** 2 + 3 * 5 + 7$ يتم حساب قيمته كالاتي:



- التعبير الرياضي التالي $z = pr \% q + w / x - y$ يتم حساب قيمته كالآتي:

Algebra: $z = pr \% q + w / x - y$

Python: $z = p * r \% q + w / x - y$

1
2
4
3
5

- التعبير الرياضي التالي $y = a * x ** 2 + b * x + c$ يتم حساب قيمته كالآتي:

$y = a * x ** 2 + b * x + c$

2
1
4
3
5

أمثلة لحساب النتيجة النهائية لتعابير حسابية :

النتيجة	التعبير الحسابي
6	$5 * 6 - 24$
7.0	$(12 + 23) / 5$
1.5	$6 / 4$
1.5	$6. / 4.$
1	$6 // 4$
1.	$6. // 4.$
256	$2 ** 2 ** 3$
-9	$-3 ** 2$
-9	$-(3 ** 2)$
9	$(-3) ** 2$
0.25	$2.75 \% 0.5$

تدريب : احسب ناتج التعابير الحسابية التالية:

التعبير الحسابي
$5. / 2.0 * 4.0 + 7. - 2 ** 3$
$5. ** 2 / 4 * 8.$
$5. + 6. / 3. + 2 * 3$
$30. // (5 * 2 + 2)$
$2 ** 3 ** 2 - 2$

2.6. التعبيرات العلائقية Relational Expressions

العبارة العلائقية هي العبارة القابلة للصواب True أو الخطأ False فمثلا العبارة : $4 < 5$ هي عبارة علائقية وقيمتها الصواب True. والعبارة : $3 == 4$ هي أيضا هنا علائقية وقيمتها False. ويجب أن نلاحظ أن معاملا المؤثر العلائقي قد تكون تعابير رياضية أو يكونا من النصوص.

المؤثر	المعنى
$X > Y$	X أكبر من Y
$M < N$	M اصغر من N
$X == Y$	X تساوي Y
$P != Q$	Q لا تساوي P
$Z >= W$	Z أكبر من أو تساوي W
$A <= B$	A أصغر من أو تساوي B

مثال 4 : ماذا يطبع البرنامج التالي ؟

```
#Use of relational operators
def main():
    x = 5 > 4
    y = 6 < 2
    z = 8 >= 7
    print('\n x =', x ,', y =', y, ', z =', z )
```

ناتج البرنامج :

x = True , y = False , z = True

ماذا يطبع البرنامج التالي :

```
def main():
    k = 8 > 7 > 6
    m = 5 < 1 + 2
    n = 3 * 2 > 5
    print('\n k =', k, ', m =', m, ', n =', n)
```

ناتج البرنامج :

k = True , m = False , n = True

2. التعابير المنطقية Logical Expressions

العبرة المنطقية هي العبرة القابلة للصواب True أو الخطأ False فمثلا العبرة : (True and False) هي عبرة منطقية وقيمتها خطأ False. والعبرة : (3 == 4 or 9 > 8) هي أيضا هنا منطقية وقيمتها True. ويجب أن نلاحظ أن معاملا المؤثر المنطقي قد تكون تعابير منطقية أو قد يكونا تعابير علائقية.

مثال 5 : ماذا يطبع البرنامج التالي ؟

```
#Use of logical and relational operators
def main():
    x = 5 > 4
    y = 6 < 2
    z = x and y
    print('x =', x, 'y =', y, 'z =', z)
```

ناتج البرنامج :

x = True , y = False , z = False

ماذا يطبع البرنامج التالي :

```
def main():
    k = 8 > 7 > 6
    m = 5 < 1 + 2
    n = 3 * 2 == 6
    y = k or m and n
    print('k =', k, 'm =', m, 'n =', n, 'y =', y)
```

ناتج البرنامج :

k = True , m = False , n = True , y = True

2.8. كتابة خوارزمية (برنامج) لبعض المسائل بلغة Python

مثال 6 : إعادة المثال 1 باستخدام لغة Python

```
def main():
    pi = 3.14159
    radius = input("Enter radius : ")
    radius = int(radius)
    area = radius * radius * pi
    circumference = 2 * radius * pi
    print(area, circumference)
```

وكما وضعنا سابقا يمكن إدخال تحسينات علي البرنامج السابق في طريقة عرض النتائج ليكون أكثر وضوحا باستبدال جملة الطباعة بالجملة التالية.

```
print('area =', area, 'circumference =', circumference)
```

حيث أن هذه الجملة ستعرض مخرجات البرنامج كالآتي:

area = xxxxxx circumference = yyyyyy

حيث أن xxxxxx و yyyyyy هما القيمتان المخزنتان في المتغيران area و circumference على الترتيب.

مثال 7 : أكتب برنامج لقراءة قيمة لدرجة الحرارة بالـ Fahrenheit وتحويلها إلى المئوية المعروف عالمياً بـ "Celsius".

التحليل:

1. أدخل الدرجة من لوحة المفاتيح لمتغير نختار له الاسم ftemp.

2. تحويل من Fahrenheit إلى Celsius باستخدام المعادلة:

$$ctemp = 5 / 9 * (ftemp - 32.0)$$

3. عرض درجة الحرارة ctemp.

البرنامج كالآتي:

```
## To convert temperature from Fahrenheit to Celsius
def main():
# ftemp درجة الحرارة بالـ Fahrenheit يشار لها بالمتغير
# ctemp درجة الحرارة بالـ Celsius يشار لها بالمتغير
    ftemp = input("Fahrenheit temperature : ")
    ctemp = (ftemp - 32) * 5.0 / 9.0
    print("5.0 / 9.0 * (" ,ftemp,"- 32.0) = " ,ctemp, "Celsius")
```

بتنفيذ البرنامج ينتج الآتي:

نلاحظ من الأسطر المعروضة على الشاشة وجود خطأ أثناء محاولة تنفيذ البرنامج. (كما هو مبين في الشكل التالي:

```
>>> main()
Fahrenheit temperature : 45
Traceback (most recent call last):
File "<pyshell#3>", line 1, in <module>
    main ()
File "C:\Python31\tempconvert", line 3, in main
    ctemp = (ftemp - 32) * 5.0 / 9.0
TypeError: unsupported operand type(s) for -: 'str' and 'int'
```

تحليل الخطأ :

• الخطأ حدث بالسطر 3 من الجمل التنفيذية :

$$ctemp = (ftemp - 32) * 5.0 / 9.0$$

• الخطأ منطقي حيث أن أثناء التنفيذ وإدخال القيمة الرقمية 45 من لوحة المفاتيح تم إسنادها كقيمة نصية أي "45" إلى

المتغير ftemp فبالتالي لا يسمح بإجراء عملية حسابية بين قيمة نصية للمتغير ftemp والقيمة الثابتة 32

تصحيح الخطأ: استخدم الوظيفة float لتحويل قيمة المتغير ftemp النصية إلى قيمة رقمية كالتالي:

$$ctemp = (float(ftemp) - 32) * 5.0 / 9.0$$

```
# To convert temperature from Fahrenheit to Celsius
```

```
def main():
```

```
    # ftemp درجة الحرارة بالـ Fahrenheit يشار لها بالمتغير
```

```
    # ctemp درجة الحرارة بالـ Celsius يشار لها بالمتغير
```

```
    ftemp = input("Fahrenheit temperature : ")
```

```
    ctemp = (float(ftemp) - 32) * 5.0 / 9.0
```

```
    print("5.0 / 9.0 * (" , ftemp, " - 32.0) = " , ctemp, "Celsius")
```

الأرقام المكتوبة في هذا الشكل  تبين أن هذه الجملة هي تطبيق للخطوة المناظرة لها في التحليل. أعد تنفيذ البرنامج بإدخال عدد من القيم المعلومة النتيجة مسبقا وقيم عشوائية للتأكد من خلو البرنامج من الأخطاء المنطقية والتنفيذية. على سبيل المثال:

Ctemp	Ftemp
0.0	32
-40.0	-40
100.0	212
37.0	98.6

مثال 8 : اكتب برنامجا يقوم بقراءة الزمن بالثواني ويحسب عدد الثواني والدقائق والساعات في هذا الزمن .

التحليل:

قبل أن نكتب البرنامج المطلوب، دعنا نحسب يدويا الثواني والدقائق والساعات في زمن قدره مثلا 10000 ثانية. الخطوة الأولى في هذا الحساب هي قسمة 10000 على 60 لنحصل على 166 دقيقة و الباقي 40 ثانية. ثم نقوم بقسمة 166 على 60 لنحصل على 2 ساعة والباقي 46 دقيقة. أي أن :

عدد الساعات = 2

وعدد الدقائق = 46

وعدد الثواني = 40

بصورة عامة فان المعطيات هي الزمن بالثواني ويشار إليه بالمتغير sec

والمطلوب حساب :

عدد الساعات ولنمثلها بالمتغير hours وعدد الدقائق ولنمثلها بالمتغير minutes وعدد الثواني ولنمثلها بالمتغير seconds

الخوارزمية :

1. ادخل قيمة الزمن بالثواني واحفظه في المتغير sec
2. اقسم قيمة sec على 60 كالاتي $sec // 60$ وبذلك نحصل على عدد الدقائق في sec. قم بتخزينه في المتغير min
(العملية // تعطي خارج القسمة)
3. اقسم قيمة sec على 60 كالاتي $sec \% 60$ وبذلك نحصل على عدد الثواني في sec. قم بتخزينه في المتغير seconds
(العملية % تعطي باقي القسمة)
4. اقسم قيمة min على 60 كالاتي $min // 60$ وبذلك نحصل على عدد الساعات في min. قم بتخزينه في المتغير hours
5. اقسم قيمة min على 60 كالاتي $min \% 60$ وبذلك نحصل على عدد الدقائق في min. قم بتخزينه في المتغير minutes
6. قم بطباعة hours و minutes و seconds

لاحظ أن قيم المتغيرات يجب أن تكون كلها من النوع الصحيح وحتى يكون ناتج القسمة صحيحا يجب استعمال // بدلا من /، أما الباقي فيمكن حسابه باستخدام المؤثر %

```
#Compute number of hours, minutes and seconds in time given in
seconds
... 1
def main():
    sec = input('Enter time in seconds : ')
    # تحويل قيمة المتغير sec إلى قيمة رقمية من النوع الصحيح
    sec = int(sec) ... 2
    min = sec // 60 ... 3
    seconds = sec % 60 ... 4
    hours = min // 60 ... 5
    minutes = min % 60
    print('hours = ', hours, ', minutes = ', minutes, ', seconds = ',
    seconds)
```

نفذ البرنامج بإدخال:

```
>>> main()
```

```
Enter time in seconds : 10000
```

```
-----
Hours = 2 , minutes = 46 , seconds = 40
```

أعد تنفيذ البرنامج بإدخال:

```
>>> main()
```

Enter time in seconds : 12015

Hours = 3 , minutes = 20 , seconds = 15

سبقاً وأنّ وضعنا أن بعض المسائل لها أكثر من طريقة للحل. في المثال السابق يمكن استبدال جمل التحويل كالتالي:

```
hours = sec // 3600
```

```
sec= sec % 3600
```

```
minutes = sec// 60
```

```
seconds= sec % 60
```

مثال 9: برنامج يقوم بقراءة قيم لأضلاع المثلث ويشار إليهم بالمتغيرات a, b, c وحساب المساحة area باستخدام القانون:

$$s = \frac{a+b+c}{2}$$

$$area = \sqrt{s \times (s-a) \times (s-b) \times (s-c)}$$

الخوارزمية :

1. إقرأ قيم أضلاع المثلث في المتغيرات التالية a, b, c

2. إحسب قيمة مساحة المثلث كالتالي:

$$s = \frac{a+b+c}{2} \quad \circ$$

$$area = \sqrt{s * (s-a) * (s-b) * (s-c)} \quad \circ$$

3. اطبع قيمة المساحة

مؤقتاً نفرض أن قيم الاضلاع تمثل مثلث على أن يتم تعديل البرنامج بالدروس اللاحقة.

```
def main():
```

```
    a= input("Enter side A : ")
```

```
    a=float(a)
```

```
    b= input("Enter side B : ")
```

```
    b=float(b)
```

```
    c= input("Enter side C : ")
```

```
    c=float(c)
```

```

s = (a + b + c) / 2
area = (s * (s - a) * (s - b) * (s - c)) ** 0.5
print("area of triangle with sides", a, b, c, " is", area)

```

مثال 10: أكتب برنامج يقوم بقراءة 3 قيم تمثل أضلاع مثلث ويقوم بطباعة True في حالة أن القيم لأضلاع مثلث قائم الزاوية. المثلث قائم الزاوية في حالة إذا وجد أن مجموع مربعي ضلعين يساوي مربع الضلع الثالث.

الخوارزمية :

1. إقرأ قيم أضلاع المثلث في المتغيرات التالية a, b, c
2. قيمة هذا التعبير $a^2 = b^2 + c^2$ or $b^2 = a^2 + c^2$ or $c^2 = a^2 + b^2$ تكون True إذا كان المثلث قائم الزاوية و False غير ذلك
3. اطبع قيمة المتغير is_right_triangle

```

def main():
    a= float(input("Enter side A : "))
    b= float(input("Enter side B : "))
    c= float(input("Enter side C : "))
    is_right_triangle = a*a == b*b + c*c or b*b == a*a + c*c or c*c == a*a + b*b
    print("is right triangle ", is_right_triangle)

```

للتأكد من صحة البرنامج استخدم على سبيل المثال الاحتمالات التالية :

الناتج	C	B	A
True	3	4	5
True	10	8	6
True	5	3	4
False	6	3	4
False	3	2	1

مثال 11: لنفرض أنه طلب منك تتبع البرنامج السابق. هذا يعني قم بتنفيذ البرنامج جملة جملة مستعيناً بورقة ترسم عليها جدولاً كل عمود يمثل متغيراً من متغيرات البرنامج وكل صف يبين القيم التي يأخذها المتغير خلال مراحل التنفيذ. والجدول الآتي يبين هذا الأسلوب للبرنامج السابق (مثال 15)

لنفرض أن المدخلات له البرنامج هي : $a = 3$, $b = 4$, $c = 5$

a	b	c	is_rigth_triangle	<i>print</i> ("is right triangle ", is_rigth_triangle)
3	4	5	True	is right triangle True

2.8. المؤثرات النصية: وتشمل " + و * "

نلاحظ أن المؤثران " + و * " إي علامتي الجمع والضرب يستخدمنا للتعامل مع القيم النصية، إلا أنه لن يقوموا بعمليتي الجمع والضرب. والعمليات المسموح بها:

المؤثر	مثال	الناتج
+	'Hello, ' + 'World!'	'Hello, World!'
*	'Hello, ' * 3	'Hello, Hello, Hello'
*	3 * 'Hello, '	'Hello, Hello, Hello'

مثال 12: نفذ المثال التالي:

```
def main():
    fname = input('Type in your first name : ')
    lname = input('Type in your last name : ')
    full_name = fname + ' ' + lname
    print ('Hello, ', full_name)
```

عند تنفيذ البرنامج يظهر الآتي:

```
Type in your first name : Ali
Type in your last name : Omar
Hello Ali Omar
```

مثال 13: نفذ المثال التالي:

```
def main():
    num = input('Type in an integer number : ')
    num = int(num)
    str = input('Type in a string : ')
    print (str, '*', num, '=', str * num)
```

عند تنفيذ البرنامج يظهر الآتي:

```
Type in an integer number : 3
Type in a string : hello
Hello * 3 = hellohellohello
```

توجيه: قبل الشروع في التدريب العملي يجب علي الطالب
تكوين مجلد بأسم CS100 لتخزين البرامج التي تقوم بتنفيذها علي الحاسوب.

1. أدخل البرنامج السابق وتخزينه وتنفيذه علي الحاسوب.
2. إلغاء \n من جملة الطباعة وإعادة تنفيذ البرنامج. ماذا تلاحظ عند التنفيذ ؟
3. سحب أحد جمل القراءة خانة إلباليسار أو اليمين وإعادة تنفيذ البرنامج.
4. أكتب برنامج لحساب حجم الكرة بمعرفة نصف القطر.
5. أكتب برنامج لقراءة قيمة درجة الحرارة بالـ Celsius وتحويلها إلبالملكافى بالـ "Fahrenheit"
6. أكتب برنامج يقوم بقراءة الزمن بالثواني والدقائق والساعات ويشار إليهم بالمتغيرات hours, minutes, seconds وتحويله إلبالثنواني sec.
7. أكتب برنامج يقوم بقراءة بعدي المستطيل (الطول، العرض) وعرض قيمتي المساحة والمحيط
8. أكتب برنامج يقوم بقراءة أبعاد أضلاع مثلث وعرض True في حالة أن القيم تمثل أظلاع مثلث وعرض False في حالة أن القيم الـ 3 لا تمثل أضلاع مثلث. القيم الـ 3 تمثل مثلث في حالة أن مجموع أي ضلعين أكبر من قيمة الضلع الثالث.
9. أكتب برنامج لإدخال من لوحة المفاتيح قيمة صحيحة وعرض على الشاشة True في حالة أن القيمة المدخلة عدد فردي وعرض False في أن القيمة المدخلة عدد زوجي

3- جمل التحكم (الجمل الشرطية) Conditional Statements

في هذا الفصل سنعرض مجموعة من المسائل نحتاج فيها لاختبار شرط (أو شروط) وعلى نتيجة الاختبار يتم تنفيذ بعض الخطوات وترك أخرى مع بساطة في تراكيب البيانات. وقبل البدء في حل هذا النوع من المسائل سنقدم بعض مفردات لغة Python التي ستساعدنا على كتابة حلول لهذه المسائل.

2.1. الجمل الشرطية

من أساسيات البرمجة الجمل الشرطية والتي تسمح باختيار تنفيذ جملة أو مجموعة من الجمل مرة واحدة في حالة تحقق شرط معين، وعدم تنفيذ جملة (أو جمل) أخرى. سنقوم في هذا الفصل بعرض الجملة الشرطية **if** على عدة مراحل حتى نصل إلى الشكل العام لهذه الجملة.

1. الجملة الشرطية ذات تفرع واحد:

if (relational_expression) :

```
|  
| S1  
| S2  
| .  
| .  
| Sn  
Sn+1
```

وتعني تنفيذ الجملة أو مجموعة الجمل $S_1, S_2, \dots, S_n$ في حالة أن قيمة التعبير العلائقي (relational_expression) يساوي True. ثم متابعة التنفيذ بداية من الجملة S_{n+1} , وإذا كان الشرط قيمته False فإن التنفيذ يبدأ من الجملة S_{n+1} ولا يتم تنفيذ الجمل $S_1, S_2, \dots, S_n$. يجب إزاحة الجمل $S_1, S_2, \dots, S_n$ عدد من الفراغات إلى اليمين، والمتعارف عليه (4 فراغات) لأن هذا الجمل تندرج تحت الجملة **if**. وتكون الجملة S_{n+1} في نفس مستوى **if**.

مثال 1 : برنامج لإيجاد وطباعة أكبر القيمتين واللذان يتم ادخالهما من لوحة المفاتيح باستخدام الجملة الشرطية ذات تفرع واحد.

الخوارزمية:

1. اراء القيمتان من لوحة المفاتيح ولمنمثلة القيمة الأولى بالمتغير value1 والمتغير الثاني بالمتغير value2
2. لنفرض أن أكبر قيمة هي value1 ولنخزنها في المتغير max
3. نقارن القيمة الثانية بالمتغير max فإذا كانت أكبر من قيمة max فإننا نخزن قيمة المتغير value2 في المتغير max وبذلك تكون هي أكبر قيمة، وإلا فستبقى قيمة max كما هي أي أن value1 هي أكبر قيمة.
4. اطبع قيمة max

```
# برنامج لإيجاد وطباعة أكبر القيمتين واللذان يتم ادخالهما من لوحة المفاتيح.
def main():
    value1 = input('Enter 1st value : ')
    value2 = input('Enter 2nd value : ')
    max = value1          # assume value1 is the maximum so far
    if (value2 > max) :   # if value2 > max is True assign value2 to max
        max = value2
    print('The largest of the 2 values is ', max)
```

نفذ البرنامج بإدخال عدد من القيم للتأكد من خلو البرنامج من الأخطاء المنطقية والتنفيذية. على سبيل المثال:

max	value2	value1
10	10	5
5	-5	5
6	6	6
ali	ahmed	Ali
محمد	احمد	محمد
محمد	10457	محمد

ملاحظة: القيم التي تم مقارنتها قيم نصية ولا يسمح بمقارنة قيم رقمية ونصية إلا بعد تحويل القيمة الرقمية إلى قيم نصية أو العكس باستخدام الآتي:

للتذكير :

لتحويل قيمة نصية إلى رقمية صحيحة
string = '54' فإن **int(string)** ستخزن قيمة عددية صحيحة قيمتها 54 في المتغير **var**
 لتحويل قيمة نصية إلى رقمية كسرية (عدد حقيقي)
string = '54.67' فإن **float(string)** ستخزن قيمة عددية كسرية قيمتها 54.67 في المتغير **var**
 لتحويل قيمة رقمية إلى نصية
numeric = 54 فإن **str(numeric)** ستخزن نص قيمته '54' في المتغير **var**

مثال 2 : إعادة برنامج حساب متوسط درجات عدد **n** من الطلبة بمقرر cs100 مع حساب عدد الطلبة الناجحين و عدد الطلبة الراسبين. و يشار لعدد الطلبة الناجحين بـ **passed**

```
# برنامج لحساب متوسط درجات عدد n من الطلبة وعدد الطلبة الناجحين والراسبين
def main():
    sum = 0
    count = 0
    passed = 0
    no_students = input('Enter No. of students : ')
    no_students = int(no_students)
    while (count < no_students) :
        grade = input('Enter student Grade : ')
        grade = float(grade)
        sum = sum + grade
        count = count + 1
        if (grade >= 50):
            passed = passed + 1
    average = sum / no_students
    print("\nAverage grade = ", average)
    print("\nNo of students passed = ", passed)
    print("\nNo of students failed = ", no_students - passed)
```

ملاحظة: يمكن ضم الجمل الثلاث الأولى كالتالي: $sum = count = passed = 0$

2. الجملة الشرطية ذات تفرعين

if (relational_expression):

S_1

S_2

.

.

.

S_j

else :

S_{j+1}

S_{j+2}

.

.

.

S_n

S_{n+1}

وتعني تنفيذ الجملة أو مجموعة الجمل $S_1, S_2, \dots, S_i$ في حالة أن قيمة التعبير العلائقي (relational_expression) يساوي True. ثم متابعة التنفيذ بداية من الجملة S_{n+1} , وإذا كان الشرط قيمته False فإن مجموعة الجمل التي تحت سيطرة الجملة else وهي $S_{i+1}, S_{i+2}, \dots, S_n$ تنفيذ ولا يتم تنفيذ الجمل $S_1, S_2, \dots, S_i$. ثم متابعة التنفيذ بداية من الجملة S_{n+1}

مثال 3: برنامج لإيجاد وطباعة أكبر القيمتين واللذان يتم ادخالهما من لوحة المفاتيح باستخدام الجملة الشرطية ذات تفرعين.

الخوارزمية:

1. اقاء القيمتان من لوحة المفاتيح ولمثل القيمة الأولى بالمتغير value1 والمتغير الثاني بالمتغير value2
2. إذا كانت القيمة value1 أكبر من القيمة value2 فإن المتغير max سيحتوي على قيمة value1 وإلا فإن المتغير max سيحتوي على قيمة value2
3. اطبع قيمة max

برنامج لإيجاد وطباعة أكبر القيمتين واللذان يتم ادخالهما من لوحة المفاتيح باستخدام الجملة الشرطية ذات تفرعين.

```
# الشرطية ذات تفرعين.
def main():
    value1 = input('Enter 1st value : ')
    value2 = input('Enter 2nd value : ')
    if (value1 > value2) :
        max = value1
    else :
        max = value2
    print ('The largest of the 2 values is ', max)
```

مثال 4: برنامج لتحديد وعرض نوع القيمة الصحيحة التي يتم إدخالها من لوحة المفاتيح (زوجي - even ، فردي - odd).

الخوارزمية:

1. اقاء القيمة من لوحة المفاتيح ولمثلها بالمتغير value
2. إذا كانت باقي قسمة القيمة المتغير value على العدد 2 تساوي 0 فاطبع أن العدد زوجي وإلا فاطبع أن العدد فردي

```
# برنامج لمعرفة وتحديد وعرض نوع القيمة الصحيحة التي يتم إدخالها من لوحة المفاتيح،
# (زوجي - even ، فردي - odd).
def main():
    value = int(input('Enter an integer value : '))
    if (value % 2 == 0) :
        print ( value, 'is even')
    else :
        print ( value, 'is odd')
```

إي الطريقتين أفضل الجملة الشرطية ذات تفرع واحد أو تفرعين ؟

مثال 5 : برنامج يقوم بقراءة قيم لاضلاع مثلث ويشار إليهم بالمتغيرات a , b , c من لوحة المفاتيح وحساب مساحة المثلث في حالة أن القيم المدخلة تمثل أضلاع مثلث، وفي حالة أن القيم المدخلة لا تمثل أضلاع مثلث أعرض الجملة التالية :

The 3 values do not represent sides of triangle.

$$s = \frac{a+b+c}{2}$$

$$area = \sqrt{s \times (s-a) \times (s-b) \times (s-c)}$$

```
"""
    برنامج لحساب مساحة مثلث في حالة أن الأضلاع تمثل مثلث
    """
def main():
    a = float(input('Enter side1: '))
    b = float(input('Enter side2: '))
    c = float(input('Enter side3: '))
    s = (a + b + c) / 2
    w = s * (s - a) * (s - b) * (s - c)
    if (w > 0) : # الأضلاع تمثل مثلث
        area = w ** 0.5
        print ('Area of triangle with sides', a, b, c, 'is', area )
    else : # ليس مثلث
        print (' The 3 values do not represent sides of triangle . .')
```

```
def main():
    a= float(input('Enter side1: '))
    b= float(input('Enter side2: '))
    c= float(input('Enter side3: '))
    if (a < b + c and b < a + c and c < a + b): # الاضلاع تمثل مثلث
        s = (a + b + c) / 2
        w = s * (s - a) * (s - b) * (s - c)
        area = w ** 0.5
        print('Area of triangle with sides', a, b, c, 'is', area)
    else: # ليس مثلث
        print('The 3 values do not represent sides of triangle ...')
```

3. الجملة الشرطية متعددة التفرع:

```
if (relational_expression1):
    S1,1
    S1,2
    .
    .
    S1,u
elif (relational_expression2):
    S2,1
    S2,2
    .
    .
    S2,k
elif (relational_expressionx):
    Sx,1
    Sx,2
    .
    .
    Sx,m
else:
    Sm+1
    Sm+2
    .
    .
    Sn
Sn+1
```

وتعني في حالة تحقق الشرط. أي أن قيمة relational_expression₁ هي **True**، سيتم تنفيذ الجمل S_{1,1}, S_{1,2}, ..., S_{1,u}. ثم متابعة التنفيذ بداية من الجملة S_{n+1}، ولا يتم تنفيذ الجمل الأخرى. وإذا كان الشرط قيمته False فإنه يتم الانتقال إلى **elif** التالية فإذا كان الشرط قيمته **True** سيتم تنفيذ الجمل S_{2,1}, S_{2,2}, ..., S_{2,k}. ثم متابعة التنفيذ بداية من الجملة S_{n+1}، ولا يتم تنفيذ الجمل الأخرى. وإذا كان الشرط قيمته False فإنه يتم الانتقال إلى **elif** التالية بنفس الكيفية. وإذا كانت كل الشروط قيمتها **False** سيتم تنفيذ جمل الجملة **else** (S_n, S_{m+2}, ..., S_{m+1}). ثم متابعة التنفيذ بداية من الجملة S_{n+1}، ولا يتم تنفيذ الجمل الأخرى. الجمل المحصورة بين القوسين [] تعتبر جمل إختيارية، أي أن مصمم الحل مخير في إستعمالها.

مثال 6 : برنامج لتحديد وعرض نوع القيمة الصحيحة التي يتم إدخالها من لوحة المفاتيح
(موجب – positive ، سالب – negative ، صفر – zero).

الخوارزمية :

1. إقرأ القيمة العددية ولنرمز له بالرمز value
2. إذا كانت قيمة المتغير value أكبر من 0 فاطبع أن العدد موجب واخرج من البرنامج وإلا إذهب للنقطة 3
3. إذا كانت قيمة المتغير value أصغر من 0 فاطبع أن العدد سالب واخرج من البرنامج وإلا إذهب للنقطة 4
4. اطبع أن العدد صفر واخرج من البرنامج

```
if (relational_expression1):  
def main():  
    value = float(input('Enter an integer number: '))  
    if (value > 0) :  
        print(value, 'is a positive number')  
    elif ( value < 0 ) :  
        print(value, 'is a negative number')  
    else :  
        print('The number is Zero')
```

تمارين

1. أكتب برنامج لقراءة 3 قيم ولنشر لهم بـ a, b, c وترتيبهم ترتيباً تصاعدياً.
2. أكتب برنامج يقوم بإيجاد مجموع أعلي درجتين من ثلاث امتحانات.
3. أكتب برنامج يقوم بقراءة من لوحة المفاتيح عدد n من القيم الحقيقية ولنشر للقيمة بـ val وأن كل قيمة تحقق الشرط $10 > =$ (val $> = 5$). المطلوب حساب وعرض متوسط القيم بعد استبعاد أكبر وأصغر قيمة من القيم المدخلة.
4. أكتب برنامجاً بلغة Python لحل المسألة الآتية:

$$y = \begin{cases} 1 - x & \text{if } x \leq -1 \\ x^2 & \text{if } x \geq 1 \\ \sqrt{(x-1)^5} & \text{if } -1 < x < 1 \end{cases}$$

5. اكتب برنامجاً بلغة Python يقرأ عدد n من الأرقام ويطبّع أكبر عدد وآخر موقع يوجد به العدد مثلاً: 1 5 3 2 0 1 5 4 أكبر عدد في هذه الأعداد هو الرقم 5 وآخر موقع له هو 7. فهذا البرنامج سيطبّع 5 7
6. اكتب برنامجاً بلغة Python يقرأ عدد n من الأرقام ويطبّع أصغر عدد وأول موقع يوجد به العدد مثلاً: 1 5 3 2 0 1 5 4 أصغر عدد في هذه الأعداد هو الرقم 1 وآخر موقع له هو 1. فهذا البرنامج سيطبّع 1 1
7. برنامج لإيجاد وطباعة أكبر قيمة من 3 قيم يتم ادخالهما من لوحة المفاتيح.
8. برنامج لإيجاد وطباعة أكبر قيمة من 4 قيم يتم ادخالهما من لوحة المفاتيح.
9. اكتب برنامج لحساب المدى لعينة تحتوي على n من القيم الصحيحة الموجبة والسالبة ولنشر للقيمة بـ X والتي يتم قراءتها من لوحة المفاتيح. (المدى: هو الفرق بين أكبر وأصغر قيمة بالعينة).

4- جمل التكرار الخطوات (الحلقات) Loops

في هذا الفصل سنعرض مجموعة من المسائل نحتاج في كتابة خوارزمية حلها لتكرار بعض الخطوات مع بساطة في تراكيب البيانات. وقبل البدء في كتابة خوارزميات هذا النوع من المسائل سنقدم بعض مفردات لغة Python التي ستساعدنا على كتابة حلول لهذه المسائل.

3.1. جمل التكرار

من أساسيات البرمجة جمل تسمح بتكرار تنفيذ الخطوات. وتنقسم جمل التكرار (سنطلق عليها حلقات التكرار) إلى الأنواع التالية:

- طالما تحقق شرط
- إلى أن يتحقق شرط
- عدد محدد من المرات

وسنقتصر في هذا الفصل على حلقة طالما تحقق شرط

1. حلقة **while** (حلقة طالما تحقق شرط)

وتعني تنفيذ الجملة أو مجموعة الجمل $S_1, S_2, \dots, S_n$ عدد من المرات طالما قيمة التعبير العلائقي (relational expression) يساوي True، مع وجوب إزاحة الجملة أو مجموعة الجمل عدد من الفراغات إلى اليمين، والمتعارف عليه (4 فراغات). المستوى.

القاعدة العامة:
while relational-expression:

S_1
 S_2
.
.
.
 S_n

مثال 1: أكتب ونفذ برنامج لعرض الأعداد من 0 إلى n. على سبيل المثال، في حالة $n = 10$. فإن البرنامج سيعرض (سيطبع) السطر التالي:

0 1 2 3 4 5 6 7 8 9 10

التحليل

المعطيات : إدخال قيمة n من لوحة المفاتيح

المخرجات : عرض الأعداد من 0 إلى n من اليسار إلى اليمين.

الخطوات : للانتقال من 0 إلى n نحتاج إلى عداد ولنختار له الاسم count يبدأ من 0 وطباعة قيمة العداد ، ثم إضافة 1 إلى العداد count وطباعة قيمة العداد مرة أخرى. وهكذا إلى أن تصل قيمة n أكبر من 10 فيتوقف البرنامج، أي نخرج من حلقة while.

نلاحظ أن الخطوات (طباعة العداد، إضافة 1 إلى العداد) تتكرر طالما أن قيمة العداد لا تتجاوز قيمة n . يمكن كتابة هذه الخطوات كالآتي :

```
count = 0
while (count <= n) :
    print(count)
    count = count + 1
```



```
# برنامج لعرض الأعداد الصحيحة من 0 إلى n
def main():
    n = input('Enter an integer value : ')
    n = int(n)
    count = 0
    while (count <= n) :
        print(count)
        count = count + 1
```

عند تنفيذ البرنامج نلاحظ أن البرنامج يعرض الأعداد عموديا وليس أفقيا. ولعرض الأعداد أفقيا استبدل جملة الطباعة **print** (count) بـ **print** (count, end = ' '). تمت إضافة ' ' end = ' ' بجملة الطباعة حتي نمنع الانتقال إلى سطر جديد عند تنفيذ جملة الطباعة كل مرة.

ليصبح البرنامج كالتالي:

```
def main():
    n = input('Enter an integer value : ')
    n = int(n)
    count = 0
    while (count <= n) :
        print(count, end = ' ')
        count = count + 1
```

تمرين : ما الذي يطبعه البرنامج السابق في حالة تم استبدال جملتي

```
print(count)
count = count + 1

count = count + 1
print(count)
```

في الترتيب على النحو التالي:

مثال 2 : أكتب برنامج لحساب مضروب عدد يتم إدخاله من لوحة المفاتيح ولنشير للعدد بـ n والمضروب بـ $n!$.

من المعروف أن المضروب لعدد n يساوي حاصل ضرب الأعداد من 1 إلى n ويشار له رياضيا بـ $n!$ أي أن:

$$n! = 1 \times 2 \times 3 \times \dots \times (n-1) \times n$$

التحليل:

المعطيات :

- أدخل قيمة n من لوحة المفاتيح

المعالجة :

- القانون لحساب المضروب $n! = 1 \times 2 \times 3 \times \dots \times (n-1) \times n$ ولحساب مضروب العدد n كالآتي:

a. لنفرض أن قيمة العدد $k = 1$ (قيمة العدد تمثل قيكة العدد الحالي)

b. نبدأ أولاً بفرض أن قيمة المضروب تساوي 1 (ولنرمز له بالرمز $\text{factotrial} = 1$)

c. نكرر خطوات الفقرة c طالما k أصغر من أو تساوي n

• قيمة المضروب الحالية = قيمة المضروب السابقة $\times$ قيمة العدد الحالية

• نزيد قيمة العدد بواحد ($k = k + 1$)

d. اطبع قيمة مضروب n

المطلوب : عرض قيمة المضروب علي سبيل المثال في حالة أن قيمة $n = 5$ على الشكل التالي:

Factorial of 5 is 120

```
# برنامج لحساب وعرض مضروب عدد يتم إدخاله من لوحة المفاتيح ويشار له بـ n
def main():
    n = input('Enter an integer value : ')
    n = int(n)
    k = 1
    factorial = 1
    while (k <= n) :
        factorial = factorial * k
        k = k + 1
    print('Factorial of', n, 'is', factorial)
```

مثال 3: لنفرض أنه طلب منك تتبع البرنامج السابق. هذا يعني قم بتنفيذ البرنامج جملة جملة مستعيناً بورقة ترسم عليها جدولاً كل عمود يمثل متغيراً من متغيرات البرنامج وكل صف يبين القيم التي يأخذها المتغير خلال مراحل التنفيذ. والجدول الآتي يبين هذا الأسلوب للبرنامج السابق (مثال 2)

لنفرض أن المدخلات له البرنامج هي : $n = 5$ (أي اطبع قيمة مضروب 5)

n	k	factorial	<i>print</i> ('Factorial of', n, 'is', factorial)
5	1	1	
5	2	2	
5	3	6	
5	4	24	
5	5	120	Factorial of 5 is 120

مثال 4 : اكتب برنامج لحساب متوسط درجات الطلبة الدارسين بمقرر CS100.

التحليل : لحساب متوسط الدرجات نحتاج لدرجات الطلبة وعددهم

المعطيات : عدد الطلبة ويشار له بـ `no_students` والدرجة بـ `grade`.

المطلوب : حساب وعرض قيمة المتوسط ويشار له بـ `average`.

المعالجة : الخطوات اللازمة لتحقيق المطلوب

a. لنفرض أن قيمة العدد `count = 1` (قيمة العدد تبين عدد الطلبة الذم تم قراءة درجاتهم حتى الآن)

b. لنفرض ان المتغير الذي سنجمع فيه الدرجات كالاتي: $sum = 0$

c. اقاء عدد الطلبة وليكون $no_students$

d. نكرر خطوات الفقرة d طالما قيمة $count$ أصغر من $no_students$

• اقاء درجة الامتحان في المتغير $grade$

• قيمة sum الحالية = قيمة sum السابقة + $grade$

• نزيد قيمة العداد بواحد ($count = count + 1$)

e. $\frac{sum}{no_students} = average$ (المتوسط)

f. اطبع قيمة المتوسط

المخرجات: عرض قيمة المتوسط $average$.

```
def main():
    sum = 0          # محايد الجمع يساوي 0
    count = 0
    no_students = input('Enter No. of students : ')
    no_students = int(no_students)
    while (count < no_students) :
        grade = input('Enter student Grade : ')
        grade = float(grade)
        sum = sum + grade
        count = count + 1
    average = sum / no_students
    print('Average grade = ', average)
```

مثال 5: اكتب برنامج لحساب $sum = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} \dots + \frac{1}{n-1} + \frac{1}{n}$

التحليل

المعطيات: قراءة قيمة صحيحة للمتغير n

المطلوب: مجموع المتسلسلة

المعالجة: حساب قيمة sum وتكون من النوع الكسري

المعادلة $sum = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} \dots + \frac{1}{n-1} + \frac{1}{n}$ ويمكن كتابتها على صيغة $sum = \sum_{i=1}^n \frac{1}{i}$

برنامج لحساب $sum = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} \dots + \frac{1}{n-1} + \frac{1}{n}$

```
def main():
    n = input('Enter an integer value for n > 0 and <= 10 : ')
    # لا تنسى عملية تحويل قيمة n من نص إلى قيمة عددية
    n = int(n)
    i = 1
    sum = 0          # محايد الجمع يساوي 0
    while (i <= n) :
        sum = sum + 1 / i
        i = i + 1
    print('sum =', sum)
```

بالإمكان إعادة كتابة $sum = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n-1} + \frac{1}{n}$ كالآتي

$$sum = \frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{4} + \frac{1}{3} + \frac{1}{2} + 1$$

للحصول على نفس المطلوب اعد كتابة البرنامج كالآتي :

```
# برنامج لحساب  $sum = \frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} + \dots + \frac{1}{3} + \frac{1}{2} + 1$ 
def main():
    n = input('Enter an integer value for n > 0 and <= 10 : ')
    n = int(n)
    sum = 0
    while (n > 0):
        sum = sum + 1 / n
        n = n - 1
    print('sum =', sum)
```

مثال 6 : اكتب برنامج لعرض أعداد متسلسلة Fibonacci أى الأعداد 0 , 1 , 1 , 2 , 3 , 5 , 8 , . . . نلاحظ أن كل عدد = مجموع العددين السابقين له وتبدأ المتسلسلة بالعددين 0 , 1

التحليل:

المعطيات :

- لنرمز للعددين الأولين بـ prv_fib2, prv_fib1
- Prv_fib = 0 و prv_fib2 = 1، تم عرضهما.

المطلوب : عرض عدد معين من اعداد Fibonacci

المعالجة :

- تكرار حساب العدد التالي ويشار له بـ new_fib بحيث أن
 1. $new_fib = prv_fib1 + prv_fib2$
 2. عرض قيمة new_fib
 3. إسناد قيمة prv_fib2 إلى prv_fib1 وإسناد قيمة new_fib إلى prv_fib2

```
# برنامج لعرض عدد n من اعداد Fibonacci 1,1,2,3,5,8,...
def main():
    n = int(input('Enter an integer value: '))
    prv_fib1 = 0
    prv_fib2 = 1
    print(prv_fib1, ' ', prv_fib2, end = ' ')
    n = n - 2
    while (n > 0) :
        new_fib = prv_fib1 + prv_fib2
        print(' ', new_fib, end = ' ')
        prv_fib1 = prv_fib2
        prv_fib2 = new_fib
        n = n - 1
```

ما التعديل علي البرنامج السابق؟ ليصبح قادر علي عرض أعداد المتسلسلة

1, 1, 1, 3, 5, 9, 17, 31, ...

أي أن كل عدد = مجموع 3 أعداد السابقة وأن كل عدد من الأعداد الثلاثة الأولى = 1

حلقة For (التكرار لعدد معين من المرات):

تمكننا حلقة For من تكرار عدد من الجمل لعدد معين من المرات يحدد من خلال عناصر مجموعة معينة collection. انما تمكننا من تكرار الخطوات أو التعليمات البرمجية مرة واحدة لكل عنصر في المجموعة. المجموعات هي أشياء مثل القوائم lists، المصفوفات arrays، والقواميس dictionaries. فمثلا البرنامج التالي:

Example	Output
>>> list_t = [111,222,333] >>> for item in list_t: ... print 'item:', item	item: 111 item: 222 item: 33

يقوم بطباعة عناصر القائمة list_t. كما يمكن إستخدام حروف جملة نصية string كعداد وطباعة الحروف أو تكرار أية عمليات كما في المثالين التاليين:

Example	Output
>>> myName = "ABDUSSALAM" >>> for x in myName : ... print (x) ...	A B D U S S A L A M

أو

Example	Output
>>>i=1 >>> myName = "Ahmed" >>> for x in myName : ... print (i) ... i=i+1	1 2 3 4 5

كما يمكن استخدام القوائم في جملة for كما في المثال التالي:

Example	Output
<pre>>>>list=['a','hey',5,'hellow'] >>>for y in list: print (y) >>></pre>	<pre>a hey 5 hellow</pre>

إستخدام وظيفة المدى range() في جملة التكرار for loop:

الشكل العام للوظيفة range هو:

range(start, stop[, step])

حيث start هي القيمة المبدئية و stop القيمة النهائية و step هي قيمة الزيادة وقد تكون هذه القيمة سالبة وهي إختيارية. إذا لم تحدد قيمة step فهي تأخذ القيمة الافتراضية (1). والشكل العام لكيفية إستخدام range مع for loop هي:

for var in range(start, stop,step):

والمثال التالي يوضح ذلك:

Example	Output
<pre>>>>for x in range(2,7,2): print(x) >>></pre>	<pre>2 4 6</pre>

حيث var أسم متغير و قيمة start=2 و stop=7 و step=2. وتبدأ الحلقة بإسناد قيمة 2 للمتغير var ثم تنفيذ الجمل التابعة للحلقة for (وهي جملة واحدة في هذه الحالة (print(x)). يلي ذلك زيادة قيمة المتغير var بمقدار 2 أي أن (var=var+step) وإختبار قيمة var الجديدة. وفي حالة أن القيمة الجديدة للمتغير var أقل من قيمة stop أي (var < stop) يجري تنفيذ الجمل التابعة للحلقة for. ويتوقف التنفيذ إذا كانت قيمة var أكبر من أو تساوي قيمة stop.

إستخدام جملة else .. if مع حلقة for:

Example	Output
<pre>>>>for y in range(1,6): if(y%2==0): print('y is even') else: print('y is odd') >>></pre>	<pre>y is odd y is even y is odd y is even y is odd</pre>

إستخدام جملة **break** مع حلقة **for**:

تستخدم جملة **break** للخروج من حلقة **for** أو حلقة **while**. في المثال التالي يقوم المترجم بالخروج من الحلقة وتنفيذ الجملة التي تلي مباشرة حلقة **for** (وهي جملة **now stop because x=3**) عندما يصادف جملة **break**.

Example	Output
<pre>def main:() for x in range(7): if (x==3): break else: print(x) print('now stop because x=3')</pre>	<pre>0 1 2 now stop because x=3</pre>

إستخدام جملة **continue** مع حلقة **for**:

تستخدم جملة **continue** للرجوع إلى بداية حلقة **for** أو حلقة **while** ولا تنفذ الجمل التي تلي جملة **continue** إلا بعد إختبار شرط التكرار. في المثال التالي يقوم المترجم بالرجوع إلى بداية الحلقة عندما تكون قيمة **x=3** أو **x=5** ولا يطبع هذه القيم ولكن يزيد قيمة **x** ويختبر شرط التكرار ($x < 7$) ويستمر في الحلقة من جديد.

Example	Output
<pre>def main:() for x in range(7): if (x==3 or x==5): continue else: print(x) print('now stop')</pre>	<pre>0 1 2 4 6 now stop</pre>

تمارين

1. أعد تنفيذ برنامج المثال6 بعد هذا التعديل ولاحظ القيم المعروضة.
2. ما هو التغيير المناسب لجعل برنامج المثال6 يقوم بطباعة الاعداد الفردية من 1 إلى n.
3. إعد كتابة برنامج المثال6 بحيث يصبح قادر علي طباعة الاعداد الزوجية من m إلى n بحيث يتم قراءة قيمتي m, n من لوحة المفاتيح مع التوضيح لمن ينفذ البرنامج أن قيمة $n > m$.

باستخدام حلقة **while** :

4. أكتب برنامج لقراءة قيمة كل من a, b, n وحساب وطباعة ناتج المعادلة:

$$sum = \frac{1}{a} + \frac{1}{a+b} + \frac{1}{a+2b} + \frac{1}{a+3b} + \dots + \frac{1}{a+nb}$$

5. أكتب برنامج لقراءة قيمة n وحساب وطباعة ناتج المعادلة:

$$prod = \frac{1}{1^2} \times \frac{3}{2^2} \times \frac{5}{3^2} \times \dots \times \frac{2n-1}{n^2}$$

6. أكتب برنامج لقراءة قيمة n وحساب وطباعة ناتج المعادلة:

$$sum = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} \dots \pm \frac{1}{n}$$

7. أكتب برنامج لقراءة قيمة صحيحة ل n وحساب ناتج المعادلة :

$$Sum = 1! + 2! + 3! + \dots + n$$

8. أكتب برنامج لطباعة الأعداد الصحيحة الموجبة من 1 إلى غاية 99 بحيث يتم طباعة 3 اعداد بكل سطر على النحو التالي:

1	2	3
4	5	6
7	8	9
.....		
97	98	99

9. أكتب برنامج لحساب وعرض عدد n من متتابعات الأعداد التالية حيث أن $n > 3$ و $n \geq 50$.

a. 1, 2, 3, 6, 12, ...

b. 1, 3, 6, 10, 15, ...

c. 1, 4, 9, 16, 25, ...

d. 1, 8, 27, 64, ...

e. 60, 140, 240, 360, ...

f. 2, 8, 18, 32, 50, ...

g. -2, 4, -8, 16, -32, 64, ...

10. أكتب برنامج لقراءة عدد N من القيم ونشر للقيمة بـ X وحساب وعرض قيمة التباين ونشر له بـ VARIANCE باستخدام المعادلة:

$$\text{التباين} = \frac{\text{مجموع مربع القيم}}{\text{العدد الكلي للقيم (N)}} - \text{مربع متوسط القيم}$$

$$\text{variance} = \frac{\sum_{i=1}^n x_i^2}{n} - \left(\frac{\sum_{i=1}^n x_i}{n} \right)^2$$

11. أكتب برنامج لقراءة قيمة صحيحة لـ n وحساب ناتج المعادلة :

$$\text{sum} = \frac{x}{2} + \frac{x^2}{5} + \frac{x^3}{8} + \frac{x^4}{11} + \dots + \frac{x^n}{3n-1}$$

12. أكتب ناتج كلا من الوظائف الآتية:

```
def h ( ):
    k = 0
    y = 0
    p = 3
    n = 2121
    while (k < 4):
        r = n % 10
        n = n // 10
        y = y + p**k * r
        k = k + 1
    print(y)
```

```
def lg ( ):
    i = 0
    n = 16
    while (n % 2 == 0):
        i = i + 1
        n = n // 2
    print(i)
```

```
def formatd ( ):
    k = 0
    d = 0
    n = 435267
    while (k < 6):
        r = n % 10
        d = r * 10**k + d
        print(d)
        n = n // 10
        k = k + 1
```

4. الخوارزميات المركبة

في العديد من الأمثلة بالفصول السابقة قمنا بإستدعاء بعض الوظائف الجاهزة مثل (int, float, str, input, print) المعرفة ضمنياً بلغة Python للإستفادة منها في العديد من البرامج، وبدون معرفة متحواها. وهناك العديد من هذه الوظائف التي يحتاجها المبرمجين للقيام بمهام محددة ومعينة. في هذا الفصل نبين كيفية تصميم وتعريف وظائف جديدة لغرض إستدعائها بالعديد من البرامج. في هذا الفصل سنتعلم كيفية كتابة وظيفة لها مدخلات ومخرجات. وسنتعلم كيف نبني خوارزمية تستخدم عدة وظائف تم تعريفها لحل مسألة معينة.

لحل المسائل المركبة سنستخدم أسلوب فرق تسد: الذي يعني تفكيك المشكلة إلى مشاكل أقل تعقيداً وحل كل مشكلة لوحدها مما يؤدي إلى حل المشكلة الأكبر. بكلام آخر كتابة خوارزمية (وظيفة) منفصلة تحل جزء من المشكلة الرئيسة.

5.1. الوظيفة Function

في الفصول السابقة استخدمنا الوظائف ولكن لم يكن لهذه الوظائف مدخلات ولا مخرجات. لقد قام مصممي لغات البرمجة بتوفير هذه الوسيلة البرمجية بحيث يتم كتابة هذه الجمل مرة واحدة والتأكد من صحتها تم استخدامها بطريقة إستدعاءها أكثر من مرة، وهذا الأسلوب يساهم في:

- تقليص حجم البرنامج.
- تجزئة البرنامج إلى وظائف، حيث تقوم كل وظيفة بتنفيذ مهمة معينة. (الشكل المركب للخوارزمية)
- إمكانية إستخدام هذه الوظائف أكثر من مرة بالبرنامج الواحد وكذلك في برامج أخرى.
- ليس بالضرورة معرفة التفاصيل البرمجية لهذه الوظائف.
- تقليل نسبة الأخطاء والتصحيح.

ويمكن تصنيف الوظائف إلى:

- وظائف تستقبل بيانات وترجع بيانات.
- وظائف تستقبل بيانات ولا ترجع أي بيانات وتقوم فقط بتنفيذ بعض المهام بإستخدام تلك البيانات.
- وظائف لا تستقبل بيانات وترجع بيانات.
- وظائف لا تستقبل أي بيانات ولا ترجع أي بيانات وتقوم فقط بتنفيذ بعض المهام.

مدخلات

الشكل العام للوظيفة كالآتي:

```
def name_of_function (arguments) :
```

جمل يراد تنفيذها
return X₁, X₂, . . . , X_n

مخرجات

لإرسال مدخلات إلى الوظيفة يتم ذلك بوضع المتغيرات المطلوبة في جملة الاستدعاء، ومن ثم تنسخ كل قيمة من هذه المتغيرات في المتغير المناظر لها في مدخلات الوظيفة (arguments) كالآتي:

لدينا الوظيفة التالية:

```
def main(x,y):  
    ◦  
    ◦  
    ◦  
  
    return q , w
```

و لدينا المتغيرات الآتية : $z = 12$, $f = 80$, ولبعث هذه القيم إلى الوظيفة `main()` يتم اصدار هذا الاستدعاء `main(z , f)`.
عند تنفيذ هذا الاستدعاء فإن $x = 12$ و $y = 80$ ثم يتم تنفيذ الوظيفة.

مثال 1 : كتابة مجموعة من الوظائف لهم علاقة بالمثلثات.

1. وظيفة تستقبل 3 قيم تمثل أضلاع مثلث وحساب وإرجاع مساحة المثلث باستخدام القانون:

$$s = \frac{a+b+c}{2}$$
$$area = \sqrt{s \times (s-a) \times (s-b) \times (s-c)}$$

```
def area_of_triangle(a, b, c):  
    s = (a + b + c) / 2  
    w = s * (s - a) * (s - b) * (s - c)  
    area = w ** 0.5  
    return area
```

2. وظيفة تستقبل 3 قيم ويشار لهم بـ a, b, c وإرجاع **True** في حالة أن القيم تمثل أضلاع مثلث أو إرجاع **False** في حالة أن القيم لا تمثل أضلاع مثلث. القيم الثلاث تمثل مثلث في حالة أن مجموع أي ضلعين < الضلع الثالث.

```
def is_triangle(a, b, c):  
    ans = a + b > c and a + c > b and b + c > a  
    return ans
```

مثال 2 : أكتب وظيفة تستقبل 3 قيم ويشار لهم بـ a, b, c وحساب مساحة المثلث في حالة أن القيم تمثل أضلاع مثلث أو إرجاع عرض الرسالة التالية :

"The numbers do not represent a triangle" في حالة أن القيم لا تمثل أضلاع مثلث

التحليل:

- المعطيات: إدخال من لوحة المفاتيح القيم الثلاثة : a, b, c
- المخرجات: عرض قيمة المساحة أو عرض الرسالة.

في هذه المسألة عندنا الشروط التالية: $a \geq 0, b \geq 0, c \geq 0$ ، ولحل هذه المسألة نفككها إلى 4 مسائل كالتالي:

- a. قراءة القيم الثلاثة و التأكد من أنها لا تتحل بالشروط :
- b. كتابة وظيفة لقراءة القيم الثلاثة والتأكد من أنها لا تتحل بالشروط
- c. اختبار هل القيم تمثل مثلثاً : كتابة وظيفة لاختبار أن القيم تمثل مثلثاً
- d. حساب مساحة المثلث : كتابة وظيفة لحساب قيمة المساحة
- e. عرض قيمة المساحة إذا كانت القيم تمثل مثلثاً أو عرض الرسالة إذا لم تكن القيم تمثل مثلثاً : كتابة وظيفة لتنفيذ المطلوب.

• الخوارزمية :

1. نفذ وظيفة القراءة
2. نفذ وظيفة الاختبار إذا كانت نتيجة الاختبار **True** نفذ وظيفة حساب المساحة ثم اعرض القيمة وإذا نتيجة الاختبار **False** اعرض الرسالة
3. توقف

في هذا المثال سنستخدم الوظائف التي تم كتابتها في المثال السابق (مثال 1)

وستكون الخوارزمية بلغة Python كالتالي:

في الوظيفة `read_float_val(prompt, minval)` قيمة المتغير `prompt` هو النص المراد عرضه لمدخل البيانات لتوضيح ما المطلوب إدخاله من لوحة المفاتيح، وقيمة المتغير `minval` يمثل الحد الأدنى، ويمثل المتغير `var` القيمة المراد استقبالتها من لوحة المفاتيح وتحقق الشروط وترجيئها إلى مصدر استدعاء الوظيفة. في هذا المثال لاستدعاء هذه الوظيفة ندرج الخطوات الآتية في البرنامج كالتالي:

```
minval = 0
side = 'a'
prompt = 'Enter value of ' + side + ' : '
read_float_val(prompt, minval)
```

```
def read_float_val (prompt, minval):
    while (True):
        var = input(prompt)
        var = float(var)
        if (var > minval):
            return var
```

عند تنفيذ هذه الوظيفة لن يخرج البرنامج من حلقة **while** حتى يقوم المستخدم بإدخال القيمة الصحيحة. وبذلك أصبح لدينا القدرة على التحكم في قبول ورفض القيم التي يتم إدخالها من لوحة المفاتيح. بهذا الأسلوب نقلل من حدوث الأخطاء المنطقية (Semantic Errors).

```
def read_float_val (prompt, minval):
    while ( True):
        var = input (prompt)
        var = float(var)
        if (var > minval):
            return var

def area_of_triangle (a, b, c):
    s = (a + b + c) / 2
    w = s * (s - a) * (s - b) * (s - c )
    area = w ** 0.5
    return area

def is_triangle(a, b, c):
    ans = a + b > c and a + c > b and b + c > a
    return ans

def main():
    minval = 0
    side = 'a'
    prompt = 'Enter value of ' + side + ' > ' +str(minval) + ' : '
    a=read_float_val (prompt, minval)
    side = 'b'
    prompt = 'Enter value of ' + side + ' > ' +str(minval) + ' : '
    b=read_float_val (prompt, minval)
    side = 'c'
    prompt = 'Enter value of ' + side + ' > ' + str(minval) + ' : '
    c=read_float_val (prompt, minval)
    if is_triangle(a, b, c) :
        area = area_of_triangle (a, b, c)
        print (' The area is : ', area)
    else :
        print('The numbers do not represent a triangle ')
```

مثال 3 : أكتب برنامج لإيجاد التوافيق $C_r^n = \frac{n!}{r!(n-r)!}$ أي عدد الطرق التي يتم بها اختيار r من بين مجموعة تحتوي على n من العناصر، حيث أن $(n \geq r)$ مع إهمال عملية الترتيب.

التحليل

- المعطيات: إدخال من لوحة المفاتيح قيمة لكل من المتغير **n**، والمتغير **r**.

- المخرجات: عرض قيمة ans على الشاشة.

ملاحظة:

$$n \times (n-1) \times \dots \times 3 \times 2 \times 1 = n! \text{ (مضروب } n \text{)}$$

$$r \times (r-1) \times \dots \times 3 \times 2 \times 1 = r! \text{ وعلى نفس المنوال}$$

$$(n-r) \times (n-r-1) \times \dots \times 3 \times 2 \times 1 = (n-r)! \text{ وكذلك}$$

نلاحظ أن طريقة حساب كل من $n!$, $r!$, $(n-r)!$ متشابهة والاختلاف فقط في الحد النهائي. لنبدأ بتصميم وكتابة تعريف لوظيفة بإسم factorial لحساب أي مضروب ويشار للقيمة المطلوب حساب مضروبها بـ n وقيمة المضروب بـ $fact$.

في هذه المسألة عندنا الشروط التالية : $n \geq 0, r \leq n$, حتي لا تكون $n-r$ قيمة سالبة. ولحل هذه المسألة نفككها إلى 4 مسائل كالآتي:

- قراءة القيمتان والتأكد من أنها لا تخل بالشروط :
- كتابة وظيفة لقراءة القيمتان والتأكد من أنها لا تخل بالشروط
- حساب قيمة مضروب عدد معين : كتابة وظيفة لحساب قيمة المضروب وارجاعها
- حساب قيمة التعبير المطلوب كتابة وظيفة لتنفيذ هذا الطلب.
- توقف

• المعالجة:

1. نفذ وظيفة القراءة لكل من n و r
2. نفذ وظيفة حساب مضروب n ويشار لها بـ $nfact$
3. نفذ وظيفة حساب مضروب r ويشار لها بـ $rfact$
4. نفذ وظيفة حساب مضروب $(n-r)$ ويشار لها بـ $nrfact$
5. حساب $C_r^n = \frac{nfact}{rfact \times nrfact}$
6. اعرض قيمة C_r^n
7. توقف

تنبيه:

الوظيفة الضمنية ($isdigit$) ترجع القيمة **True** في حالة أن المتغير المرتبط معها يمثل عدداً صحيحاً وترجع القيمة **False** في غير هذه الحالة.

```

def read_integer_val (prompt, minval, ):
    while (True):
        var = input(prompt)
        if (var.isdigit()):
            var = int(var)
            if (var >= minval):
                return var

def factorial (n):
    fact = 1
    i = 1
    while (i <= n):
        fact = fact * i
        i = i + 1
    return fact

def main():

    minval = 0
    prompt = 'Enter the value of ' + 'n' + '> 0 : '
    n = read_integer_val(prompt,minval)
    prompt = 'Enter the value of ' + 'r' + '> 0 : '
    r = read_integer_val (prompt,minval)
    nfact = factorial (n)
    rfact = factorial (r)
    nrfact = factorial (n - r)
    ans = nfact / (rfact * nrfact)
    print('p(', n, ',', r, ') =', n, '!' / ('r, '!' *, n-r, '!') =', ans)

```

ولاستدعاء هذه الوظائف نقوم بإصدار الأمر الآتي: main()

تنبيه : البرنامج السابق يقوم بالمهمة والهدف هو توضيح كيفية تعريف واستخدام الوظائف ولكن ليس بالحل الامثل.

حول الجمل التالية من البرنامج السابق إلى وظيفة يتم إستدعائها وإسناد الناتج للمتغير ans علي الشكل التالي:

ans = **combination** (n, r)

والجمل هي:

```

nfact = factorial (n)
rfact = factorial (r)
nrfact = factorial (n - r)
ans = nfact / (rfact * nrfact)

```

```
def main():
    n = read_integer_val ("n:", 2, 10)
    r = read_integer_val ("r: ", 1, n - 1) # why n-1 ?
    ans = combination (n, r)
    print ("p(", n, ", ", r, ") =", n, "! / (", r, "! *", n - r, "!)" =, ans)
```

مثال 4: تم تكليفك من قبل اللجنة المنظمة لسباق الركض لمسافة 20 كم لكتابة برنامج لحساب وقت الوصول للرياضيين من خلال معرفة وقت الانطلاق المتكون من (ساعة ويشار لها بـ h1 ، ودقائق ويشار له بـ m1، وثواني ويشار له بـ s1) والزمن المستغرق بالثواني ويشار له بـ sec لقطع مسافة السباق للرياضي وحساب وعرض علي الشاشة وقت الوصول المتكون من (ساعة ويشار لها بـ h2 ، ودقائق ويشار له بـ m2، وثواني ويشار له بـ s3).

التحليل:

• المدخلات من لوحة المفاتيح :

- وقت الانطلاق ويتكون من 3 قيم ويشار لهم بـ h1 ، m1 ، s1.
- الزمن المستغرق للركض بالثواني ويشار له بـ sec.

• الشروط المطلوبة لقبول القيم التي يتم إدخالها من لوحة المفاتيح:

- $59 \geq s1 \geq 0$
- $59 \geq m1 \geq 0$
- $23 \geq h1 \geq 0$
- $7200 \geq sec \geq 0$

• المخرجات : وقت الوصول ويتكون من 3 قيم ويشار لهم بـ h2 ، m2 ، s2.

• المعالجة :

- تحويل وقت الانطلاق إلثواني وإضافته إلالزمن المستغرق للسباق ولنرمز له بـ total_seconds (وقت الوصول بالثواني).
- تحويل total_seconds الزمن بالثواني إلى ساعات ويشار له بـ h2 ودقائق ويشار له بـ m2 وثواني ويشار له بـ s2. وهذا هو وقت الوصول المطلوب عرضه.

البرنامج : سيتم تطوير البرنامج كما تعودنا علي مراحل لتوضيح عملية البرمجة بالأسلوب التدريجي:

أولاً:

- قراءة وقت الانطلاق من لوحة المفاتيح والمتكون من 3 قيم (h1, m1, s1) وتحويله إلثواني

حيث أن: $h1_m1_s1_to_sec$ ويشار له بـ

$$h1_m1_s1_to_sec = h1 * 3600 + m1 * 60 + s1$$

- تنفيذ البرنامج على جهاز الحاسوب والتأكد من صحة النتائج.

```
def main():  
    print("وقت الانطلاق")  
    h1 = int(input("h1:"))  
    m1 = int(input("m1: "))  
    s1 = int(input("s1: "))  
    h1_m1_s1_to_sec = h1 * 3600 + m1 * 60 + s1  
    print(h1_m1_s1_to_sec)
```

ثانياً:

- إضافة جمل لقراءة الزمن المستغرق للسباق وإضافته إلى زمن الإنطلاق بالثواني. الجزء المظلل يبين الإضافة.

$$total_sec = h1_m1_s1_to_sec + sec$$

- تنفيذ البرنامج على جهاز الحاسوب والتأكد من صحة النتائج.

```
def main():  
    print("وقت الانطلاق")  
    h1 = int(input("h1:"))  
    m1 = int(input("m1: "))  
    s1 = int(input("s1: "))  
    h1_m1_s1_to_sec = h1 * 3600 + m1 * 60 + s1  
    sec = int(input("sec: "))  
    total_sec = sec + h1_m1_s1_to_sec  
    print(total_sec)
```

ثالثاً:

- تحويل زمن الوصول من ثواني إلى (ساعات ويشار له بـ $h2$ ودقائق ويشار له بـ $m2$ وثواني ويشار له بـ $s2$) تم عرض الناتج النهائي والمطلوب على الشاشة.
- تنفيذ البرنامج على جهاز الحاسوب والتأكد من صحة النتائج.

```
def main():  
    print("وقت الانطلاق")  
    h1 = int(input("h1:"))  
    m1 = int(input("m1: "))  
    s1 = int(input("s1: "))  
    h1_m1_s1_to_sec = h1 * 3600 + m1 * 60 + s1  
    print("الزمن المستغرق")  
    sec = int(input("sec: "))  
    total_sec = sec + h1_m1_s1_to_sec  
    min = total_sec // 60  
    s2 = total_sec % 60  
    h2 = min // 60  
    m2 = min % 60  
    print("وقت الوصول")  
    Print("h:", h2, "m:", m2, "s:", s2)
```

البرنامج السابق ينجز المطلوب ولكن لا يلتزم بالشروط التي يجب توفرها بالقيم التي يتم إدخالها من لوحة المفاتيح. والشروط هي:

maxval	minval	var
59	0	s1
59	0	m1
23	0	h1
7200	0	sec

رابعاً:

لتحسين البرنامج وجعله يلتزم بالشروط يتم استخدام الوظيفة **read_integer_val** تمنع المدخل للبيانات من تجاوز الشروط المذكورة أعلاه عند تنفيذ البرنامج. وهذه الوظيفة كالآتي:

```
def read_integer_val (prompt, minval, maxval):
    while (True):
        var = int(input(prompt))
        if (minval <= var <= maxval):
            return var
```

حيث أن قيمة المتغير **prompt** هو النص المراد عرضه لمدخل البيانات لتوضيح ما المطلوب إدخاله من لوحة المفاتيح، وقيمة المتغير **minval** يمثل الحد الأدنى التي يمكن قبولها، وقيمة المتغير **maxval** يمثل الحد الأكبر، ويمثل المتغير **var** القيمة المراد استقبالتها من لوحة المفاتيح وتحقق الشروط وترجيئها إلى المصدر استدعاء الوظيفة. في هذا المثال لاستدعاء هذه الوظيفة ندرج الخطوات الآتية في البرنامج كالآتي:

minval = 0

maxval = 59

prompt = 'Enter seconds ' + str(minval) + ' and <= ' + str(maxval) + ' : '

read_integer_val (prompt, minval, maxval):

وبالامكان استدعاء نفس الوظيفة لقراءة عدد الثواني والدقائق والساعات لزمان معين حيث أن:

maxval	minval	var	prompt
59	0	hr	'Enter seconds ' + str(minval) + ' and <= ' + str(maxval) + ' : '
59	0	mn	'Enter ' + str(minval) + ' and <= ' + str(maxval) + ' : ' minutes
23	0	sc	'Enter hours ' + str(minval) + ' and <= ' + str(maxval) + ' : '

أصبح لدينا القدرة على التحكم في قبول ورفض القيم التي يتم إدخالها من لوحة المفاتيح. بهذا الأسلوب نقلل من حدوث الأخطاء المنطقية (Semantic Errors).

```
def read_integer_val (prompt, minval, maxval):
    while (True):
        var = input(prompt)
        if (var.isdigit()):
            var = int(var)
            if (minval <= var <= maxval):
                return var

def main():
    print("وقت الانطلاق")
    minval = 0
    maxval = 23
    prompt = 'Enter hours ' + str(minval) + ' and <= ' + str(maxval) + ' : '

    h1 = read_integer_val (prompt, minval, maxval)
    maxval = 59
    prompt = 'Enter minutes ' + str(minval) + ' and <= ' + str(maxval) + ' : '

    m1 = read_integer_val (prompt, minval, maxval)
    prompt = 'Enter seconds ' + str(minval) + ' and <= ' + str(maxval) + ' : '

    s1 = read_integer_val (prompt, minval, maxval)

    h1_m1_s1_to_sec = h1 * 3600 + m1 * 60 + s1
    print("الزمن المستغرق")
    maxval = 7200
    prompt = 'Enter seconds ' + str(minval) + ' and <= ' + str(maxval) + ' : '
    sec = read_integer_val (prompt, minval, maxval)

    total_sec = sec + h1_m1_s1_to_sec
    min = total_sec // 60
    s2 = total_sec % 60
    h2 = min // 60
    m2 = min % 60
    print(" وقت الوصول ")
    print("h:", h2, "m:", m2, "s:", s2)
```

لنحاول الآن كتابة تعريف وظيفية بإسم to_seconds تستقبل وقت متكون من (ساعات ويشار له بـ h ودقائق ويشار له بـ m وثواني ويشار له بـ s) وتحويله إلى ثواني ويشار له بـ seconds وإرجاع نتيجة التحويل باستخدام **return**.

```
def to_seconds (h, m, s):
    seconds = h * 3600 + m * 60 + s
    return seconds
```

وكذلك تعريف وظيفية بإسم to_hms تستقبل زمن بالثواني ويشار له بـ seconds وتحويله إلى المكائئ (بالساعات ويشار له بـ h والدقائق ويشار له بـ m والثواني ويشار له بـ s). نلاحظ أن هذه الوظيفة تقوم بإرجاع 3 قيم (s, m, h).

```
def to_hms (seconds):
    min = seconds // 60
    s = seconds % 60
    h = min // 60
    m = min % 60
    return h, m, s
```

خامسا: البرنامج النهائي بعد إجراء التحسينات المطلوبة:

```
def to_seconds (h, m, s):
    seconds = h * 3600 + m * 60 + s
    return seconds
```

```
def to_hms (seconds):
    min = seconds // 60
    s = seconds % 60
    h = min // 60
    m = min % 60
    return h, m, s
```

```
def read_integer_val (prompt, minval, maxval):
    while (True):
        var = input (prompt)
        if (var.isdigit()):
            var = int(var)
            if (minval <= var <= maxval):
                return var
```

```
def main():
    print ("وقت الانطلاق")
    minval = 0
    maxval = 23
    prompt = 'Enter hours ' + str(minval) + ' and <= ' + str(maxval) + ' : '
```

```
h1 = read_integer_val (prompt, minval, maxval)
maxval = 59
prompt = 'Enter minutes ' + str(minval) + ' and <= ' + str(maxval) + ' : '
```

```
m1 = read_integer_val (prompt, minval, maxval)
prompt = 'Enter seconds ' + str(minval) + ' and <= ' + str(maxval) + ' : '
```

```
s1 = read_integer_val (prompt, minval, maxval)
print ("الزمن المستغرق")
maxval = 7200
prompt = 'Enter seconds ' + str(minval) + ' and <= ' + str(maxval) + ' : '
sec = read_integer_val (prompt, minval, maxval)
```

```
h1_m1_s1_to_sec = to_second(h1, m1, s1)
total_sec = sec + h1_m1_s1_to_sec
h2, m2, s2 = to_hms(total_sec)
print ("وقت الوصول")
print ("h:", h2, "m:", m2, "s:", s2)
```

توضيح: كعملية تنظيمية يستحسن وضع تعريف الوظائف المصاحبة للبرنامج الرئيس في بداية البرنامج قبل الوظيفة الرئيسية `main`.
مثال 5: أكتب برنامج قادر علي قراءة درجة الحرارة وتحويل درجة الحرارة من نظام `celsius` إلى `fahrenheit` والعكس بناء على رغبة مستخدم البرنامج.

التحليل

- المدخلات من لوحة المفاتيح : باستخدام الوظيفة `read_integer_val`.
 - نوع التحويل ويشار له بالمتغير `choice`.
 - درجة الحرارة ويشار لهما بالمتغيران `ctemp`, `ftemp`.
 - المخرجات: عرض درجة الحرارة بناء على نوع التحويل.
 - المعالجة : لتحويل درجة الحرارة من نظام `fahrenheit` إلى `celsius` نحتاج إلى القانون الذي تم توضيحه سابقا :

$$ctemp = (ftemp - 32) * 5 / 9$$

$$= 9 / 5 * ctemp + 32$$
- سيتم تطوير البرنامج علي مراحل لتوضيح عملية البرمجة بالأسلوب التدريجي:
أولاً: عرض لائحة لخيارات التحويل لاختيار نوع التحويل المطلوب وإدخال نوع التحويل ويشار له ب `choice` باستخدام الوظيفة `read_integer_val`.

```
def read_integer_val (prompt, minval, maxval):
    while (True):
        var = input(prompt)
        if (var.isdigit()):
            var = int(var)
            if (minval <= var <= maxval):
                return var

def main():
    print('\n')
    print('Options:')
    print('1 - Convert temperature from fahrenheit to celsius')
    print('2 - Convert temperature from celsius to fahrenheit')
    print('3 - Quit the program')
    choice = read_integer_val('Enter your choice {1, 2, 3}: ', 1, 3)
```

عند تنفيذ البرنامج سنلاحظ أن البرنامج لا يسمح بإدخال أى قيمة للمتغير `choice` عدا القيم المحددة (والذي يمثل الخيارات المتاحة وهي { 3, 2, 1 }).

ثانياً: إضافة جمل معالجة الخيارات المتاحة من اللائحة المعروضة والتأكد من صحة الإدخال من لوحة المفاتيح. للتبسيط وباستخدام الأسلوب التدريجي للبرمجة سنقوم في البداية باستخدام حلقة **while** للسماح بتنفيذ عمليات التحويل طالما هناك الرغبة في ذلك. وإضافة جمل للتحكم في مسار التنفيذ باستخدام جملة **if** ذات التفرعين وعرض جملة توضيحية تبين مسار التنفيذ بعد إختيار نوع التحويل المطلوب وسيتم أستبدال هذه الجملة بالخطوات التفصيلية لتنفيذ الخيار المطلوب. البرنامج التالي يبين الإضافات.

```

def read_integer_val (prompt, minval, maxval):
    while (True):
        var = input(prompt)
        if (var.isdigit()):
            var = int(var)
            if (minval <= var <= maxval):
                return var

def main():
    print('\n')
    print('Options:')
    print('1 - Convert temperature from fahrenheit to celsius')
    print('2 - Convert temperature from celsius to fahrenheit')
    print('3 - Quit the program')
    choice = read_integer_val('Enter your choice {1, 2, 3}: ', 1, 3)

    while (choice != 3):
        if (choice == 1):
            print('code for conversion from fahrenheit to celsius goes here')
        elif (choice == 2):
            print('code for conversion from Celsius to Fahrenheit goes here')

        print('\n')
        print('options:')
        print('1 - convert temperature from fahrenheit to celsius')
        print('2 - convert temperature from celsius to fahrenheit')
        print('3 - quit the program')
        choice = read_integer_val('Enter your choice {1, 2, 3}: ', 1, 3)

```

نلاحظ أن الجمل المظلمة مكررة. لنقلص هذا التكرار بتحويلها إلى وظيفة ولنختار لها الاسم **menu**. هذه الوظيفة لا تستقبل بيانات وتستقبل من لوحة المفاتيح وترجع أحد القيم من القيم المعروضة الثلاث {1, 2, 3}.
ثالثا: تقليص التكرار وتحويله إلى وظيفة.

```

def menu():
    print('\n')
    print('Options:')
    print('1 - Convert temperature from fahrenheit to celsius')
    print('2 - Convert temperature from celsius to fahrenheit')
    print('3 - Quit the program')
    choice = read_integer_val('Enter your choice {1, 2, 3}: ', 1, 3)
    return choice

```

البرنامج بعد التقليص. لاحظ إستدعاء الوظيفة **menu** وإسناد ناتج تنفيذها إلى المتغير **choice** مرتين بالوظيفة **main** علي النحو التالي: **choice = menu()**

```

def read_integer_val (prompt, minval, maxval):
    while ( True):
        var = input (prompt)
        if (var.isdigit()):
            var = int(var)
            if (minval <= var <= maxval):
                return var

def menu():
    print ('\n')
    print ('Options:')
    print ('1 - Convert temperature from fahrenheit to celsius')
    print ('2 - Convert temperature from celsius to fahrenheit')
    print ('3 - Quit the program')
    choice = read_integer_val('Enter your choice {1, 2, 3}: ', 1, 3)
    return choice

def main():
    choice = menu()
    while (choice != 3):
        if (choice == 1):
            print ('code for conversion from fahrenheit to celsius goes here')
        elif (choice == 2):
            print ('code for conversion from celsius to fahrenheit goes here')
        choice = menu()

```

رابعاً: استبدال الجملتين بخطوات محددة لإدخال درجة الحرارة وحساب المكافئ وعرض النتائج.

1. إستبدال الجملة : `print ('code for conversion from fahrenheit to Celsius goes here')`

بجمل لقراءة درجة الحرارة ب Fahrenheit وتحويلها إلى المكافئ ب celsius باستخدام قانون التحويل وعرض الناتج.

2. إستبدال الجملة : `print ('code for conversion from fahrenheit to Celsius goes here')`

بجمل لقراءة درجة الحرارة ب celsius وتحويلها إلى المكافئ ب fahrenheit باستخدام قانون التحويل وعرض الناتج.

```

def read_integer_val (prompt, minval, maxval):
    while (True):
        var = input(prompt)
        if (var.isdigit()):
            var = int(var)
            if (minval <= var <= maxval):
                return var

def menu():
    print('\n')
    print('Options:')
    print('1 - Convert temperature from fahrenheit to celsius')
    print('2 - Convert temperature from celsius to fahrenheit')
    print('3 - Quit the program')
    choice = read_integer_val('Enter your choice {1, 2, 3}: ', 1, 3)
    return choice

def main():
    choice = menu()
    while (choice != 3):
        if (choice == 1):
            ftemp = read_integer_val('fahrenheit temperature: ', -1000, 1000)
            ctemp = (ftemp - 32) * 5 / 9
            print('equivalent temperature in celsius =', ctemp)
        elif (choice == 2):
            ctemp = read_integer_val('celsius temperature: ', -1000, 1000)
            ftemp = 9 / 5 * ctemp + 32
            print('equivalent temperature in fahrenheit =', ftemp)
        choice = menu()
    choice = menu()    print('\nEnd of program\n')

```

ركن التفكير: بالإمكان إجراء تعديل بسيط بالبرنامج الرئيسي لجعل البرنامج يحتاج لكتابة جملة إستدعاء الوظيفة menu() choice مرة واحدة.

مثال 6: أكتب وظيفة تستقبل عدد صحيح وأرجاع **True** في حالة أن العدد أولي أو **False** في حالة أن العدد ليس بعدد أولي. حقائق عن العدد الأولي:

- هو عدد صحيح موجب يقبل القسمة على واحد ونفسه فقط .
- لا يكون عدد زوجي عدا العدد 2.
- عدد فردي.

سيتم تطوير البرنامج علي مراحل لتوضيح عملية البرمجة وتحسن الأداء بالأسلوب التدريجي:

اولا : سنكتب الوظيفة ولنختار لها الاسم is_prime مهمتها استقبال عدد صحيح موجب ولنشير له بـ n وإرجاع **True** في حالة أن العدد أولي أو إرجاع **False** في حالة أن العدد ليس أولي.


```

def is_prime(n):
    if ( n < 4 ):
        return True          # أعداد أولية 2، 3
    if (n%2 == 0):
        return False         # الأعداد الزوجية ليست أعداد أولية عدا 2
    j = 3
    while (j < n):
        if(n%j == 0):
            return False     # العدد الذي يقبل القسمة علي عدد آخر ليس عدد أولي
        j = j + 1
    return True              # في حالة أن العدد أولي

```

ثانياً : إستخدام الوظيفة is_prime لعرض الاعداد الأولية من العدد m إلى العدد n، حيث $n > m$.

```

def is_prime(n):
    if ( n < 4 ):
        return True          # أعداد أولية 2، 3
    if (n%2 == 0):
        return False         # الأعداد الزوجية ليست أعداد أولية عدا 2
    j = 3
    while (j < n):
        if (n%j == 0):
            return False     # العدد الذي يقبل القسمة علي عدد آخر ليس عدد أولي
        j = j + 1
    return True              # في حالة أن العدد أولي

```

```

def read_integer_val (prompt, minval, maxval):
    while ( True):
        var = input(prompt)
        if (var.isdigit()):
            var = int(var)
            if (minval <= var <= maxval):
                return var

```

```

def main():
    max_int = 9999999999999999
    m = read_integer_val('Enter an odd integer > 1: ', 2, max_int)
    while (m%2 == 0):
        m = read_integer_val('Enter an odd integer > 1: ', 2, max_int)
    n = read_integer_val('Enter an integer > ' + str(m) + ' : ', m, max_int)
    while (n <= m):
        n = read_integer_val('Enter an integer > ' + str(m) + ' : ', m, max_int)
    while (m <= n):
        if (is_prime(m)):
            print(m)
        m = m + 2          # لأن الاعداد الأولية أعداد فردية

```

نفذ البرنامج علي سبيل المثال بحيث أن قيمة $m=1090101$ وقيمة $n=1090500$ يمكن تحسين أداء الوظيفة (أي تسريع التنفيذ) وذلك بالاستفادة من خصائص الأعداد الأولية. علي سبيل المثال في الوظيفة :

- استبدال الجملة $j = j + 1$ بالجملة $j = j + 2$ لان الأعداد الأولية أعداد فردية فهي لا تقبل القسمة على أعداد زوجية عدا 2.

- استبدال الجملة : $while (j < n)$ بالجملة : $while (j < n // 2)$ والأحسن $while (j <= n ** 0.5)$:

بعد تنفيذ هذه التعديلات تكون الوظيفة كالتالي:

```
def is_prime(n):
    if (n < 4):
        return True          # أعداد أولية 1، 2، 3
    if (n%2 == 0):
        return False         # الأعداد الزوجية ليست أعداد أولية عدا 2
    j = 3
    while (j < n ** 0.5):
        if (n % j == 0):
            return False     # العدد الذي يقبل القسمة علي عدد آخر ليس عدد أولي
        j = j + 2
    return True              # في حالة أن العدد أولي
```

اعد تنفيذ البرنامج علي سبيل المثال بنفس القيم أي أن قيمة $m=1090101$ وقيمة $n=1090500$ ولاحظ سرعة التنفيذ.

تمارين: باستخدام الوظائف

4. أكتب برنامج لحساب وقت الوصول لرحلات جوية بمعرفة وقت الإقلاع والزمن المستغرق للرحلة على سبيل المثال:

وقت الإقلاع			زمن المستغرق للرحلة			وقت الوصول		
s1	m1	h1	s2	m2	h2	s3	m3	h3
0	30	2	0	40	1			
0	0	4	0	15	2			
0	45	12	0	50	0			

5. أكتب برنامج لحساب الزمن المستغرق لرحلات جوية بمعرفة وقت الإقلاع ووقت الوصول على سبيل المثال:

وقت الإقلاع			وقت الوصول			زمن المستغرق للرحلة		
s1	m1	h1	s2	m2	h2	s3	m3	h3
0	30	3	0	10	5			
0	0	4	0	50	4			
0	15	12	0	40	13			

3. أكتب وظيفة تستقبل 3 قيم ويشار لهم بـ a, b, c وحساب وإرجاع **True** في حالة أن القيم تمثل أضلاع مثلث قائم الزاوية أو إرجاع **False** في حالة أن القيم لا تمثل أضلاع مثلث قائم الزاوية. المثلث قائم الزاوية في حالة أن مجموع مربعي أي ضلعين = مربع الضلع الثالث.

```
def is_right_triangle(a, b, c):
```

4. أكتب وظيفة تستقبل 3 قيم ويشار لهم بـ a, b, c وحساب وإرجاع **True** في حالة أن القيم لا أضلاع مثلث متساوي الأضلاع أو إرجاع **False** في حالة أن القيم لا تمثل أضلاع مثلث متساوي الأضلاع.

```
def is_equilateral(a, b, c):
```

5. أكتب وظيفة تستقبل 3 قيم ويشار لهم بـ a, b, c وحساب وإرجاع **True** في حالة أن القيم لأضلاع مثلث متساوي الساقين أو إرجاع **False** في حالة أن القيم لا تمثل أضلاع مثلث متساوي الساقين.

```
def is_isoseismal (a, b, c):
```

6. أكتب وظيفة تستقبل 3 قيم ويشار لهم بـ a, b, c تمثل أضلاع مثلث وحساب وإرجاع محيط المثلث.

```
def circumference (a, b, c):
```

7. أكتب برنامج رئيسي يقوم بقراءة 3 قيم تمثل أضلاع مثلث والتأكد من أن القيم تمثل أضلاع مثلث. في حالة تحقق الشرط عرض لائحة تتضمن مجموعة الخيارات على النحو التالي:

Option

- 1 : Is it right triangle (قائم الزاوية)
- 2 : Is it equilateral (متساوي الأضلاع)
- 3 : Is it equilateral (متساوي الساقين)
- 4 : Calculate area (إحسب المساحة)
- 5 : Calculate circumference (إحسب المحيط)
- 6 : End program

واستدعاء الوظيفة المناسبة والمطلوب تنفيذها وعرض ناتج الوظيفة مع الجملة المناسبة.

6. المتغيرات المركبة

في الفصول السابقة كل المتغيرات المستخدمة بسيطة التركيب: أي تأخذ قيمة واحدة في نفس اللحظة. ولكن في هذا الفصل سنعرض بعض الخوارزميات التي ستستخدم متغيرات مركبة: أي متغير ذو اسم واحد يحتوي على عدة قيم في نفس اللحظة و يمكن استخدام أي قيمة على حدة.

في هذا الفصل سندرس كيفية كتابة وظيفة تستخدم المتغيرات المركبة.

6.1. القوائم Lists

القائمة هي سلسلة من القيم (وتسمى بعناصر القائمة) يطلق عليها اسم واحد. هناك العديد من الطرق لإنشاء قائمة. وبسط الطرق هو إحاطة مجموعة من القيم بقوسين مربعين علي سبيل المثال:

المعنى	الجملة بلغة Python
قائمة فارغة	<code>empty_list = []</code>
قائمة بها العناصر 17,123	<code>numbers = [17, 123]</code>
قائمة تحتوي على بعض ايام الاسبوع	<code>days = ['Saturday' , 'Sunday' , 'Monday' , 'Tuesday']</code>
قائمة تحتوي على بعض لغات البرمجة	<code>programming_languages = ['Python' , 'C++' , 'Java']</code>
قائمة تحتوي على معلومات عن طالب	<code>student_record = [123456780, 'Mohammed Ali', 88.75]</code>
قائمة تحتوي على معلومات عن طلبة	<code>cs100_std_lst = [[123, 'Ali A.', 87.5] , [124, 'Noor Abduallah' , 85], [125, 'Fatema Ramadan' , 67.5], [126, 'Sara Mahmood' , 87.5]]</code>

<pre>>>> a = [1, 2, 3] >>> b = [4, 5, 6] >>> c = a + b >>> print(c) [1, 2, 3, 4, 5, 6]</pre>	مؤثر ال + مؤثر يدمج قائمتان ليكون قائمة جديدة
<pre>>>> a = [0] * 4 [0, 0, 0, 0] >>> [1, 2, 3] * 3 [1, 2, 3, 1, 2, 3, 1, 2, 3]</pre>	مؤثر ال * مؤثر يكرر عنصر القيمة عدد المرات المطلوبة
<pre>>>> t = ['a', 'b', 'c', 'd', 'e', 'f'] >>> n = len(t) >>> print(n) 6</pre>	الوظيفة الضمنية len() وظيفة ترجع عدد عناصر القائمة
<pre>>>> t = [1, 2, 3, 4, 5, 6] >>> s = sum(t) >>> print(s) 21</pre>	الوظيفة الضمنية sum() وظيفة ترجع مجموع عناصر القائمة
<pre>>>> t = ['a', 'b', 'c',] >>> t.append('d') >>> print(t) ['a', 'b', 'c', 'd']</pre>	الوظيفة الضمنية append() وظيفة تقوم بإضافة عنصر لقائمة
<pre>>>> a = [1, 2, 3] >>> b = [4, 5, 6] >>> a.extend(b) >>> print(a) [1, 2, 3, 4, 5, 6] >>> print(b) [4, 5, 6]</pre>	الوظيفة الضمنية extend() وظيفة تقوم بإضافة عناصر قائمة لقائمة

<pre>>>> t = ['a', 'b', 'c', 'd'] >>> t[0] 'a'</pre>	العنصر الأول ويشار له بـ $t[0]$
<pre>>>> t[1] 'b'</pre>	العنصر الثاني ويشار له بـ $t[1]$
<pre>>>> t[2] 'c'</pre>	العنصر الثالث ويشار له بـ $t[2]$
<pre>>>> t[3] 'd'</pre>	العنصر الرابع ويشار له بـ $t[3]$
<pre>Traceback (most recent call last): File "<pyshell#15>", line 1, in <module> T[4] IndexError: list index out of range</pre>	العنصر الخامس ويشار له بـ $t[4]$ غير موجود بالقائمة
<pre>>>> print(t) ['a', 'b', 'c', 'd']</pre>	عرض جميع عناصر القائمة
<pre>>>> print(t[:]) ['a', 'b', 'c', 'd']</pre>	عرض جميع عناصر القائمة
<pre>>>> t = ['a', 'b', 'c', 'd', 'e', 'f'] >>> t[1:3] = ['x', 'y'] >>> print(t) ['a', 'x', 'y', 'd', 'e', 'f']</pre>	تعديل قيم بقائمة. أي استبدال القيمة بالعنوان $t[1]$ بالقيمة 'x' والقيمة بالعنوان $t[2]$ بالقيمة 'y' ملاحظة: $t[i:j]$ تعني $t[j-1], t[j-2], \dots, t[i+1],$ $t[i]$
<pre>>>> t = ['d', 'c', 'e', 'b', 'a'] >>> t.sort() >>> print(t) ['a', 'b', 'c', 'd', 'e']</pre>	الوظيفة الضمنية sort() ترتب عناصر القائمة
<pre>>>> t = ['a', 'b', 'c', 'd', 'e', 'f'] >>> print(t[1:3]) ['b', 'c'] >>> print(t[:4])</pre>	عرض مقاطع من قائمة

<pre>['a', 'b', 'c', 'd'] >>> print(t[3:]) ['d', 'e', 'f']</pre>	
<pre>>>> t = ['a', 'b', 'c'] >>> t.remove('b') >>> print(t) ['a', 'c']</pre>	remove() الوظيفة الضمنية تقوم بإلغاء عنصر من قائمة
<pre>>>> t = ['a', 'b', 'c'] >>> del t [1] >>> print (t) ['a', 'c']</pre>	del() الوظيفة الضمنية تقوم بإلغاء عنصر من القائمة بمعرفة موقعه
<pre>>>> t = ['a', 'b', 'c', 'd', 'e', 'f'] >>> del t[1:5] >>> print(t) ['a', 'f']</pre>	الغاء مقطع من قائمة
<pre>>>> t = ['a', 'b', 'c'] >>> x = t.pop(1) >>> print (t) ['a','c'] >>> print (x) b</pre>	pop() الوظيفة الضمنية تقوم بإلغاء عنصر بموقع معين من قائمة وإسناده لمتغير
<pre>>>> t = ['a', 'b', 'c'] >>> item = 'a' >>> in_list = item in t >>> in_list True >>> item = 'e' >>> in_list = item in t >>> in_list False</pre>	in المؤثر يرجع القيمة True إذا كان العنصر في القائمة ويرجع القيمة False إذا غير موجود بالقائمة
<pre>>>> t.index('a') >>> 0</pre>	index() الوظيفة الضمنية ترجع موقع العنصر في قائمة
<pre>>>> t[3, ['a','m'], [5,[4, ['y',9]], 88],</pre>	الوصول إلى عنصر في قائمة داخل قائمة


```
90]
>>>t[2][1]
>>>[5,[4, ['y',9]]
>>> t[2][1][1][0]
>>>9
```

مثال 1: لنقم بإنشاء مجموعة من الوظائف للتعامل مع القوائم:

- وظيفة لعرض محتوى قائمة ولنختار لها الاسم `display_items`.

```
def display_item(list):
    if (len(list) > 0):
        i=0
        while(i < len(list)):
            print(list[i])
            i = i + 1
        else:
            print ('List is empty')
```

- وظيفة لتبديل عنصر في قائمة ولنختار لها الاسم `chnage_items`. حيث ستقوم الوظيفة بتغيير العنصر القديم `olditem` بالعنصر الجديد `newitem` بعد التأكد من وجود العنصر القديم في القائمة وإن لم يوجد فإنه تعرض الرسالة التالية:
is not in the list

```
def change_item(list, olditem, newitem):
    if olditem in list :
        item_index =list.index(olditem)
        list[item_index] = newitem
    else:
        print(olditem, ' is not in the list')
```

مثال 2: البرنامج التالي يقوم بترتيب عدد n من الأرقام ، مستخدماً بعض الوظائف السابقة. في هذا البرنامج تم إضافة بعض جمل الطباعة (الجمل المضللة) لغرض تنقيح البرنامج من الأخطاء، وبذلك لو نفذنا البرنامج فإننا نحصل على المخرجات الموضحة التالي. طريقة عمل البرنامج كالتالي: يقوم البرنامج بقراءة القائمة ثم يتم تحديد أصغر قيمة في القائمة ووضعها في قائمة جديدة وفي نفس الوقت يتم إلغاء هذا العنصر من القائمة الأصلية، ثم يقوم البرنامج مرة أخرى بتحديد أصغر عنصر في القائمة الأصلية ومن ثم إضافته إلى القائمة الجديدة وإلغائه من القائمة الأصلية. هذا العمل يستمر إلى أن نمر على كل عناصر القائمة الأصلية(بذلك تكون القائمة الأصلية فارغة).

```
def read_list(n):
# وظيفة لقراءة الأرقام ووضعهم في قائمة
item_list = []
i = 0
while (i < n) :
    item = input('enter the item of the list :')
    item_list.append(item)
    i = i + 1

# statements to tarce the porogram
print("the list :", end = "")
print(item_list)
return item_list

def display_items(list):
m = len(list)
if (m > 0):
    i=0
    while (i < m):
        print(list[i], end = ' ')
        i = i + 1
    else:
        print ('List is empty')

def min_list(list):
# وظيفة لإيجاد أصغر رقم في القائمة
number_of_items = len(list)
min_item = list[0]
i = 1
while (i < number_of_items):
    if (min_item > list[i]):
        min_item = list[i]
    i = i + 1

# statements to tarce the porogram
print("the smallest item in the list :", end="")
print(min_item)
return min_item

def order_list(list,n):
# وظيفه لترتيب القائمة
i = 0
list_ord = []
while (i < n):
    min_it = min_list(list)
    list_ord.append( min_it)
    i = i +1

# statements to tarce the porogram
print("the parially ordered list :", end="")
print(list_ord)
list.remove( min_it)

# statements to tarece the porogram
print("the original list :", end="")
print(list_ord)

return list_ord

def main():
البرنامج الرئيس الذي يستدعي بقية الوظائف للقيام
بالمهمة
num_of_items = int(input('enter the size of the
list :'))
sct_list = read_list(num_of_items)
ord_list = order_list(sct_list,num_of_items)
print("the sorted list :", end= "")
display_items(ord_list)
```

```
enter the size of the list :3
enter the item of the list :2
enter the item of the list :1
enter the item of the list :5
```

المخرجات التي تعرضها جمل print

```
the list :['2', '1', '5']
the smallest item in the list :1
the parially ordered list :['1']
the original list :['2', '5']
the smallest item in the list :2
the parially ordered list :['1', '2']
the original list :['5']
the smallest item in the list :5
the parially ordered list :['1', '2', '5']
the original list :[]
the sorted list : ['1', '2', '5']
```

مثال 3: يمكننا أن نعيد كتابة البرنامج السابقة باستخدام الوظيفة `sort()` الموجودة ضمناً في لغة `python`. نلاحظ أن الوظيفة `simple_sort` حلت مكان الوظيفتان `min_list` و `order_lits`

```
def read_list(n):
# وظيفة لقراءة الأرقام ووضعهم في قائمة
    item_list = []
    i = 0
    while (i < n) :
        item = input('enter the item of the list :')
        item_list = add_item(item_list, item)
        i = i + 1
    return item_list

def simple_sort(list):
    list.sort()
    return list

def display_items(list):
    m = len(list)
    if (m > 0):
        i=0
        while (i < m):
            print(list[i], end = ' ')
            i = i + 1
    else:
        print ('List is empty')

def main():
# البرنامج الرئيس الذي يستدعي بقية الوظائف
# لقيام بالمهمة
    num_of_items = int(input('enter the size of
the list :'))
    sct_list = read_list(num_of_items)
    ord_list = simple_sort(sct_list)
    display_items(ord_list)
```

مثال 4: المثال التالي يبين بعض العمليات على القوائم

```
def main():
    student_rec= []
    print ('\nstudent_rec =', student_rec)
    birth_date= int(input('Birth date YYYY-MM-DD: '))
    student_rec.append(birth_date)
    print ('\nstudent_rec =', student_rec)
    avg = float(input('High school average:'))
    student_rec.append(avg)
    print ('\nstudent_rec =', student_rec)
    print ('\nLength of student_rec =', len(student_rec))
    print ('\n')
    i = 0
    while (i < len(student_rec)):
        print ('student_rec[' , i , ']' =', student_rec [i]))
        i = i + 1
```

مثال 5: المثال التالي يبين برنامج يوضح استدعاء بعض الوظائف السابقة على القوائم.

لنقوم اولاً بإنشاء وظيفة لعرض لائحة تحتوي علي خيارات وتسمح فقط بإختيار خيار واحد فقط ولنطلق عليها الاسم menu

```
def menu():
    print('-----')
    print('1. Print the list')
    print('2. Add a name to a list')
    print('3. Remove a name form then list')
    print('4. Change an item in the list')
    print('5. Sort the items in the list')
    print('0. Exit program')
    choice = read_integer_val('Enter your choice [0 - 5]: ', 0, 5)
    return choice
```

فيكون البرنامج كالاتي:

```
def main():
    list = []
    choice = menu()
    while (choice != 0):
        if (choice == 1):
            list_item(list)
        elif (choice == 2):
            item = input('Type in a name to add: ')
            add_item(list, item)
        elif (choice == 3):
            item = input('Type in the name you like to remove: ')
            remove_item(list, item)
        elif (choice == 4):
            olditem = input('Type in the name you like to change: ')
            if (olditem in list):
                newitem = input('Type in the new name: ')
                change_item(list, olditem, newitem)
        elif (choice == 5):
            list.sort()
        choice = menu()
    print('\nEnd of program')
```

مثال 6: اكتب برنامج لتكوين قائمة تحتوي علي عدد من السجلات تخص طلبة لمقرر CS100 بحيث كل سجل يتكون من (رقم القيد،

اسم الطالب، الدرجة النهائية). البرنامج يجب أن يكون قادر علي:

- عرض قائمة بمعلومات الطلبة
- إضافة طالب جديد للقائمة.
- إلغاء سجل طالب من القائمة.
- ترتيب القائمة حسب رقم القيد.

ملاحظة: شكل القائمة علي سبيل المثال كالتالي وعدد السجلات ليس محدد مسبقا:

[[209111001, 'student1', 75], [209111215, 'student215', 60],

...,

[209111012, 'student12', 40]]

الخطوة الأولى:

إستخدام عدد من الوظائف والتي تم تعريفها سابقا لإستغلالها بهذا البرنامج.

- الوظيفة `read_integer_val` . لإدخال رقم القيد.
- الوظيفة `read_float_val` . لإدخال الدرجة.
- تعريف الوظيفة `read_string_val` . الوظيفة تستقبل 3 قيم (`prompt` - نص يساعد علي تحديد ما المطلوب إدخاله من لوحة المفاتيح، `minval` - أصغر قيمة يسمح بقبولها، `maxval` - أكبر قيمة يسمح بقبولها). الوظيفة تقوم بترجيع قيمة من النوع النصي. علي سبيل المثال اسم الطالب.
- الوظيفة `menu` لعرض قائمة الخيارات المتاحة.
- الوظيفة `sort_list` لترتيب القائمة حسب رقم القيد (علي يتم تعديلها أو تغييرها في حالة تحديد حقل آخر غير رقم القيد للترتيب).
- الوظيفة `creat_rec` لتكوين سجل الطالب.
- الوظيفة `add_item` لإضافة سجل للقائمة.
- الوظيفة `remove_item` لإلغاء سجل طالب من القائمة بإستخدام رقم القيد.
- الوظيفة `list_item` لعرض قائمة.
- الوظيفة `list_rec` لعرض سجلات الطلبة على هيئة جدول.

```

def read_integer_val (prompt, minval, maxval):
    while (True):
        var = input (prompt)
        if (var.isdigit()):
            var = int(var)
            if (minval <= var <= maxval):
                return var
def read_string_val (prompt, minval, maxval):
    var = input(prompt)
    while (var < minval or var > maxval):
        var = input(prompt)
    return var
def menu():
    print ('-----')
    print ('1. Print student record ')
    print ('2. Add student record to a list ')
    print ('3. Remove student record form then list ')
    print ('4. Sort records in the list')
    print ('0. Exit Program')
    choice = read_integer_val ('Enter your choice [0 – 4]: ', 0, 4)
    return choice

def sort_list(list):
    list.sort()
def creat_rec():
    rec = []
    std_no = read_integer_val('Student No.: ', 100000000, 999999999)
    if (std_no == 999999999):
        return rec
    std_name = read_string_val("Student name", "ا", "بي")
    std_grade = read_integer_val('Student Grade: ', 0, 100)
    rec.append(std_no)
    rec.append(std_name)
    rec.append(std_grade)
    return rec
def add_item(list):
    item = creat_rec()
    if (len(item) > 0):
        list.append(item)
def remove_item(list):
    std_no = read_integer_val('Student No.: ', 0, 999999999):
    for rec in list :
        if (std_no in rec):
            list.remove(rec)
def list_rec(list)
    for rec in list :
        print(rec[2], rec[0], rec[1])

def main():
    list = []
    choice = menu()
    while (choice != 0):
        if (choice == 1):
            print("\n")
            list_rec(list)
        elif (choice == 2):
            print("\n")
            add_item(list)
        elif (choice == 3):
            remove_item(list)
        elif (choice == 4):
            sort_list(list)
        choice = menu()
    print ("\nEnd of program")

```

تمارين

1. أكتب ناتج الوظيفة الآتية:

<pre>def g(): x = [1,3,2,[5,6,0]] k = 0 m = 0 y = 0 while (k <= 2): y = y + x[k] * x[3][m] k = k + 1 m = m + 1 print(y)</pre>
الناتج:

2. أضف للقائمة menu() الخيارات التالية:

- حساب متوسط الدرجات
- حساب الإنحدار المعياري للدرجات
- حساب أعلى وإقل درجة

بحيث تصبح القائمة كالتالي:

```
def menu():
    print ('-----')
    print ('1. Print student record ')
    print ('2. Add student record to a list ')
    print ('3. Remove student record form then list ')
    print ('4. Sort records in the list')
    print ('5. compute average grade')
    print ('6. compute standard deviation')
    print ('7. find maximum and minimum grade')
    print ('0. Exit Program')
    choice = read_integer_val ('Enter your choice [0 – 7]: ', 0, 7)
    return choice
```

وقم بالتغيرات اللازمة بالبرنامج الرئيس لإنجاز المهام الجديدة. بالإضافة إلى كتابة تعريف لوظائف لحساب (متوسط الدرجات، الإنحدار المعياري، أعلى وأقل درجة).

6.2. تمارين:

4. صمم ونفذ على الحاسوب

- اكتب وظيفة بإسم dec2bin والتي تستقبل قيمة رقمية عشرية صحيحة وتقوم بإرجاع المكافئ بالنظام الثنائي.
- اكتب وظيفة بإسم bin2oct والتي تستقبل قيمة رقمية ثنائية صحيحة وتقوم بإرجاع المكافئ بالنظام الثماني.
- اكتب وظيفة بإسم bin2hex والتي تستقبل قيمة رقمية ثنائية صحيحة وتقوم بإرجاع المكافئ بالنظام السادس عشر.
- اكتب البرنامج الرئيس والذي يقوم بقراءة قيمة عشرية صحيحة وتحويل وطباعة المكافئ له بكل من النظام الثنائي، الثماني، والسادس عشر.